You said:

Research và tìm cho tui những video giảng, paper thuật toán chi tiết về thuật toán RMQ để tui chuẩn bị nội dung cho buổi seminar, nội dung trình bày của seminar tối thiểu những nội dung sau:
* Nội dung nên có phần trình bày (30p):
1. Đặt vấn đề, đưa ra bài toán.
2. Trình bày cơ sở thuật toán, sơ lược về mặt lý thuyết (nếu đã học rồi)
3. Ứng dụng thuật toán để giải quyết bài toán đặt ra.
4. Tối ưu hóa thuật toán và phân tích độ phức tạp của thuật toán (cả về mặt thời gian và không gian)
5. Ứng dụng thực tiễn của thuật toán hoặc các bài tập ứng dụng.
Có thể dùng skill https://github.com/Panniantong/Agent-Reach.git để tìm kiếm thông tin chính xác

Worked for 17m 44s >

You said:

Có so sánh, đánh giá benchmark.
Có trình bày mở rộng thuật toán
có phần so sánh với CTDL khác
Có ví dụ cụ thể cho từng trường hợp
ứng dụng / phân tích, so sánh với các thuật toán khác.
Và bạn hãy research thêm những nội dung này, đây là những nội dung mà các nhóm khác đã thiếu sót khi trình bày seminar

ChatGPT said:

Mình đã gom thành **bộ nghiên cứu RMQ có video, paper, chứng minh, ví dụ, so sánh CTDL và benchmark chạy thật**. Với các phần bạn bổ sung, mình đề xuất trình bày **40 phút**, kèm phương án rút xuống 30 phút.

- rmq_seminar_research_vi.md: toàn bộ ghi chú tiếng Việt, đề cương, ví dụ và câu hỏi phản biện.
- rmq_seminar_materials.zip: thêm mã C++, CSV kết quả, biểu đồ và hướng dẫn chạy lại benchmark.

Hướng trình bày nên chọn

Tên phù hợp: **"Range Minimum Query: từ duyệt tuyến tính đến truy vấn O(1), và khi nào nên dùng cấu trúc nào?"**

Lấy **Sparse Table làm thuật toán triển khai chính**, sau đó mở rộng sang Cartesian tree/Fischer–Heun. Segment Tree, chia khối, Fenwick và deque dùng để phân tích điều kiện lựa chọn. Cách này vừa có thuật toán cụ thể, vừa có chiều sâu nghiên cứu.

1. Video nên xem

Chuỗi của Sebastian Wild có trang học phần, transcript và chapter để xem lại. Các thời lượng dưới đây thuộc bản Spring 2022. www2.wild-inter...

Thứ tự	Video	Thời lượng	Nội dung lấy cho seminar
1	RMQ introduction	11:50	Định nghĩa, mô hình bài toán
2	Primitive RMQ & Sparse Tables	16:09	Cách cơ bản, xây bảng, truy vấn
3	Cartesian trees	28:24	Quan hệ RMQ–LCA, hình dạng cây
4	Exhaustive Tabulation	21:56	Chia khối, bảng tra, tiền xử lý tuyến tính
5	LCE using RMQ	12:43	Ứng dụng vào xử lý chuỗi

Video 2 nên xem kỹ đoạn **7:26 và 9:54**; video 4 xem lại phần từ **13:48** khi chuẩn bị phần tối ưu. Lưu ý video Cartesian tree chuyển sang minh họa **range maximum** ở một số đoạn; khi làm slide cần đổi chiều so sánh nhất quán. www2.wild-inter... +1

Để đọc sâu hơn: MIT 6.851 — Lecture 15: Static Trees, Erik Demaine, có video và ghi chú về LCA, RMQ, lookup tables. classes.csail.mit...

2. Paper nên đọc, theo mục đích

Paper	Phần đáng đọc	Dùng cho nội dung nào
Bender & Farach-Colton, 2000 — The LCA Problem Revisited	§2–5	RMQ ↔ LCA, Sparse Table, ±1 RMQ, tiền xử lý tuyến tính
Fischer & Heun, 2006 — Theoretical and Practical Improvements on the RMQ-Problem	§3–5	RMQ tổng quát trực tiếp, ứng dụng LCE, thực nghiệm
Fischer & Heun, 2011 — Space-Efficient Preprocessing Schemes...	Introduction, §5	Mở rộng về bộ nhớ $2n + o(n)$ bit

Ba paper này tạo thành trục lý thuyết từ thuật toán dễ giải thích đến cấu trúc tối ưu và nén bộ nhớ. www3.cs.stonyb... +2

Để bổ sung đúng các điểm các nhóm khác thiếu:

Paper bổ sung	Giá trị cho seminar
Dima & Ceterchi, 2015 — Efficient RMQ using Binary Indexed Trees	Biến thể hai BIT, so sánh thực nghiệm với Segment Tree
Baumstark et al., 2017 — Practical Range Minimum Queries Revisited	Độ dài query, dữ liệu gần tăng/giảm, cache, các implementation succinct
Alzamel et al., preprint 2017 — How to Answer a Small Batch of RMQs or LCA Queries in Practice	Mở rộng offline khi biết trước ít truy vấn, $q \ll n$

Đây là các nguồn nghiên cứu trực tiếp, có thuật toán hoặc thực nghiệm để phân tích, thay vì chỉ lấy bảng Big-O từ tutorial. ioinformatics.org +2

3. So sánh CTDL cần có trong bài

Bảng tính bộ nhớ phụ ngoài mảng đầu vào; đơn vị là word.

Cách làm	Tiền xử lý	Một query	Cập nhật điểm	Bộ nhớ phụ
Duyệt trực tiếp	$O(1)$	$O(r-l+1)$	$O(1)$	$O(1)$
Chia khối $\sqrt{n}$	$O(n)$	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(\sqrt{n})$
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Sparse Table	$O(n \log n)$	$O(1)$	Dành cho mảng tĩnh	$O(n \log n)$
Fischer–Heun	$O(n)$	$O(1)$	Dành cho mảng tĩnh	$O(n)$

Các kết quả này được trình bày trong tài liệu Stanford, tài liệu Segment Tree và các paper nền tảng. web.stanford.edu +2

Có hai so sánh cần nói chính xác:

- **Fenwick prefix-min đơn giản** có hạn chế về đoạn truy vấn và cập nhật; nhưng **biến thể hai BIT** có thể giải RMQ động. Không nên kết luận "Fenwick không giải được RMQ" một cách tuyệt đối. Algorithms for ... +1
- **Deque** phù hợp khi hỏi các cửa sổ trượt tuần tự; có thể giải toàn bộ trong $O(n)$, nhưng không tạo chỉ mục trả lời mọi đoạn tùy ý về sau. Algorithms for ...

4. Benchmark đã chạy và những nhận xét đáng đưa lên slide

Mình chạy bốn implementation C++: duyệt, chia khối, Segment Tree lặp và Sparse Table. Môi trường AMD EPYC 9V74 ảo hóa, GCC 13.3, `-O3 -march=native`, một thread, **5 lần mỗi cấu hình**.

Với $n = 200.000,\ q = 20.000$, hai đầu mút truy vấn ngẫu nhiên:

Thuật toán	Build, ms	20.000 query, ms	Tổng, ms	Bộ nhớ phụ, MiB
Duyệt trực tiếp	≈0	59,885	59,885	0
Chia khối	0,533	1,614	**2,217**	0,0017
Segment Tree	0,642	2,194	2,888	2,0000
Sparse Table	5,184	**0,081**	5,265	12,7330

Các cột là median; tổng được tính từ từng lượt build + query. Bộ nhớ là payload các mảng phụ, chưa gồm mảng đầu vào và overhead.

Những kết luận có căn cứ từ phép đo này:

1. **Sparse Table thắng ở query, nhưng chưa thắng tổng thời gian** tại 20.000 query vì chi phí build.
2. Khi mọi đoạn chỉ dài **16 phần tử**, duyệt trực tiếp mất khoảng **0,065 ms**, Sparse Table khoảng **0,131 ms** cho cả lô. Đoạn ngắn làm thay đổi lựa chọn.
3. Ở workload động gồm **50% cập nhật**, Segment Tree mất **1,301 ms**, chia khối **1,705 ms**; tại **90% cập nhật**, lần lượt **0,597 ms** và **1,990 ms**.
4. Không thể dùng riêng Big-O để dự đoán thứ hạng trên một kích thước cụ thể.

Bộ đo có **220 lượt chạy**, kiểm tra **915.200 khoảng nhỏ**, đối chiếu toàn bộ đáp án các case static trước khi đo. CSV có cả độ dao động IQR. Đây là microbenchmark trong môi trường chia sẻ; chưa tự chạy Fischer–Heun hoặc succinct RMQ.

Để đối chiếu với nghiên cứu công bố, paper hai BIT năm 2015 báo cáo:

Thao tác	Segment Tree	Hai BIT
10 triệu cập nhật	1,274 s	0,672 s
10 triệu truy vấn	2,397 s	0,551 s

Đây là số liệu **của tác giả trên máy và implementation của họ**, không gộp với số đo mình vừa chạy để lập bảng xếp hạng chung. ioinformatics.org

5. Ví dụ và hướng mở rộng nên trình bày

Dùng xuyên suốt:

$$A = [5, 2, 6, 3, 1, 7, 4, 8]$$

Chỉ số từ 1; các thay đổi dưới đây đều bắt đầu từ mảng gốc.

Trường hợp	Ví dụ cụ thể	Thuật toán/phân tích
RMQ tĩnh	$[2,7]$ trả giá trị **1**, vị trí **5**	Sparse Table dùng hai đoạn $[2,5]$, $[4,7]$
Cập nhật điểm	Gán $A[5]=9$, min $[3,7]$ thành **3**	Segment Tree
Cập nhật đoạn	Cộng 3 vào $[2,5]$, min $[2,7]$ thành **4**	Lazy Segment Tree
Cửa sổ dài 3	Kết quả **2,2,1,1,1,4**	Monotonic deque
Bài toán chuỗi	Hai hậu tố của “banana” ở vị trí 2 và 4 có tiền tố chung dài **3**	Suffix array + LCP + RMQ
Ít query biết trước	Co các vùng giữa đầu mút thành min của vùng	Offline RMQ; giữ ánh xạ về chỉ số gốc

Các hướng động, cửa sổ trượt, LCE và offline đều có nguồn giải thích trong tài liệu kèm theo.

Algorithms for ... +3

Phần tối ưu nên giải thích được ý tưởng **Fischer–Heun**: chia khối, xây Sparse Table trên cực tiểu các khối, rồi dùng chung bảng nội khối cho những khối có cùng hình dạng Cartesian tree. Nhờ vậy đạt $O(n)$ tiền xử lý và bộ nhớ, $O(1)$ query. web.stanford.edu

Các mở rộng sâu hơn đã có ví dụ trong bản ghi chú: **RMQ 2D**, **Disjoint Sparse Table** và **succinct RMQ**. Riêng $2n+o(n)$ bit là cấu trúc trả **vị trí cực tiểu**, không chứa toàn bộ các giá trị tùy ý của mảng. ae.iti.kit.edu

6. Ứng dụng phải có đối chiếu lời giải

Với **LCA**, nên so sánh Euler tour + RMQ với binary lifting: bản Sparse Table cho query $O(1)$, còn binary lifting thông thường cho $O(\log n)$ và thuận tiện khi cần thêm truy vấn tổ tiên thứ k.

www3.cs.stonyb... +1

Một phản ví dụ đáng đưa vào phần hỏi đáp: **RMQ trên độ sâu Euler giải LCA; lấy RMQ trên trọng số Euler không tự giải min trọng số đường đi**. Đoạn Euler có thể chứa nhánh ngoài đường đi. Nếu có cập nhật trọng số, HLD + Segment Tree là hướng phù hợp; bản cơ bản có path query $O(\log^2 n)$. Algorithms for ...

Bài tập nên giao:

- CSES Static RMQ: Sparse Table.
- CSES Dynamic RMQ: Segment Tree.
- CSES Company Queries II: LCA, so sánh hai cách giải. Static Range Mi... +2

Khung trình bày 40 phút đề xuất

Thời gian	Nội dung
0–3 phút	Đặt vấn đề, static/dynamic, min/argmin
3–6 phút	Duyệt trực tiếp và lưu mọi đoạn
6–14 phút	Sparse Table: xây, chứng minh, chạy ví dụ
14–20 phút	So sánh CTDL theo điều kiện bài toán
20–26 phút	Benchmark và đánh giá kết quả
26–33 phút	Cartesian tree, Fischer–Heun, một hướng mở rộng
33–40 phút	Ứng dụng, đối chiếu lời giải và bài tập

Nếu chỉ có 30 phút, nên rút phần thuật toán nền và chuyển chi tiết 2D/offline/succinct sang slide dự phòng; **giữ phần benchmark, so sánh CTDL và phản ví dụ**, vì đây là các phần giúp seminar có phân tích thay vì chỉ mô tả thuật toán.

You said:

1 selection

giờ bạn trình bày giảng cho tui nội dung video này đi

Worked for 1m 58s

ChatGPT said:

Video này chủ yếu giúp bạn hiểu **RMQ hỏi điều gì, dữ liệu được cho theo cách nào và phải đánh giá thuật toán bằng những chi phí nào**. Mình giảng lại bằng ví dụ riêng; dùng **chỉ số bắt đầu từ 0**, giống trong video. www2.wild-inter...

Trước hết, RMQ là bài toán gì?

Giả sử mình đưa bạn mảng:

Chỉ số i	0	1	2	3	4	5	6	7
$A[i]$	7	4	9	4	6	2	2	8

Sau đó mình hỏi:

> Trong đoạn từ vị trí 1 đến vị trí 4, phần tử nhỏ nhất nằm ở đâu?

Đoạn đang xét gồm:

$$A[1..4] = [4, 9, 4, 6]$$

Giá trị nhỏ nhất là **4**, xuất hiện ở vị trí **1 và 3** của mảng gốc. Chọn vị trí bên trái, đáp án là:

$$\boxed{\text{RMQ}_A(1, 4) = 1}$$

Trong video, RMQ trả về **chỉ số của phần tử nhỏ nhất**; nếu bằng nhau thì lấy vị trí bên trái. Đây là quy ước cần giữ xuyên suốt bài. www2.wild-inter...

Bạn hãy phân biệt:

Câu hỏi	Đáp án
Giá trị nhỏ nhất trong $[1, 4]$ là bao nhiêu?	4
RMQ của $[1, 4]$ là gì?	1
Lấy giá trị từ kết quả RMQ thế nào?	$A[\text{RMQ}(1, 4)] = A[1] = 4$

Biết **vị trí** thì lấy được giá trị. Nhưng chỉ biết giá trị 4 thì chưa biết đó là vị trí 1 hay 3.

Đoạn truy vấn phải liên tiếp và bao gồm cả hai đầu

Ký hiệu $[l, r]$ nghĩa là xét:

$$A[l], A[l + 1], \ldots, A[r]$$

Số phần tử của đoạn là:

$$L = r - l + 1$$

Ví dụ $[3, 6]$ chứa bốn phần tử:

$$A[3..6] = [4, 6, 2, 2]$$

Min là 2, xuất hiện đầu tiên tại **chỉ số 5 của mảng gốc**, nên:

$$\text{RMQ}(3, 6) = 5$$

Đừng trả về 2 vì nó là phần tử thứ ba trong đoạn con khi đếm từ 0. Kết quả cần là **chỉ số trong mảng gốc**.

Thử thêm vài trường hợp:

Truy vấn	Đoạn được xét	Giá trị nhỏ nhất	Kết quả RMQ
$(1, 4)$	$[4, 9, 4, 6]$	4	1
$(3, 6)$	$[4, 6, 2, 2]$	2	5
$(6, 7)$	$[2, 8]$	2	6
$(2, 2)$	$[9]$	9	2

Trường hợp cuối giúp nhớ rằng **đoạn chỉ có một phần tử thì RMQ trả ngay chỉ số của phần tử đó**, dù giá trị của nó lớn hay nhỏ.

Có thể viết bài toán bằng ký hiệu:

$$\mathrm{RMQ}_A(l, r) = \arg\min_{l \le k \le r} A[k]$$

Ở đây, `min` cho **giá trị nhỏ nhất**, còn `argmin` cho **vị trí đạt giá trị nhỏ nhất**. Định nghĩa trả về chỉ số cũng được dùng trong paper nền tảng về RMQ–LCA. www3.cs.stonyb...

Vậy cứ quét đoạn đó là xong, sao phải nghiên cứu?

Đúng, nếu chỉ có một câu hỏi thì cách giải rất đơn giản:

```
int rmq(int l, int r) {
    int pos = l;

    for (int i = l + 1; i <= r; ++i) {
        if (A[i] < A[pos]) {
            pos = i;
        }
    }

    return pos;
}
```

Để ý điều kiện là `<`. Nếu gặp một phần tử **bằng** min hiện tại, ta giữ vị trí cũ, tức giữ vị trí bên trái.

Với đoạn dài L, cách này cần $L - 1$ phép so sánh:

$$Q = O(L)$$

Trong trường hợp hỏi cả mảng, $L = n$, nên tệ nhất:

$$Q(n) = O(n)$$

Điểm khiến RMQ đáng nghiên cứu là: **cùng một mảng nhưng có rất nhiều truy vấn**. Quét lại từng đoạn sẽ lặp lại công việc. Đây cũng là cách bài giảng Stanford đặt vấn đề cho RMQ.

web.stanford.edu

Ví dụ tự tính: mảng có 100.000 phần tử, rồi có 100.000 truy vấn, mỗi truy vấn hỏi cả mảng. Cách quét trực tiếp cần gần:

$$100.000 \times 100.000 = 10^{10}$$

phép so sánh. Vì vậy, mục tiêu chuyển thành: **làm một số công việc trước để những lần hỏi sau nhanh hơn**.

Đó chính là vai trò của tiền xử lý

Mô hình trong video gồm hai giai đoạn:

1. **Tiền xử lý:** nhận toàn bộ mảng, phân tích nó và lưu thông tin hỗ trợ.
2. **Truy vấn:** sau đó mới nhận từng cặp (l, r), dùng thông tin đã lưu để trả lời.

Có hai điều kiện quan trọng: **mảng không đổi** và **chưa biết trước những đoạn sẽ được hỏi**.

www2.wild-inter...

Chẳng hạn, bạn nhận mảng trên và xây cấu trúc dữ liệu. Sau đó mình có thể hỏi lần lượt:

$$(1, 4), \quad (3, 6), \quad (0, 7), \quad (2, 2)$$

Bạn phải sẵn sàng trả lời bất kỳ đoạn hợp lệ nào.

“Không biết trước truy vấn” không có nghĩa phải trả lời trước khi nhận đủ mảng. **Mảng đã được cung cấp đầy đủ từ đầu**; chỉ có các câu hỏi xuất hiện sau.

Điều kiện mảng tĩnh cũng có ý nghĩa thực sự. Ban đầu:

$$\mathrm{RMQ}(3, 6) = 5$$

Nếu sửa $A[5]$ từ 2 thành 10 thì đoạn đó thành:

$$[4, 6, 10, 2]$$

Đáp án mới là 6. Những thông tin đã tính trước có thể không còn đúng. Khi cho phép thay đổi như vậy, ta chuyển sang **RMQ động**, một mô hình khác.

Phải đánh giá hai loại thời gian riêng biệt

Giảng viên dùng một cặp để mô tả hiệu quả của lời giải. Mình viết:

$$\boxed{\langle P(n), Q(n) \rangle}$$

Trong đó:

- $P(n)$: thời gian tiền xử lý.
- $Q(n)$: thời gian trả lời **một** truy vấn. www2.wild-inter...

Để hiểu sự đánh đổi này, xét hai cách cực đoan sau. Đây là ví dụ mình bổ sung để giải thích ký hiệu, chưa phải phần triển khai thuật toán của video.

Cách làm	Tiền xử lý	Một truy vấn	Bộ nhớ phụ
Không tính trước, khi hỏi mới quét	$O(1)$	$O(n)$	$O(1)$
Tính và lưu đáp án cho mọi đoạn	$O(n^2)$ nếu xây phù hợp	$O(1)$	$O(n^2)$

Cách thứ hai có thể xây bằng quy hoạch động để tránh quét lại từng đoạn riêng. www3.cs.stonyb...

Với mảng 8 phần tử, số đoạn hợp lệ là:

$$8 + 7 + \cdots + 1 = \frac{8 \times 9}{2} = 36$$

Nếu đã lưu đủ 36 đáp án, mỗi lần hỏi chỉ cần tra bảng. Truy vấn rất nhanh, nhưng phải trả chi phí tính trước và lưu trữ.

Trong bảng trên, $O(1)$ tiền xử lý của cách quét nghĩa là **không xây thêm cấu trúc**; việc đọc mảng đầu vào vẫn tốn $O(n)$.

Nếu có m truy vấn, phần thời gian xử lý được đánh giá theo:

$$\boxed{T(n, m) \approx P(n) + mQ(n)}$$

Công thức này giải thích vì sao không nên chỉ nhìn vào tốc độ một query. Một cấu trúc xây rất tốn thời gian có thể chưa đáng dùng nếu chỉ hỏi một vài lần.

Video kết thúc bằng câu hỏi nghiên cứu nào?

Sau khi đặt ra hai chi phí trên, giảng viên cho người học dự đoán những cặp độ phức tạp có thể đạt được. Video chưa chứng minh thuật toán tối ưu ở bước này. www2.wild-inter...

Câu hỏi bạn cần mang sang bài tiếp theo là:

> Có thể lưu ít thông tin hơn bảng mọi đoạn, nhưng vẫn trả lời nhanh hơn quét từng đoạn bằng cách nào?

Với mảng đang học, bạn có thể tự kiểm tra ba truy vấn sau trước khi xem đáp án:

Truy vấn	Đáp án	Lý do
$\mathrm{RMQ}(2, 4)$	**3**	Đoạn $[9, 4, 6]$, min ở chỉ số 3
$\mathrm{RMQ}(0, 7)$	**5**	Hai số 2 ở vị trí 5 và 6, chọn bên trái
$\mathrm{RMQ}(6, 6)$	**6**	Đoạn chỉ chứa $A[6]$

You said:

1 selection

giờ bạn trình bày giảng cho tui nội dung video này đi

Worked for 2m 54s >

ChatGPT said:

Bài này trả lời câu hỏi: **Có thể tính trước những gì để mỗi lần hỏi RMQ không phải quét lại cả đoạn?**

Giảng viên đi từ hai cách cơ bản đến **Sparse Table**, đạt $O(n \log n)$ tiền xử lý và bộ nhớ, $O(1)$ cho mỗi truy vấn. Mình sẽ giải thích bằng mảng ở bài trước, vẫn dùng **chỉ số từ 0 và trả vị trí cực tiểu bên trái**. www2.wild-inter...

Chỉ số i	0	1	2	3	4	5	6	7
$A[i]$	7	4	9	4	6	2	2	8

Ví dụ:

$$\mathrm{RMQ}(1, 6) = 5$$

vì trong $[4, 9, 4, 6, 2, 2]$, số nhỏ nhất là 2, xuất hiện đầu tiên ở vị trí 5.

1. Cách cơ bản thứ nhất: khi được hỏi mới quét

Với truy vấn $[l, r]$, duyệt từ trái sang phải, giữ vị trí nhỏ nhất đã gặp.

```
int pos = l;
for (int i = l + 1; i <= r; ++i)
    if (A[i] < A[pos])
        pos = i;
```

Một đoạn dài $L = r - l + 1$ cần $L - 1$ phép so sánh. Tệ nhất hỏi cả mảng nên mất $O(n)$.

Cách này có cặp chi phí:

$$\boxed{\langle O(1), O(n) \rangle}$$

Nghĩa là không xây cấu trúc hỗ trợ, nhưng mỗi query có thể tốn tuyến tính. Chi phí đọc mảng đầu vào vẫn là $O(n)$.

2. Cách cơ bản thứ hai: lưu đáp án cho mọi đoạn

Ta tạo bảng:

$$M[l][r] = \text{RMQ}(l, r)$$

Khi có câu hỏi, trả ngay `M[l][r]` .

Mảng n phần tử có:

$$\frac{n(n+1)}{2}$$

đoạn hợp lệ, nên cần $O(n^2)$ ô lưu trữ.

Nhưng **xây bảng thế nào** mới là điểm cần chú ý.

Nếu với mỗi ô `M[l][r]` lại quét đoạn $[l, r]$, tổng thời gian sẽ là $O(n^3)$. Ta có thể giảm xuống $O(n^2)$ bằng cách tận dụng đáp án của đoạn ngắn hơn. Đây là bước quy hoạch động trong bài giảng. www3.cs.stonyb...

Giả sử đã biết cực tiểu của:

$$A[l..r-1]$$

Khi thêm $A[r]$, cực tiểu mới chỉ có hai khả năng:

- Vẫn là cực tiểu cũ.
- Chính là phần tử mới $A[r]$.

Để viết gọn, gọi `better(p,q)` là phép chọn **chỉ số tốt hơn**:

```
int better(int p, int q) {
    if (A[p] != A[q])
        return A[p] < A[q] ? p : q;
    return min(p, q); // bằng giá trị thì chọn bên trái
}
```

Khi đó:

$$M[l][l] = l$$

$$\boxed{M[l][r] = \text{better}(M[l][r-1], r)}$$

Ví dụ cố định $l = 1$:

Đoạn	Đáp án đã lưu	Diễn giải
$[1,1]$	1	Chỉ có số 4
$[1,2]$	1	Thêm 9, giữ số 4
$[1,3]$	1	Thêm 4, giữ vị trí bên trái
$[1,4]$	1	Thêm 6, vẫn giữ vị trí 1
$[1,5]$	5	Thêm 2, cập nhật vị trí 5
$[1,6]$	5	Thêm 2, giữ vị trí 5

Mỗi ô chỉ cần một lần ghép hai ứng viên, nên:

$$\boxed{\text{Xây } O(n^2), \quad \text{query } O(1), \quad \text{bộ nhớ } O(n^2)}$$

3. Sparse Table: chỉ lưu một số độ dài đặc biệt

Bảng mọi đoạn trả lời nhanh nhưng quá lớn. Ý tưởng tiếp theo là:

> Chỉ lưu đáp án cho các đoạn có độ dài $1, 2, 4, 8, \ldots$.

Đó là các độ dài bằng lũy thừa của 2.

Ta định nghĩa:

$$\boxed{st[k][i] = \text{RMQ}\,(i,\ i + 2^k - 1)}$$

Bạn cần đọc hai chỉ số đúng ý nghĩa:

- k: **số mũ của độ dài**.
- i: **vị trí bắt đầu**.
- Giá trị trong ô: **chỉ số cực tiểu**.

Ví dụ:

Ô	Đoạn mà ô đại diện	Đáp án
$st[0][3]$	$[3, 3]$, dài 1	3
$st[1][3]$	$[3, 4]$, dài 2	3
$st[2][3]$	$[3, 6]$, dài 4	5
$st[3][0]$	$[0, 7]$, dài 8	5

Chẳng hạn:

$$st[2][3] = 5$$

có nghĩa: **đoạn bắt đầu tại 3, dài 4, có cực tiểu bên trái tại vị trí 5**.

Vì đang lưu vị trí, hàng đầu tiên phải là:

$$st[0][i] = i$$

chứ không phải $A[i]$.

4. Xây Sparse Table bằng cách ghép hai nửa

Một đoạn dài 8 tách được thành hai đoạn dài 4. Một đoạn dài 4 tách được thành hai đoạn dài 2.

Vậy muốn tính cực tiểu của đoạn lớn, ta chỉ cần so sánh cực tiểu của hai nửa:

$$\boxed{st[k][i] = \text{better}\,(st[k-1][i], st[k-1][i+2^{k-1}])}$$

Hai nửa có độ dài 2^{k-1}, nên nửa thứ hai bắt đầu cách nửa đầu đúng 2^{k-1} vị trí. Cách xây theo độ dài tăng dần bảo đảm cả hai ô nhỏ hơn đã tồn tại. Algorithms for ...

Xét cụ thể:

$$st[2][1] = \text{RMQ}(1, 4)$$

Đoạn $[1, 4]$ tách thành:

- $[1, 2]$: cực tiểu ở vị trí 1, giá trị 4.
- $[3, 4]$: cực tiểu ở vị trí 3, giá trị 4.

Hai giá trị bằng nhau, chọn vị trí bên trái:

$$st[2][1] = \text{better}(1, 3) = 1$$

Tiếp theo:

$$st[2][3] = \text{RMQ}(3, 6)$$

Hai nửa là:

- $[3, 4]$: vị trí 3, giá trị 4.
- $[5, 6]$: vị trí 5, giá trị 2.

Do đó:

$$st[2][3] = \text{better}(3, 5) = 5$$

Toàn bộ bảng của ví dụ là như sau. **Các số trong bảng đều là chỉ số**, dấu "—" nghĩa là đoạn vượt quá cuối mảng.

k / độ dài	$i = 0$	1	2	3	4	5	6	7
0 / 1	0	1	2	3	4	5	6	7
1 / 2	1	1	3	3	5	5	6	—
2 / 4	1	1	5	5	5	—	—	—
3 / 8	5	—	—	—	—	—	—	—

5. Đoạn dài 6 chưa được lưu, vậy truy vấn thế nào?

Đây là ý tưởng quan trọng nhất của video.

Muốn trả lời $[l, r]$, đặt:

$$L = r - l + 1, \qquad k = \lfloor \log_2 L \rfloor, \qquad b = 2^k$$

Nói bằng lời: **chọn độ dài lũy thừa của 2 lớn nhất không vượt quá độ dài đoạn hỏi**.

Sau đó lấy hai khối cùng dài b:

- Khối trái bắt đầu tại l: $[l, l + b - 1]$.
- Khối phải kết thúc tại r: $[r - b + 1, r]$.

Cả hai đều đã có đáp án trong Sparse Table.

Bạn có thể đổi l, r dưới đây để nhìn cách hai khối phủ đoạn; vị trí cực tiểu của mỗi khối được gạch chân.

6. Vì sao hai khối luôn phủ kín đoạn hỏi?

Từ cách chọn k, ta có:

$$2^k \le L < 2^{k+1}$$

Hay:

$$b \le L < 2b$$

Điều này bảo đảm:

- Mỗi khối nằm hoàn toàn trong đoạn hỏi vì $b \le L$.
- Một khối bám đầu trái, một khối bám đầu phải.
- Hai khối đủ dài để phủ kín đoạn, không để lại khoảng trống ở giữa.

Cực tiểu của toàn đoạn vì thế phải xuất hiện trong ít nhất một khối. Chỉ cần so sánh cực tiểu của hai khối:

$$\boxed{\mathrm{RMQ}(l, r) = \mathrm{better}\left(st[k][l], st[k][r - 2^k + 1]\right)}$$

Đó là lý do truy vấn chỉ cần **hai lần tra bảng và một lần ghép hai ứng viên**. www2.wild-inter...

Phân biệt hai phép ghép bạn vừa học:

Khi nào?	Ghép những đoạn nào?
Xây bảng	Hai nửa bằng nhau, không giao nhau
Trả lời query	Hai khối bám hai đầu, có thể chồng lấn

Đừng áp dụng công thức bắt đầu nửa phải khi xây bảng cho khối phải lúc truy vấn. Khối phải lúc query phải **kết thúc đúng tại** r.

7. Phần chồng lấn có làm sai kết quả không?

Với min, xét lại một phần tử nhiều lần không đổi đáp án:

$$\min(x, x) = x$$

Đây gọi là **tính lũy đẳng**.

Nhưng nếu thay min bằng tổng thì phần giao bị cộng hai lần. Chẳng hạn với:

$$B = [1, 2, 3, 4, 5, 6]$$

Lấy hai khối dài 4 ở hai đầu:

- Tổng $[0, 3]$ là 10.
- Tổng $[2, 5]$ là 18.

Cộng lại được 28, trong khi tổng thật là 21. Hai phần tử 3 và 4 bị đếm lại.

Vì vậy, **công thức query bằng hai khối chồng lấn dùng được cho min, nhưng không thể thay nguyên xi bằng phép cộng**. Algorithms for ...

8. Viết thuật toán để thấy rõ thứ tự tính

Đây là pseudocode lưu **chỉ số**, đúng quy ước của video:

```
better(p, q):
    nếu A[p] < A[q]: trả p
    nếu A[p] > A[q]: trả q
    trả min(p, q)

build(A):
    n = số phần tử của A

    lg[1] = 0
    với x từ 2 đến n:
        lg[x] = lg[floor(x / 2)] + 1

    với i từ 0 đến n - 1:
        st[0][i] = i

    với k từ 1 đến lg[n]:
        len = 2^k
        half = len / 2

        với i từ 0 đến n - len:
            st[k][i] = better(
                st[k - 1][i],
                st[k - 1][i + half]
            )

rmq(l, r):
    k = lg[r - l + 1]

    p = st[k][l]
    q = st[k][r - 2^k + 1]
```

```
    trả better(p, q)
```

Ở đây, `lg[x]` lưu sẵn $\lfloor \log_2 x \rfloor$. Ví dụ:

x	1	2	3	4	5	6	7	8
`lg[x]`	0	1	1	2	2	2	2	3

Một lỗi rất dễ mắc: nếu `st` lưu chỉ số, bạn phải dùng `better(p,q)` để so sánh $A[p]$ **và** $A[q]$. Viết `min(p,q)` ngay lập tức sẽ chỉ chọn chỉ số nhỏ hơn, kể cả khi giá trị ở đó lớn hơn.

9. Phân tích thời gian và bộ nhớ

Bảng có $\lfloor \log_2 n \rfloor + 1$ tầng. Mỗi tầng có tối đa n ô, mỗi ô xây trong $O(1)$, nên:

$$P(n) = O(n \log n), \qquad S(n) = O(n \log n)$$

Mỗi query dùng `lg`, tra hai ô và ghép hai ứng viên:

$$Q(n) = O(1)$$

Với m truy vấn, tổng thời gian là:

$$\boxed{O(n \log n + m)}$$

Ba cách trong bài có sự đánh đổi như sau:

Cách làm	Tiền xử lý	Một query	Bộ nhớ phụ
Quét từng đoạn	$O(1)$	$O(n)$ tệ nhất	$O(1)$
Lưu mọi đoạn bằng DP	$O(n^2)$	$O(1)$	$O(n^2)$
Sparse Table	$O(n \log n)$	$O(1)$	$O(n \log n)$

Cuối video, mục tiêu tiếp theo là giảm phần $O(n \log n)$ xuống $O(n)$ mà vẫn giữ query $O(1)$.

www2.wild-inter...

Để luyện công thức, hãy tự tính truy vấn $[2, 6]$ trên mảng ban đầu:

- Độ dài $L = 5$, nên $k = 2$, mỗi khối dài 4.
- Hai khối là $[2, 5]$ và $[3, 6]$.
- Tra $st[2][2]$ và $st[2][3]$, đều nhận chỉ số 5.
- Kết quả: **RMQ trả 5; giá trị nhỏ nhất là** $A[5] = 2$.

You said:

1 selection

cho tui code c++ hoàn chỉnh

Worked for 28s

ChatGPT said:

Đây là code **C++17 hoàn chỉnh**, dùng chỉ số từ **0**, trả về **vị trí cực tiểu bên trái** khi có nhiều phần tử bằng nhau.

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

class SparseTable {
private:
    vector<long long> a;
    vector<int> lg;
    vector<vector<int>> st;

    // Chọn chỉ số có giá trị nhỏ hơn.
    // Nếu bằng giá trị, chọn chỉ số bên trái.
    int better(int p, int q) const {
        if (a[p] != a[q]) {
            return a[p] < a[q] ? p : q;
        }
        return min(p, q);
    }

public:
    explicit SparseTable(const vector<long long>& values) : a(values) {
        int n = static_cast<int>(a.size());

        // lg[x] = floor(log2(x))
        lg.assign(n + 1, 0);
        for (int x = 2; x <= n; ++x) {
            lg[x] = lg[x / 2] + 1;
        }

        int levels = lg[n] + 1;
```

```
        st.assign(levels, vector<int>(n));

        // Đoạn dài 1: cực tiểu chính là phần tử tại i.
        for (int i = 0; i < n; ++i) {
            st[0][i] = i;
        }

        // st[k][i] lưu chỉ số cực tiểu của đoạn
        // bắt đầu tại i, có độ dài 2^k.
        for (int k = 1; k < levels; ++k) {
            int len = 1 << k;
            int half = len / 2;

            for (int i = 0; i + len <= n; ++i) {
                int left = st[k - 1][i];
                int right = st[k - 1][i + half];

                st[k][i] = better(left, right);
            }
        }
    }

    // Trả về chỉ số cực tiểu bên trái trên đoạn đóng [l, r].
    int rmq(int l, int r) const {
        int k = lg[r - l + 1];
        int len = 1 << k;

        int left = st[k][l];
        int right = st[k][r - len + 1];

        return better(left, right);
    }

    // Trả về giá trị nhỏ nhất trên đoạn [l, r].
    long long minimum(int l, int r) const {
        return a[rmq(l, r)];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q) || n <= 0 || q < 0) {
```

```
        return 0;
    }

    vector<long long> a(n);
    for (long long& value : a) {
        cin >> value;
    }

    SparseTable table(a);

    // Mỗi truy vấn phải thỏa 0 <= l <= r < n.
    while (q--) {
        int l, r;
        cin >> l >> r;

        cout << table.rmq(l, r) << '\n';
    }

    return 0;
}
```

Input: dòng đầu là `n q`, dòng tiếp theo gồm `n` phần tử, sau đó là `q` đoạn `[l, r]`.

```
8 4
7 4 9 4 6 2 2 8
1 6
1 4
2 2
6 7
```

Output — các chỉ số cực tiểu:

```
5
1
2
6
```

Nếu muốn in **giá trị nhỏ nhất** thay cho vị trí, đổi dòng xuất thành:

```
cout << table.minimum(l, r) << '\n';
```

Tiền xử lý và bộ nhớ: $O(n \log n)$. Mỗi truy vấn: $O(1)$.

You said:

1 selection

khúc này sao lại là i+half, không phải đúng phải là i + 2^(k-1) sao

ChatGPT said:

Đúng rồi, phải là $i + 2^{k-1}$. Trong code, **`i + half` chính là biểu thức đó**, vì:

```
int len = 1 << k;  // len = 2^k
int half = len / 2; // half = 2^(k-1)
```

Do đó:

$$i + \text{half} = i + \frac{2^k}{2} = i + 2^{k-1}$$

Ví dụ $k = 3,\ i = 2$: đoạn cần xây dài $2^3 = 8$, tức **[2,9]**. Chia thành:

Nửa	Đoạn	Ô đã tính
Trái	[2,5], dài 4	`st[2][2]`
Phải	[6,9], dài 4	`st[2][6]`

Nửa phải bắt đầu tại:

$$6 = i + \text{half} = 2 + 4 = 2 + 2^{3-1}$$

Bạn có thể viết trực tiếp như sau, kết quả hoàn toàn giống nhau:

```
int left  = st[k - 1][i];
int right = st[k - 1][i + (1 << (k - 1))];
```

Lưu ý: trong C++, **`^` là XOR**, không phải lũy thừa. Để biểu diễn 2^{k-1} ở đây, dùng `1 << (k - 1)`.

You said:

1 selection

tại sao cái này lại tương đương với log2(x) làm tròn xuống

ChatGPT said:

Vì **mỗi lần chia nguyên cho 2, phần nguyên của $\log_2 x$ giảm đúng 1**, với $x \geq 2$.

Ở đây `x` là `int`, nên `x / 2` bỏ phần thập phân. Ví dụ:

$$13 \to 6 \to 3 \to 1$$

Cần **3 lần chia** để về 1, và:

$$\lfloor \log_2 13 \rfloor = 3$$

Vì vậy, nếu đã biết cần bao nhiêu lần chia để đưa `x / 2` về 1, thì với `x` chỉ cần thêm **một lần chia đầu tiên**:

```
lg[x] = lg[x / 2] + 1;
```

Chứng minh bằng khoảng lũy thừa của 2:

Đặt $k = \lfloor \log_2 x \rfloor$, nghĩa là:

$$2^k \leq x < 2^{k+1}$$

Chia nguyên cho 2, ta được:

$$2^{k-1} \leq \left\lfloor \frac{x}{2} \right\rfloor < 2^k$$

Do đó:

$$\left\lfloor \log_2 \left(\left\lfloor \frac{x}{2} \right\rfloor \right) \right\rfloor = k - 1$$

Cộng thêm 1 thì trở lại k.

Ví dụ quá trình tính bảng:

x	x / 2	Công thức	lg[x]
1	—	Giá trị gốc: $\log_2 1 = 0$	0
2	1	`lg[1] + 1`	1
3	1	`lg[1] + 1`	1
4	2	`lg[2] + 1`	2
5	2	`lg[2] + 1`	2
8	4	`lg[4] + 1`	3

x	x / 2	Công thức	lg[x]
13	6	`lg[6] + 1`	3

Vòng lặp chạy từ nhỏ đến lớn nên **`lg[x / 2]` luôn được tính trước `lg[x]`**.

You said:

1 selection

giờ bạn trình bày giảng cho tui nội dung video này đi

Worked for 1m 52s

ChatGPT said:

Bài này có một ý tưởng rất đáng chú ý:

> **Để biết vị trí nhỏ nhất của mọi đoạn, ta không cần giữ đầy đủ các giá trị của mảng. Hình dạng một cây phù hợp đã chứa đủ thông tin đó.**

Video dùng **range maximum** để minh họa bằng các cột cao thấp. Mình sẽ chuyển về **range minimum** cho thống nhất với hai bài trước; chỉ cần đảo chiều so sánh. www2.wild-inter...

Mình tiếp tục dùng mảng:

Chỉ số i	0	1	2	3	4	5	6	7
$A[i]$	7	4	9	4	6	2	2	8

Nếu có nhiều cực tiểu bằng nhau, vẫn chọn **vị trí bên trái**.

1. Cartesian tree được dựng như thế nào?

Với một đoạn mảng:

1. Tìm vị trí nhỏ nhất của đoạn, chọn nó làm gốc.
2. Các phần tử bên trái vị trí đó tạo thành cây con trái.
3. Các phần tử bên phải tạo thành cây con phải.
4. Tiếp tục áp dụng cùng quy tắc cho hai đoạn con.

Đoạn rỗng không tạo node.

Với mảng đang xét, giá trị nhỏ nhất toàn mảng là 2, ở vị trí 5 và 6. Chọn vị trí **5** làm gốc.

Sau đó:

- Bên trái là đoạn $[0, 4]$, cực tiểu bên trái ở vị trí **1**.

- Bên phải là đoạn $[6, 7]$, cực tiểu ở vị trí **6**.

Tiếp tục:

- Node 1 có đoạn trái $[0, 0]$, tạo node **0**.
- Node 1 có đoạn phải $[2, 4]$, cực tiểu ở vị trí **3**.
- Node 3 có hai con là **2** và **4**.
- Node 6 chỉ có con phải là **7**.

Cây nhận được dưới đây có nhãn dạng `chỉ số : giá trị`:

Mỗi phần tử của mảng tương ứng với đúng một node.

2. Cây này giữ được hai tính chất gì?

Cartesian tree kết hợp hai loại thứ tự:

Tính chất	Ý nghĩa
Thứ tự theo chỉ số	Mọi node trong cây con trái có chỉ số nhỏ hơn node gốc; cây con phải có chỉ số lớn hơn
Min-heap theo giá trị	Giá trị ở cha không lớn hơn giá trị ở con

Đó là hai đặc điểm xác định Cartesian tree; khi có giá trị bằng nhau, ta thêm quy tắc ưu tiên chỉ số nhỏ hơn để cây được xác định duy nhất. static.aminer.org

Nhìn node **1 : 4** trong hình:

- Con trái là **0 : 7**: chỉ số 0 nhỏ hơn 1, nhưng giá trị 7 lớn hơn 4.
- Con phải là **3 : 4**: chỉ số 3 lớn hơn 1, giá trị bằng 4.

Vậy cây được sắp theo **chỉ số**, còn quan hệ cha–con tuân theo **giá trị**.

Một cách kiểm tra tính chất đầu tiên là duyệt **inorder**:

> Duyệt cây con trái → thăm gốc → duyệt cây con phải.

Trên cây này, thứ tự node nhận được là:

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7$$

Tức đúng thứ tự các vị trí trong mảng.

3. LCA là gì?

LCA là viết tắt của **Lowest Common Ancestor**, tức **tổ tiên chung sâu nhất** của hai node.

Một node cũng được xem là tổ tiên của chính nó.

Xét hai node **2** và **4** trong hình:

- Đi từ node 2 lên gốc: $2 \to 3 \to 1 \to 5$.
- Đi từ node 4 lên gốc: $4 \to 3 \to 1 \to 5$.

Chúng có các tổ tiên chung là 3, 1 và 5. Trong đó, node **3** nằm sâu nhất:

$$\mathrm{LCA}(2, 4) = 3$$

"Sâu nhất" rất quan trọng. Node 5 cũng là tổ tiên chung, nhưng nằm cao hơn nên không phải LCA.

4. Vì sao LCA lại cho đáp án RMQ?

Đây là quan hệ chính của bài:

$$\boxed{\mathrm{RMQ}_A(l, r) = \mathrm{LCA}_T(l, r)}$$

Ở đây, mỗi node của Cartesian tree T được nhận diện bằng chỉ số của nó trong mảng. Quan hệ này biến truy vấn cực tiểu trên mảng thành truy vấn tổ tiên chung trên cây. www3.cs.stonyb...

Ví dụ vừa rồi:

$$\mathrm{LCA}(2, 4) = 3$$

Đối chiếu với đoạn mảng:

$$A[2..4] = [9, 4, 6]$$

Phần tử nhỏ nhất nằm tại chỉ số 3. Đúng là:

$$\mathrm{RMQ}(2,4) = 3$$

Thử thêm vài đoạn:

Truy vấn	Đoạn mảng	Vị trí cực tiểu bên trái	LCA của hai đầu
$[0,4]$	$[7,4,9,4,6]$	1	1
$[4,7]$	$[6,2,2,8]$	5	5
$[5,6]$	$[2,2]$	5	5
$[6,7]$	$[2,8]$	6	6

Ở hàng $[5,6]$, node 5 là tổ tiên của node 6, nên LCA chính là node 5.

5. Hiểu chứng minh bằng cách chia đoạn

Giả sử gốc của một cây con là vị trí m. Theo cách dựng, $A[m]$ nhỏ nhất trong đoạn mà cây con đó đại diện.

Khi hỏi $[l,r]$, có ba trường hợp:

- **Cả hai đầu đều bên trái m:** tiếp tục tìm trong cây con trái.
- **Cả hai đầu đều bên phải m:** tiếp tục tìm trong cây con phải.
- $l \le m \le r$**:** đoạn hỏi chứa m, nên m chính là cực tiểu của đoạn hỏi.

Trường hợp thứ ba cũng là lúc hai đầu đoạn nằm ở hai phía của gốc, hoặc một đầu chính là gốc. LCA của chúng vì vậy là m.

Nói trực quan: **LCA là gốc đầu tiên, khi đi từ trên xuống, phân tách được hai đầu đoạn. Gốc đó cũng chính là cực tiểu nằm giữa chúng.**

Đây là lý do kết quả đúng cho mọi đoạn, chứ không chỉ trùng hợp ở các ví dụ.

6. Chỉ biết hình dạng cây mà không biết giá trị thì sao?

Bạn vẫn trả lời được **vị trí cực tiểu**.

Để thấy điều này, xét hai mảng nhỏ:

$$B = [2,1,3]$$

$$C = [3,1,2]$$

Cả hai đều có:

- Gốc ở vị trí 1.
- Con trái ở vị trí 0.
- Con phải ở vị trí 2.

Vì vậy, chúng có cùng hình dạng Cartesian tree và cùng đáp án cho mọi truy vấn:

Đoạn	RMQ trên B	RMQ trên C
$[0,0]$	0	0
$[1,1]$	1	1
$[2,2]$	2	2
$[0,1]$	1	1
$[1,2]$	1	1
$[0,2]$	1	1

Điểm thú vị là thứ tự giữa hai phần tử ngoài cùng đã đổi:

- Trong B: $B[0] < B[2]$.
- Trong C: $C[0] > C[2]$.

Nhưng không có đoạn liên tiếp nào chứa cả hai vị trí 0 và 2 mà bỏ qua vị trí 1. Khi cả hai cùng xuất hiện, số 1 ở giữa đã quyết định cực tiểu.

RMQ không cần biết mọi quan hệ lớn nhỏ giữa mọi cặp phần tử. Nó cần ít thông tin hơn việc biết toàn bộ thứ tự sắp xếp.

Kết quả học thuật chính xác là: hai mảng có cùng đáp án RMQ cho mọi đoạn khi và chỉ khi chúng có cùng hình dạng canonical Cartesian tree, với quy tắc xử lý bằng nhau nhất quán.

static.aminer.org

Nếu chỉ giữ hình dạng cây:

- Gán lại chỉ số bằng inorder.
- Tìm LCA của hai node mang chỉ số cần hỏi.
- Trả chỉ số của LCA.

Nhưng **không thể khôi phục giá trị số ban đầu chỉ từ hình dạng cây**. Biết RMQ trả vị trí 1 không cho biết giá trị ở đó là 1, 10 hay một số khác.

7. Vậy tại sao phải đếm số hình dạng cây?

Đây là phần quan trọng ở nửa sau video.

Ta vừa thấy hai mảng khác nhau có thể có cùng toàn bộ đáp án RMQ. Có thể gom chúng vào cùng một **loại** theo hình dạng cây.

Câu hỏi mới là:

> Với n phần tử, có bao nhiêu loại hình dạng Cartesian tree khác nhau?

Vì đây là cây nhị phân **có phân biệt con trái và con phải**, số hình dạng là số Catalan:

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

Một vài giá trị:

Số node n	Số hình dạng C_n
1	1
2	2
3	5
4	14
5	42

Đối với hướng tối ưu tiếp theo, ta chủ yếu cần chặn trên:

$$\boxed{C_n < 4^n}$$

Video giải thích chặn này bằng cách **mã hóa hình dạng cây thành bit**, thay vì yêu cầu người học chứng minh công thức Catalan. www2.wild-inter...

8. Mã hóa cây bằng hai bit cho mỗi node

Duyệt cây theo **preorder**:

> Thăm gốc → duyệt cây con trái → duyệt cây con phải.

Khi thăm mỗi node, xuất hai bit:

Hai bit	Node có những con nào?
`00`	Không có con
`10`	Chỉ có con trái

Hai bit	Node có những con nào?
`01`	Chỉ có con phải
`11`	Có cả hai con

Ví dụ với mảng nhỏ:

$$D = [4, 2, 1, 3]$$

Cartesian tree của nó là:

Thứ tự preorder là:

$$2,\ 1,\ 0,\ 3$$

Mã từng node:

Node	Con trái?	Con phải?	Hai bit
2	Có	Có	`11`
1	Có	Không	`10`
0	Không	Không	`00`

Node	Con trái?	Con phải?	Hai bit
3	Không	Không	`00`

Ghép lại:

`11 10 00 00`

Tức chuỗi bit:

`11100000`

Cây có 4 node nên dùng $2 \times 4 = 8$ bit.

Đừng nhầm hai cách duyệt:

Cách duyệt	Dùng làm gì trong bài này?
Inorder	Khôi phục thứ tự chỉ số trong mảng
Preorder	Ghi mã hình dạng cây theo thứ tự có thể giải mã

9. Vì sao giải mã được?

Nếu chỉ ghi bit mà không khôi phục được cây thì cách mã hóa chưa có ý nghĩa.

Với chuỗi `11 10 00 00`, đọc theo preorder:

1. Đọc `11`: tạo gốc có hai vị trí dành cho con trái và con phải.
2. Đọc `10`: điền con trái của gốc; node này lại có một con trái.
3. Đọc `00`: điền node lá vào vị trí đó. Nhánh trái đã hoàn tất.
4. Quay lại vị trí con phải còn chờ của gốc, đọc `00`: tạo node lá cuối cùng.

Ta khôi phục đúng hình dạng cây đã mã hóa.

Cần giữ rõ **trái/phải**. Một cây chỉ có con trái và một cây chỉ có con phải là hai hình dạng khác nhau.

Vì mỗi cây n node được mã hóa duy nhất bằng chuỗi dài $2n$ bit, mà số chuỗi bit dài $2n$ là:

$$2^{2n} = 4^n$$

nên số hình dạng cây không thể vượt quá 4^n. Không phải mọi chuỗi $2n$ bit đều là mã hợp lệ của một cây, nên đây là **chặn trên**, không phải số lượng chính xác. Cách mã hóa và đếm này cũng xuất hiện trong slide Stanford về Cartesian tree. web.stanford.edu

10. Những điều chưa thể kết luận ngay

Có hai điểm cần tách riêng khi học bài này.

Thứ nhất, định nghĩa dựng cây chưa tự cho thuật toán $O(n)$. Nếu mỗi lần chia đoạn lại quét tìm min, một mảng tăng dần có thể khiến ta xử lý lần lượt các đoạn dài $n, n-1, \dots, 1$, tổng $O(n^2)$. Có cách dựng tuyến tính bằng stack đơn điệu; đó là bước triển khai cần bổ sung khi phân tích độ phức tạp. static.aminer.org

Thứ hai, có cây chưa có nghĩa đã có query $O(1)$. Nếu tìm LCA bằng cách lần theo cha, cây lệch vẫn có thể khiến mỗi query mất $O(n)$. Tương tự, mã hình dạng dài $2n$ bit chưa tự cung cấp một thuật toán truy vấn nhanh.

Vai trò của bài này là chuẩn bị cho cách dùng lại kết quả: **những khối mảng có cùng hình dạng Cartesian tree có thể dùng chung một bảng đáp án RMQ nội khối**. Bài kế tiếp sẽ khai thác số loại khối bị chặn bởi 4^b, với b là số phần tử mỗi khối, để giảm chi phí tiền xử lý. web.stanford.edu

You said:

1 selection

giờ bạn trình bày giảng cho tui nội dung video này đi

Worked for 3m 14s >

ChatGPT said:

Bài này ghép ba thứ bạn vừa học — **Sparse Table, Cartesian tree và mã hóa cây** — để đạt:

$$\boxed{\text{Tiền xử lý } O(n), \quad \text{query } O(1), \quad \text{bộ nhớ } O(n) \text{ word}}$$

Ý tưởng chính là: **chia mảng thành nhiều khối nhỏ, nhưng chỉ tính bảng đáp án một lần cho mỗi loại khối**. Trong video, kỹ thuật này gọi là *Exhaustive Tabulation*, cũng thường được gọi là *Four Russians*. www2.wild-inter...

Mình tiếp tục dùng chỉ số từ **0**, RMQ trả **vị trí cực tiểu bên trái**.

1. Vì sao vẫn giữ Sparse Table?

Sparse Table đã có query $O(1)$. Vấn đề còn lại là tiền xử lý và bộ nhớ $O(n \log n)$.

Ta sẽ xây Sparse Table trên **một mảng nhỏ hơn**, gồm cực tiểu của từng khối.

Gọi:

- n: số phần tử.
- b: số phần tử mỗi khối.
- $m = \lceil n/b \rceil$: số khối.

Nếu chọn $b = \Theta(\log n)$, ta chỉ còn:

$$m = O\left(\frac{n}{\log n}\right)$$

phần tử đại diện. Khi đó, xây Sparse Table trên mảng đại diện mất:

$$O(m \log m) = O\left(\frac{n}{\log n} \log n\right) = O(n)$$

Đây là phần **bootstrapping**: dùng lại thuật toán đã biết trên đầu vào nhỏ hơn để có chi phí tốt hơn.

2. Chia khối và lưu thông tin gì?

Để nhìn rõ, xét ví dụ 16 phần tử, chủ động chọn $b = 4$. Đây là kích thước minh họa; công thức chọn b để bảo đảm tuyến tính sẽ giải thích phía sau.

Khối t	Chỉ số gốc	Các phần tử	Min	Vị trí min
0	$[0, 3]$	$[2, 7, 5, 8]$	2	0
1	$[4, 7]$	$[9, 4, 1, 6]$	1	6
2	$[8, 11]$	$[8, 5, 3, 7]$	3	10
3	$[12, 15]$	$[10, 2, 0, 4]$	0	14

Ta lưu hai mảng:

$$V = [2, 1, 3, 0]$$

$$G = [0, 6, 10, 14]$$

Trong đó:

- $V[t]$: **giá trị** nhỏ nhất của khối t.
- $G[t]$: **chỉ số gốc** đạt giá trị đó.

Xây Sparse Table trên V, với kết quả trả về **chỉ số khối**.

Ví dụ hỏi cực tiểu trong hai khối 1 và 2:

$$V[1..2] = [1, 3]$$

Sparse Table trả khối 1. Chuyển về chỉ số mảng gốc bằng:

$$G[1] = 6$$

Không lấy min trực tiếp các số trong G: **chỉ số nhỏ hơn không đồng nghĩa giá trị nhỏ hơn**.

3. Query không khớp biên khối thì xử lý thế nào?

Giả sử cần:

$$\mathrm{RMQ}(7, 12)$$

Đoạn hỏi là:

$$[6, 8, 5, 3, 7, 10]$$

Ta tách thành tối đa ba phần:

Phần	Đoạn gốc	Nội dung
Đuôi khối trái	$[7, 7]$	$[6]$
Các khối đầy đủ ở giữa	$[8, 11]$	$[8, 5, 3, 7]$
Đầu khối phải	$[12, 12]$	$[10]$

Phần giữa giải bằng Sparse Table trên V. Hai phần biên nằm hoàn toàn trong từng khối.

Nếu biết đáp án ba phần, chỉ cần so sánh ba ứng viên để chọn đáp án chung. Đây là cách ghép truy vấn trong cấu trúc chia khối. web.stanford.edu

Tại sao không dùng luôn min của cả hai khối biên?

- Khối trái có min 1 ở vị trí 6, **ngoài đoạn hỏi**.
- Khối phải có min 0 ở vị trí 14, cũng **ngoài đoạn hỏi**.

Lấy min cả khối sẽ trả sai.

Vậy đến đây ta đã giải được phần giữa. Việc còn thiếu là **RMQ bên trong một khối**.

4. Chỉ chia khối vẫn chưa đủ

Nếu mỗi khối có $b = \Theta(\log n)$ phần tử:

Cách xử lý bên trong khối	Vấn đề
Mỗi query quét khối	Mất $O(b) = O(\log n)$, chưa đạt query $O(1)$
Xây bảng mọi đoạn riêng cho từng khối	Tổng build $O((n/b)b^2) = O(n \log n)$

Cách xử lý bên trong khối	Vấn đề
Xây Sparse Table riêng cho từng khối	Tổng build $O(n \log b) = O(n \log \log n)$

Ta cần tránh **xây lại thông tin tương đương ở quá nhiều khối**.

Đây là lúc Cartesian tree có tác dụng.

5. Nhiều khối khác giá trị nhưng dùng chung một bảng được

Xét ba khối trong ví dụ:

$$B_1 = [9, 4, 1, 6]$$

$$B_2 = [8, 5, 3, 7]$$

$$B_3 = [10, 2, 0, 4]$$

Cả ba đều có cùng hình dạng Cartesian tree khi dùng chỉ số **tương đối trong khối**:

Vì chúng cùng hình dạng cây, mọi truy vấn nội khối có cùng đáp án **tương đối**.

Ví dụ:

Đoạn tương đối	Trên B_1	Trên B_2	Trên B_3	Vị trí trả về
$[0, 1]$	$[9, 4]$	$[8, 5]$	$[10, 2]$	1

Đoạn tương đối	Trên B_1	Trên B_2	Trên B_3	Vị trí trả về
$[0,3]$	Cả khối	Cả khối	Cả khối	2
$[2,3]$	$[1,6]$	$[3,7]$	$[0,4]$	2

Ta có thể **lưu một bảng chung cho cả ba khối**.

Tính chất "cùng hình dạng Cartesian tree thì cùng mọi đáp án RMQ" là nền tảng của cách chia sẻ bảng này. static.aminer.org

6. Bảng tra chung được đánh chỉ số như thế nào?

Ở bài trước, hình dạng cây trên được mã hóa preorder thành:

```
11 10 00 00
```

Tức:

```
11100000
```

Dùng chuỗi bit này làm **mã loại khối**, gọi là `type`.

Ta lưu:

$$\boxed{T[\text{type}][u][v]}$$

Ý nghĩa: với một khối thuộc loại `type`, RMQ trên đoạn tương đối $[u,v]$ trả về vị trí nào?

Ví dụ, với loại cây `11100000`:

$[u,v]$	$T[\text{type}][u][v]$
$[0,1]$	1
$[0,2]$	2
$[0,3]$	2
$[1,2]$	2
$[1,3]$	2
$[2,3]$	2

Còn đoạn một phần tử thì:

$$T[\text{type}][u][u] = u$$

Mỗi khối chỉ cần lưu mã loại của mình; các khối cùng loại đều trỏ tới cùng bảng.

7. Từ vị trí tương đối chuyển về vị trí gốc

Đây là phần dễ nhầm khi viết code.

Khối t bắt đầu tại chỉ số:

$$s = t \cdot b$$

Nếu bảng tra trả vị trí tương đối p, chỉ số gốc là:

$$\boxed{s + p = t \cdot b + p}$$

Ví dụ hỏi đoạn gốc $[4, 5]$:

- Nằm trong khối 1, bắt đầu tại $s = 4$.
- Đoạn tương đối là $[0, 1]$.
- Bảng chung trả $p = 1$.
- Vị trí gốc là $4 + 1 = 5$.

Đúng vì:

$$A[4..5] = [9, 4]$$

cực tiểu nằm tại chỉ số 5.

Hàm truy vấn nội khối có thể viết:

```
inBlock(t, u, v):
    p = T[type[t]][u][v]
    return t * b + p
```

8. Ghép lại toàn bộ ví dụ $\text{RMQ}(7, 12)$

Với $b = 4$:

$$\text{khối trái} = 7/4 = 1, \qquad \text{khối phải} = 12/4 = 3$$

Các phép chia ở đây là chia nguyên.

Phần	Cách lấy đáp án	Chỉ số gốc	Giá trị
Đuôi khối 1	Tra đoạn tương đối $[3,3]$	7	6
Khối đầy đủ 2	Query trên $V[2..2]$, rồi lấy $G[2]$	10	3
Đầu khối 3	Tra đoạn tương đối $[0,0]$	12	10

So sánh:

$$A[7]=6,\quad A[10]=3,\quad A[12]=10$$

Kết quả:

$$\boxed{\mathrm{RMQ}(7,12)=10}$$

Trong query chỉ có các phép tra bảng, tính chỉ số và so sánh; **không có vòng lặp theo độ dài đoạn**.

Hai trường hợp đặc biệt:

- **Hai đầu cùng khối:** tra trực tiếp một đoạn nội khối.
- **Hai đầu ở hai khối kề nhau:** không có phần giữa, chỉ ghép hai phần biên.

9. Vì sao bảng cho "mọi loại khối" không quá lớn?

Khối dài b có Cartesian tree b node. Mã cây dùng $2b$ bit, nên số mã có thể có bị chặn bởi:

$$2^{2b}=4^b$$

Không phải mọi mã đều là cây hợp lệ, nhưng dùng chặn trên này đã đủ.

Mỗi loại khối có $O(b^2)$ đoạn cần lưu đáp án. Do đó, bảng tra chung có kích thước:

$$\boxed{O(4^b b^2)}$$

Điều quan trọng là nó phụ thuộc vào **số loại khối**, chứ không phải số khối thực tế.

Đây là ý nghĩa của *exhaustive tabulation*: tính trước toàn bộ đáp án cho các trường hợp nhỏ có thể xuất hiện. www2.wild-inter...

10. Vì sao chọn $b=\frac{1}{4}\log_2 n$?

Ta cần cân bằng hai yêu cầu:

- Khối đủ lớn để mảng đại diện nhỏ.

- Khối đủ nhỏ để số loại 4^b không bùng nổ.

Chọn:

$$b \approx \frac{1}{4} \log_2 n$$

Khi đó:

$$\begin{aligned} 4^b &= 4^{\frac{1}{4}\log_2 n} \\ &= (2^2)^{\frac{1}{4}\log_2 n} \\ &= 2^{\frac{1}{2}\log_2 n} \\ &= \sqrt{n} \end{aligned}$$

Ta có nhiều khối, nhưng số loại chỉ bị chặn bởi $\sqrt{n}$.

Ví dụ với:

$$n = 65.536 = 2^{16}$$

ta chọn $b = 4$, nên:

- Số khối: $65.536/4 = 16.384$.
- Số mã loại tối đa: $4^4 = 256$.
- Thực tế chỉ có **14 hình dạng cây nhị phân 4 node**.

Có thể dùng lại một số ít bảng thay vì xây riêng cho 16.384 khối.

Để dùng số nguyên trong chương trình:

$$b = \max\left(1, \left\lfloor \frac{1}{4} \log_2 n \right\rfloor\right)$$

Lựa chọn này cũng được dùng trong cách trình bày cấu trúc Fischer–Heun của Stanford.

web.stanford.edu

11. Chứng minh tiền xử lý $O(n)$

Có ba khoản chính.

Tính min và xác định loại của các khối

Mỗi khối mất $O(b)$, có khoảng n/b khối:

$$O\left(\frac{n}{b} \cdot b\right) = O(n)$$

Để đạt chi phí này, dựng Cartesian tree bằng **stack đơn điệu**, rồi duyệt mã hóa trong $O(b)$. Nếu cứ quét tìm min đệ quy, có thể vượt chi phí mong muốn.

Xây Sparse Table trên mảng đại diện

$$O\left(\frac{n}{b}\log\frac{n}{b}\right)=O(n)$$

vì $b=\Theta(\log n)$.

Tạo bảng tra chung

Video dùng một chặn đơn giản:

- Tối đa 4^b loại.
- Mỗi loại có $O(b^2)$ đoạn.
- Tính từng đáp án bằng cách đơn giản mất $O(b)$.

Vậy:

$$O(4^b b^3)=O(\sqrt{n}\log^3 n)=o(n)$$

Bước cuối dùng tính chất: **một lũy thừa cố định của** $\log n$ **tăng chậm hơn** $\sqrt{n}$. Đây là kết quả tiệm cận khi n tăng.

Cộng lại:

$$\boxed{P(n)=O\left(n+\frac{n}{b}\log\frac{n}{b}+4^b b^3\right)=O(n)}$$

Nếu xây bảng mọi đoạn của mỗi loại bằng DP, có thể giảm phần b^3 xuống b^2; nhưng chặn đơn giản trong video đã đủ chứng minh tuyến tính. www2.wild-inter... +1

12. Query và bộ nhớ đạt bao nhiêu?

Một truy vấn khác khối cần:

- Hai lần tra bảng nội khối.
- Tối đa một query Sparse Table trên mảng đại diện.
- Ghép tối đa ba ứng viên.

Do đó:

$$\boxed{Q(n)=O(1)}$$

Bộ nhớ gồm mảng đại diện, Sparse Table nhỏ, mã loại của từng khối và bảng tra chung:

$$S(n) = O\left(n + \frac{n}{b}\log\frac{n}{b} + 4^b b^2\right) = \boxed{O(n) \text{ word}}$$

Trong mô hình word-RAM của bài giảng, mã loại dài $2b = O(\log n)$ bit vừa một word, nên có thể dùng trực tiếp để đánh chỉ số bảng trong $O(1)$. www2.wild-inter...

Đây là $O(n)$ **word**, chưa phải cấu trúc succinct $2n + o(n)$ bit. Khi ghép ứng viên từ các khối khác nhau, cách này vẫn dùng giá trị trong mảng gốc để so sánh.

13. Pseudocode của toàn bộ một query

Giả sử:

- `macro.rmq(x, y)` trả **chỉ số khối** đạt min trong `V[x..y]`.
- `better(p, q)` so sánh `(A[p], p)` với `(A[q], q)`.
- Các khối đủ b phần tử.

```
inBlock(t, u, v):
    p = T[type[t]][u][v]
    return t * b + p

rmq(l, r):
    a = l / b
    c = r / b

    nếu a == c:
        return inBlock(a, l % b, r % b)

    left  = inBlock(a, l % b, b - 1)
    right = inBlock(c, 0, r % b)

    ans = better(left, right)

    nếu a + 1 <= c - 1:
        t = macro.rmq(a + 1, c - 1)
        ans = better(ans, G[t])

    return ans
```

Khi triển khai, khối cuối có thể thiếu phần tử. Có thể đệm bằng giá trị lớn hơn mọi giá trị thật và chỉ cho phép query trên chỉ số thật. Quy tắc chọn cực tiểu bên trái phải nhất quán ở cả **Cartesian tree, bảng nội khối và bước ghép ứng viên**.

You said:

1 selection

chưa hiểu, chứng minh lại dễ hiểu tại sao RMQ = LCA

Worked for 58s

ChatGPT said:

Mình đổi cách giải thích: **ta cần chứng minh LCA vừa nằm trong đoạn hỏi, vừa nhỏ nhất trong đoạn đó.**

Trước hết, xem một ví dụ nhỏ, không có giá trị trùng nhau:

Chỉ số	**0**	**1**	**2**	**3**	**4**
$A[i]$	5	2	6	1	4

Cartesian tree của mảng là như sau. Nhãn node là `chỉ số : giá trị`:

Xét truy vấn $[0, 2]$.

Đoạn được hỏi là:

$$[5, 2, 6]$$

Min nằm ở chỉ số **1**.

Nhìn trên cây, hai node **0 và 2** gặp nhau lần đầu khi đi lên tại node **1**. Vì vậy:

$$\text{LCA}(0,2) = 1 = \text{RMQ}(0,2)$$

Node 3 cũng là tổ tiên chung, thậm chí có giá trị nhỏ hơn. Nhưng **chỉ số 3 nằm ngoài đoạn** $[0,2]$, nên không thể là đáp án. Đó là lý do cần tổ tiên chung **sâu nhất**, không phải một tổ tiên chung bất kỳ.

Giờ chứng minh cho mọi đoạn.

Bước 1: LCA phải nằm trong đoạn $[l,r]$

Gọi $m = \text{LCA}(l,r)$.

Nhớ cách dựng Cartesian tree:

- Chỉ số nhỏ hơn m nằm trong cây con trái của m.
- Chỉ số lớn hơn m nằm trong cây con phải của m.

Giả sử m nằm **bên trái cả đoạn hỏi**, tức $m < l$. Khi đó cả l, r đều nằm trong cây con phải của m.

Nhưng như vậy, **con phải của** m **cũng là tổ tiên chung của cả hai**, và nằm sâu hơn m. Điều này trái với việc m là tổ tiên chung sâu nhất.

Nếu $m > r$, lập luận tương tự với cây con trái.

Vậy m không thể nằm ngoài đoạn:

$$\boxed{l \le m \le r}$$

Bước 2: Giá trị tại LCA không lớn hơn bất kỳ phần tử nào trong đoạn

Đây là điểm quan trọng: **mỗi cây con của Cartesian tree đại diện cho một đoạn chỉ số liên tiếp**.

Trong hình:

- Cây con gốc 1 chứa đoạn $[0,2]$.
- Cây con gốc 3 chứa đoạn $[0,4]$.

Vì m là tổ tiên của cả l, r, cây con gốc m chứa hai đầu đoạn. Do các chỉ số trong cây con liên tiếp, nó cũng chứa **mọi chỉ số ở giữa** l **và** r.

Mà khi dựng Cartesian tree, gốc được chọn là cực tiểu của đoạn tạo ra cây con. Do đó:

$$A[m] \le A[k] \qquad \text{với mọi } k \in [l,r]$$

Ta đã có đủ hai điều:

Điều cần chứng minh	Đã có
m là một vị trí hợp lệ trong đoạn hỏi	$l \le m \le r$
Giá trị ở m nhỏ nhất trong đoạn hỏi	$A[m] \le A[k]$ với mọi $k \in [l, r]$

Suy ra:

$$\boxed{\text{RMQ}(l, r) = m = \text{LCA}(l, r)}$$

Nếu có giá trị bằng nhau, ta giữ quy tắc **chọn cực tiểu bên trái khi dựng cây**, để vị trí trả về cũng nhất quán.

Bạn có thể kiểm tra thêm ngay trên hình: với đoạn $[2, 4]$, hai node 2 và 4 có LCA là **3**; đoạn mảng $[6, 1, 4]$ cũng có cực tiểu tại **chỉ số 3**.

You said:

1 selection

giải thích lại dễ hiểu hơn giúp tui,, đồng thời \(b=\Theta(\log n)\) nghĩa là gì

Worked for 52s

ChatGPT said:

$b = \Theta(\log n)$ **nghĩa là kích thước khối** b **tăng cùng bậc với** $\log n$**.** Mình giải thích ký hiệu này trước, rồi quay lại chuyện Sparse Table.

1. "Cùng bậc với $\log n$**" nghĩa là gì?**

Những cách chọn sau đều thuộc $\Theta(\log n)$:

$$b = \log_2 n, \qquad b = \frac{1}{4}\log_2 n, \qquad b = 2\log_2 n$$

Chúng chỉ khác nhau bởi **hệ số hằng số**.

Nói chính xác: tồn tại hai hằng số dương c_1, c_2, sao cho khi n đủ lớn:

$$c_1 \log n \le b \le c_2 \log n$$

Vậy $\Theta(\log n)$ mô tả **tốc độ tăng**, không bắt buộc b phải bằng đúng $\log n$.

Ví dụ chọn $b = \log_2 n$:

n	b
16	4
256	8
65.536	16

Mảng tăng rất nhiều thì kích thước khối chỉ tăng chậm.

2. Vì sao số phần tử đại diện giảm xuống $n/\log n$?

Mỗi khối có b phần tử, nhưng ta chỉ đưa **một phần tử đại diện** của khối vào mảng mới.

Ví dụ:

- Mảng gốc có $n = 65.536$ phần tử.
- Chọn $b = 16 = \log_2 n$.
- Mỗi nhóm 16 phần tử tạo một đại diện.

Số đại diện là:

$$m = \frac{65.536}{16} = 4.096$$

Tổng quát, khi chọn $b = \log_2 n$:

$$\boxed{m \approx \frac{n}{\log_2 n}}$$

Dấu "xấp xỉ" ở đây để bỏ qua chuyện khối cuối có thể thiếu phần tử; công thức chính xác là $m = \lceil n/b \rceil$.

3. Vì sao xây Sparse Table trên mảng mới chỉ tốn $O(n)$?

Bạn nhớ Sparse Table có:

- Khoảng $\log m$ tầng.
- Mỗi tầng tối đa m ô.

Vì thế chi phí xây là:

$$O(m \log m)$$

Nhưng lần này, m chỉ khoảng $n/\log n$.

Ngoài ra, mảng mới không dài hơn mảng gốc, nên:

$$m \leq n \quad \Rightarrow \quad \log m \leq \log n$$

Do đó, bỏ qua làm tròn:

$$m \log m \leq \frac{n}{\log n} \cdot \log n = n$$

Trực giác là: số phần tử giảm khoảng $\log n$ lần, đủ để bù lại số tầng khoảng $\log n$ của Sparse Table.

Với ví dụ cụ thể:

$$n = 65.536, \quad m = 4.096$$

Ta có:

$$m \log_2 m = 4.096 \times 12 = 49.152$$

Đây là ước lượng theo biểu thức $m \log m$, không phải số phép tính chính xác, nhưng giúp bạn thấy vì sao chi phí có thể được chặn bởi một hằng số nhân n.

4. Vậy trong thuật toán, chọn $b = \frac{1}{4} \log_2 n$ có còn đúng không?

Có. Lúc đó mỗi khối nhỏ hơn 4 lần, nên số đại diện nhiều hơn khoảng 4 lần:

$$m \approx \frac{4n}{\log n}$$

Chi phí xây được chặn bởi:

$$m \log m \leq \frac{4n}{\log n} \cdot \log n = 4n$$

Mà:

$$\boxed{O(4n) = O(n)}$$

$O(n)$ không có nghĩa đúng n phép tính; nó cho phép một hệ số hằng số như 4. Đó là lý do ta có thể dùng cả họ kích thước $b = \Theta(\log n)$.

Phần này mới chứng minh **Sparse Table trên mảng đại diện** xây trong $O(n)$. Các truy vấn nằm bên trong khối vẫn cần bảng tra theo loại Cartesian tree như phần tiếp theo của thuật toán.

You said:

cái ký hiệu theta nghĩa là gì

ChatGPT said:

Θ đọc là **"theta"**, dùng để nói một hàm tăng **đúng theo một bậc nào đó**, khi bỏ qua hệ số hằng số và các thành phần tăng chậm hơn.

Ví dụ:

$$3n = \Theta(n), \qquad 100n = \Theta(n)$$

Cả hai đều tăng theo **bậc tuyến tính**: khi n tăng gấp đôi, chúng cũng tăng gấp đôi. Hệ số 3 hay 100 không làm thay đổi bậc tăng trưởng.

Tương tự:

$$5n^2 + 3n + 7 = \Theta(n^2)$$

Vì khi n lớn, thành phần n^2 quyết định tốc độ tăng.

Trong bài của mình:

$$b = \Theta(\log n)$$

nghĩa là b **tăng cùng bậc với** $\log n$. Chẳng hạn, cả ba cách chọn sau đều thỏa mãn:

$$b = \log n, \qquad b = \frac{1}{4}\log n, \qquad b = 10\log n$$

Về định nghĩa, viết $f(n) = \Theta(g(n))$ khi có hai hằng số dương c_1, c_2 sao cho, với n đủ lớn:

$$\boxed{c_1 g(n) \le f(n) \le c_2 g(n)}$$

Tức là f được kẹp **cả phía dưới lẫn phía trên** bởi các hằng số nhân với g. Còn $O(g(n))$ chỉ yêu cầu chặn phía trên.

You said:

vậy chọn hệ số nào cũng cho ra là O(n) đúng không, kiểu c1*log2(n) với c1 là số tuỳ ý

Worked for 38s

ChatGPT said:

Đúng với phần xây Sparse Table trên mảng đại diện, miễn hệ số đó là **hằng số dương, không phụ thuộc vào** n.

Gọi hệ số là c:

$$b = c \log_2 n$$

Bỏ qua làm tròn, số khối là:

$$m \approx \frac{n}{c \log_2 n}$$

Vì $\log_2 m \le \log_2 n$, chi phí xây Sparse Table được chặn bởi:

$$m \log_2 m \le \frac{n}{c \log_2 n} \cdot \log_2 n = \frac{n}{c}$$

Với c cố định:

$$\boxed{O(n/c) = O(n)}$$

Ví dụ $c = 1/4$ cho chặn $4n$; $c = 1/10$ cho chặn $10n$. Cả hai vẫn là $O(n)$.

Nhưng để toàn bộ thuật toán có tiền xử lý $O(n)$, không chọn c tùy ý được.

Ta còn bảng tra nội khối. Số loại khối bị chặn bởi:

$$4^b = 4^{c \log_2 n} = n^{2c}$$

Theo cách phân tích trong video, chi phí tạo bảng tra được chặn bởi:

$$O(4^b b^3) = O\left(n^{2c} \log^3 n\right)$$

Để khoản này vẫn nhỏ hơn tuyến tính về mặt tiệm cận, chọn:

$$\boxed{0 < c < \frac{1}{2}}$$

Chọn c	4^b	Chặn chi phí bảng tra
$1/4$	$\sqrt{n}$	$O(\sqrt{n}\log^3 n) = o(n)$
$1/3$	$n^{2/3}$	$O(n^{2/3}\log^3 n) = o(n)$
$1/2$	n	$O(n\log^3 n)$, không còn chặn tuyến tính
1	n^2	$O(n^2\log^3 n)$

Điểm cần chú ý là **hệ số c nằm trong số mũ của 4^b**. Nó làm thay đổi bậc tăng trưởng của bảng tra, chứ không chỉ nhân thời gian lên một hằng số.

Vì vậy, video chọn $c = 1/4$: vừa làm Sparse Table đại diện xây trong $O(n)$, vừa giữ chi phí bảng tra đủ nhỏ.

You said:

1 selection

bạn giải thích dễ hiểu hơn, tui hiểu là nếu chia mảng ban đầu thành nhiều khối rồi sparse table trên từng khối sẽ tốt hơn là làm trên nguyên mảng, nhưng tui không biết nó tốt hơn thế nào

Worked for 54s

ChatGPT said:

Bạn hiểu đúng: **xây Sparse Table riêng cho từng khối giúp giảm thời gian xây và bộ nhớ. Lý do là mỗi bảng nhỏ có ít tầng hơn bảng trên toàn mảng.**

Mình giải thích bằng số trước.

1. Bảng nhỏ hơn thì ít tầng hơn

Giả sử mảng có:

$$n = 65.536 = 2^{16}$$

Sparse Table trên toàn mảng phải hỗ trợ các độ dài:

$$1, 2, 4, \ldots, 65.536$$

Tức **17 tầng**, từ $k = 0$ đến $k = 16$.

Bây giờ chia thành các khối dài $b = 4$. Mỗi Sparse Table nhỏ chỉ cần hỗ trợ:

$$1, 2, 4$$

Tức chỉ **3 tầng**.

Nếu cấp phát bảng hình chữ nhật giống code bạn đã học:

Cách làm	Số bảng	Số ô mỗi bảng	Tổng số ô
Trên nguyên mảng	1	65.536×17	**1.114.112**
Riêng từng khối dài 4	16.384	4×3	**196.608**

Dù có nhiều bảng nhỏ, **tổng số phần tử của các khối vẫn chỉ là** n. Ta không nhân bản mảng; ta giảm số tầng cần lưu cho mỗi phần của mảng.

Các số trên là số ô cấp phát, không phải số phép tính chính xác.

2. Tổng quát, tại sao thành $O(n \log b)$?

Một khối có b phần tử, nên xây Sparse Table của nó mất:

$$O(b \log b)$$

Có khoảng n/b khối. Vậy xây tất cả mất:

$$\frac{n}{b} \times O(b \log b) = \boxed{O(n \log b)}$$

Bạn có thể nhìn như sau:

- Trước đây: khoảng n ô mỗi tầng, có khoảng $\log n$ **tầng**.
- Sau khi chia khối: cộng tất cả bảng nhỏ vẫn khoảng n ô mỗi tầng, nhưng chỉ có khoảng $\log b$ **tầng**.

Vì b nhỏ hơn n, nên $\log b$ cũng nhỏ hơn $\log n$.

3. Nếu $b = \Theta(\log n)$, cải thiện đến mức nào?

Kích thước mỗi khối cỡ $\log n$, nên số tầng của bảng trong khối cỡ:

$$\log b = \Theta(\log \log n)$$

Chi phí giảm từ:

$$\underbrace{O(n \log n)}_{\text{Sparse Table toàn mảng}}$$

xuống:

$$\underbrace{O(n \log \log n)}_{\text{Sparse Table riêng từng khối}}$$

Query nằm trong một khối vẫn là $O(1)$ như trước. Phần được cải thiện ở đây là **thời gian xây và bộ nhớ**, không phải bậc thời gian của query nội khối.

4. Nhưng các bảng nhỏ chưa trả lời được mọi đoạn

Nếu truy vấn nằm vắt qua nhiều khối, không bảng nhỏ nào chứa toàn bộ đoạn đó.

Ta vẫn phải ghép:

- Đuôi khối trái: dùng Sparse Table của khối trái.
- Các khối đầy đủ ở giữa: dùng Sparse Table trên mảng đại diện.
- Đầu khối phải: dùng Sparse Table của khối phải.

Sparse Table trên mảng đại diện xây trong $O(n)$, như phần trước. Cộng cả hai phần:

$$O(n) + O(n \log \log n) = O(n \log \log n)$$

Vậy cách này **đã tốt hơn xây trên nguyên mảng**, nhưng chưa đạt mục tiêu $O(n)$.

Bảng tra theo loại Cartesian tree giúp giảm tiếp chi phí: những khối có cùng hình dạng cây dùng chung một bảng, thay vì mỗi khối lại xây một Sparse Table riêng.

You said:

1 selection

trình bày lại dễ hiểu hơn

Worked for 34s

ChatGPT said:

$\mathrm{RMQ}(7, 12)$ **nghĩa là tìm vị trí có giá trị nhỏ nhất trong đoạn từ** $A[7]$ **đến** $A[12]$.

Với mỗi khối dài 4 phần tử:

- Khối 1 chứa các vị trí **4–7**.
- Khối 2 chứa các vị trí **8–11**.
- Khối 3 chứa các vị trí **12–15**.

Đoạn đang hỏi gồm đúng 6 phần tử này:

Chỉ số gốc	**7**	**8**	**9**	**10**	**11**	**12**
Giá trị	6	8	5	3	7	10
Thuộc khối	1	2	2	2	2	3

Bước 1: Tách đoạn hỏi theo ranh giới khối

Nhìn hàng cuối của bảng, đoạn hỏi chia thành:

- **Phần trái:** chỉ có vị trí **7**, thuộc khối 1.
- **Phần giữa:** toàn bộ các vị trí **8–11**, tức cả khối 2.
- **Phần phải:** chỉ có vị trí **12**, thuộc khối 3.

Ba phần này ghép lại chính là đoạn $[7, 12]$, không thiếu hay thêm phần tử nào.

Bước 2: Lấy phần tử nhỏ nhất của từng phần

Phần	**Các giá trị được xét**	**Nhỏ nhất**	**Nằm ở chỉ số gốc**
Trái: $[7, 7]$	$[6]$	6	**7**
Giữa: $[8, 11]$	$[8, 5, 3, 7]$	3	**10**
Phải: $[12, 12]$	$[10]$	10	**12**

Các bảng đã tiền xử lý giúp lấy ba kết quả này mà không cần quét lại các phần.

Bước 3: So sánh ba ứng viên

Bây giờ chỉ còn so sánh:

$$\underbrace{A[7] = 6}_{\text{ứng viên trái}}, \qquad \underbrace{A[10] = 3}_{\text{ứng viên giữa}}, \qquad \underbrace{A[12] = 10}_{\text{ứng viên phải}}$$

Giá trị **3** nhỏ nhất, nằm ở vị trí **10**:

$$\boxed{\text{RMQ}(7, 12) = 10}$$

10 là vị trí trả về; 3 là giá trị nhỏ nhất.

Còn hai ký hiệu trong phần giải thích cũ có nghĩa:

- **"Đoạn tương đối $[3,3]$"**: vị trí 7 là phần tử có chỉ số nội khối $7-4=3$ trong khối 1.
- **"Đoạn tương đối $[0,0]$"**: vị trí 12 là phần tử đầu tiên, có chỉ số nội khối $12-12=0$ trong khối 3.
- **"Lấy $G[2]$"**: $G[2]$ lưu sẵn vị trí cực tiểu của toàn khối 2, chính là **10**.

You said:

1 selection

trinh bày lại dễ hiểu hơn

Worked for 31s

ChatGPT said:

Bạn cần phân biệt **số khối đang có** và **số loại khối khác nhau**. Nhiều khối có thể cùng loại và dùng chung một bảng đáp án.

Mình lấy $b=4$ để giải thích bằng số.

1. Một loại khối chỉ cần một "tờ đáp án"

Hai khối:

$$[9,4,1,6] \qquad \text{và} \qquad [8,5,3,7]$$

có cùng hình dạng Cartesian tree. Vì vậy, chúng trả cùng **vị trí tương đối** cho mọi đoạn nội khối.

Ví dụ:

Đoạn tương đối	Khối thứ nhất trả	Khối thứ hai trả
$[0,1]$	1	1
$[0,3]$	2	2
$[2,3]$	2	2

Ta chỉ cần lưu **một tờ đáp án chung**, thay vì hai bản giống nhau. Mỗi khối giữ mã loại để tìm đúng tờ đó.

2. Một tờ đáp án cần bao nhiêu ô?

Khối dài 4 có các vị trí $0,1,2,3$.

Đếm mọi đoạn có thể hỏi:

Điểm bắt đầu	Những đoạn có thể hỏi	Số đoạn
0	$[0,0], [0,1], [0,2], [0,3]$	4
1	$[1,1], [1,2], [1,3]$	3
2	$[2,2], [2,3]$	2
3	$[3,3]$	1

Tổng cộng:

$$4 + 3 + 2 + 1 = 10$$

Vậy **mỗi loại khối dài 4 chỉ cần 10 ô đáp án**.

Với khối dài b, số đoạn là:

$$b + (b-1) + \cdots + 1 = \frac{b(b+1)}{2} = O(b^2)$$

3. Có tối đa bao nhiêu loại?

Một cây có $b = 4$ node được mã hóa bằng:

$$2b = 8 \text{ bit}$$

Có $2^8 = 256$ chuỗi bit dài 8. Vì mỗi hình dạng cây có mã riêng, số loại cây **không thể vượt quá 256**.

Thực tế còn ít hơn vì nhiều chuỗi bit không biểu diễn cây hợp lệ.

Tổng quát:

$$\text{số loại} \leq 2^{2b} = 4^b$$

4. Nhân hai con số lại

Với $b = 4$:

- Tối đa **256 loại**.
- Mỗi loại cần **10 ô**.

Bảng chung vì vậy cần không quá:

$$256 \times 10 = 2.560 \text{ ô}$$

Giả sử mảng có **16.384 khối**:

Cách lưu	Số ô đáp án
Mỗi khối giữ bảng riêng	$16.384 \times 10 = 163.840$
Dùng bảng chung theo loại	Không quá $256 \times 10 = 2.560$

Dù có thêm nhiều khối cùng loại, ta vẫn dùng lại những tờ đáp án đã có. **Mỗi khối chỉ cần lưu thêm mã loại của nó.**

Đó là nguồn gốc công thức:

$$\underbrace{O(4^b)}_{\text{số loại}} \times \underbrace{O(b^2)}_{\text{số đáp án mỗi loại}} = \boxed{O(4^b b^2)}$$

Công thức này tính **bảng đáp án dùng chung**; phần lưu mã loại cho từng khối được tính riêng.

You said:

1 selection

trình bày cho tui nội dung

Worked for 2m 37s

ChatGPT said:

Bài này cho thấy một ứng dụng cụ thể của RMQ:

> **Tìm xem hai đoạn bắt đầu ở hai vị trí trong chuỗi giống nhau liên tiếp được bao nhiêu ký tự.**

Ta sẽ đi qua ba bước: **sắp xếp các hậu tố → lưu độ dài tiền tố chung giữa các hậu tố kề nhau → dùng RMQ**. Đây là hướng giải LCE bằng suffix array và LCP trong video. www2.wild-inter...

1. LCE hỏi điều gì?

LCE là **Longest Common Extension**.

Cho chuỗi S và hai vị trí i, j, ta bắt đầu đọc từ hai vị trí đó sang phải, đếm số ký tự giống nhau liên tiếp cho đến khi khác nhau hoặc hết chuỗi.

Ví dụ văn bản gốc:

bananaban

Dùng chỉ số từ 0:

Chỉ số	0	1	2	3	4	5	6	7	8
Ký tự	b	a	n	a	n	a	b	a	n

Hỏi $\mathrm{LCE}(1, 7)$:

- Từ vị trí 1 đọc được: **an**anaban.
- Từ vị trí 7 đọc được: **an**.

Hai bên giống nhau được 2 ký tự rồi bên thứ hai hết chuỗi:

$$\boxed{\mathrm{LCE}(1, 7) = 2}$$

Cách đơn giản là so từng ký tự. Nhưng nếu có nhiều truy vấn, mỗi lần so lại có thể tốn $O(n)$.

2. LCE chính là LCP của hai hậu tố

Hậu tố bắt đầu tại vị trí i là đoạn từ i đến cuối chuỗi.

Ví dụ:

- Hậu tố tại 1: `ananaban` .
- Hậu tố tại 7: `an` .

LCP — Longest Common Prefix là độ dài tiền tố chung dài nhất của hai chuỗi.

Vì vậy:

$$\boxed{\mathrm{LCE}(i, j) = \mathrm{LCP}(S[i..], S[j..])}$$

Hai cách gọi này nhìn cùng một việc:

- **LCE:** nhận hai vị trí trong văn bản.
- **LCP:** xét hai hậu tố bắt đầu tại các vị trí đó.

3. Suffix array giúp sắp xếp các hậu tố

Ta thêm ký tự kết thúc `$` , quy ước nhỏ hơn các chữ cái và không xuất hiện trong văn bản gốc:

```
S = bananaban$
```

Liệt kê các hậu tố rồi sắp theo thứ tự từ điển:

Hạng k	Vị trí bắt đầu $SA[k]$	Hậu tố
0	9	$
1	5	aban$
2	7	an$
3	3	anaban$
4	1	ananaban$
5	6	ban$
6	0	bananaban$
7	8	n$
8	4	naban$
9	2	nanaban$

Mảng **suffix array** lưu cột vị trí bắt đầu:

$$SA = [9, 5, 7, 3, 1, 6, 0, 8, 4, 2]$$

Ví dụ:

$$SA[2] = 7$$

nghĩa là **hậu tố đứng ở hạng 2 bắt đầu tại vị trí 7 trong văn bản**.

Trong chương trình, ta chỉ lưu các chỉ số này; cột chuỗi hậu tố ở bảng trên dùng để minh họa.

4. Mảng `rank` làm chiều ngược lại

Ta cần biết hậu tố bắt đầu tại một vị trí đang đứng ở đâu trong thứ tự đã sắp xếp.

Mảng `rank` trả lời điều đó:

$$rank[SA[k]] = k$$

Trong ví dụ:

$$rank[7] = 2, \qquad rank[1] = 4$$

Phân biệt hai chiều:

Phép tra	Ý nghĩa
$SA[2] = 7$	Từ hạng 2 tìm được vị trí bắt đầu 7
$rank[7] = 2$	Từ vị trí bắt đầu 7 tìm được hạng 2

Nhờ `rank`, ta không phải tìm kiếm tuyến tính trong suffix array mỗi khi có query.

5. Mảng LCP chỉ lưu thông tin giữa hai hậu tố kề nhau

Ta dùng quy ước:

$$LCP[k] = \text{LCP}\,(S[SA[k-1]..], S[SA[k]..]), \qquad k \geq 1$$

Đặt $LCP[0] = 0$.

Nghĩa là: **ô $LCP[k]$ đo độ giống nhau giữa hậu tố ở hạng $k-1$ và hậu tố ở hạng k.**

Ví dụ:

Cặp hạng	Hai hậu tố	Tiền tố chung	Ô LCP
1 và 2	`aban$` , `an$`	`a`	$LCP[2] = 1$
2 và 3	`an$` , `anaban$`	`an`	$LCP[3] = 2$
3 và 4	`anaban$` , `ananaban$`	`ana`	$LCP[4] = 3$

Toàn bộ mảng là:

$$LCP = [0, 0, 1, 2, 3, 0, 3, 0, 1, 2]$$

Vấn đề là: LCP chỉ lưu cho **các cặp kề nhau**, trong khi người dùng có thể hỏi bất kỳ hai vị trí nào.

6. Hai hậu tố không kề nhau thì lấy min ở giữa

Quay lại truy vấn:

$$\text{LCE}(1, 7)$$

Ta biết:

$$rank[1] = 4, \qquad rank[7] = 2$$

Hai hậu tố cần so sánh nằm ở hạng **2 và 4**:

Hạng	Hậu tố
2	**an**`$`
3	**an**`aban$`
4	**an**`anaban$`

Giữa chúng có hai cặp kề nhau:

- Hạng 2 với 3 giống nhau 2 ký tự: $LCP[3] = 2$.
- Hạng 3 với 4 giống nhau 3 ký tự: $LCP[4] = 3$.

Muốn tiền tố giống nhau được duy trì từ hàng đầu đến hàng cuối, nó phải vượt qua **cả hai cặp**. Cặp chỉ giống nhau 2 ký tự sẽ giới hạn kết quả:

$$\text{LCE}(1, 7) = \min(2, 3) = \boxed{2}$$

Đây chính là chỗ RMQ xuất hiện.

Tổng quát, nếu hai hậu tố có hạng $p < q$:

$$\boxed{\text{LCP}(S[SA[p]..], S[SA[q]..]) = \min LCP[p+1..q]}$$

Quan hệ này là cơ sở để dùng RMQ trên mảng LCP. Algorithms for ...

7. Vì sao lấy min lại đúng?

Có hai ý cần hiểu.

Nếu mọi cặp kề nhau đều giống ít nhất h ký tự, thì các hậu tố trong cả dãy đều có cùng h ký tự đầu. Hai hậu tố ở hai đầu cũng giống ít nhất h ký tự.

Hai hậu tố ở hai đầu không thể giống nhiều hơn mức min đó. Vì các hậu tố đã được sắp từ điển, những chuỗi có cùng một tiền tố luôn nằm liền nhau.

Ví dụ, nếu hai đầu đều bắt đầu bằng `ana`, thì mọi hậu tố nằm giữa chúng cũng phải bắt đầu bằng `ana`. Không thể có một cặp ở giữa chỉ giống nhau `an`.

Vì vậy, độ dài tiền tố chung của hai đầu **bằng đúng min của các LCP ở giữa**.

Điều kiện sắp xếp từ điển là phần quan trọng của lập luận này.

8. Vì sao đoạn RMQ là $[p+1, q]$, không phải $[p, q]$?

Hai hậu tố cần hỏi nằm ở hạng 2 và 4. Những cặp nối chúng là:

Cặp cần xét	Ô chứa thông tin
Hạng 2 với hạng 3	$LCP[3]$
Hạng 3 với hạng 4	$LCP[4]$

Do đó cần đoạn:

$$LCP[3..4]$$

Còn $LCP[2]$ so sánh **hạng 1 với hạng 2**, là cặp nằm ngoài phạm vi cần xét.

Đó là lý do phải bắt đầu từ $p+1$.

9. Ghép với RMQ bạn đã học

Với hai vị trí i, j, đặt:

$$p = \min(rank[i], rank[j]), \qquad q = \max(rank[i], rank[j])$$

Sau đó tìm min trên:

$$LCP[p+1..q]$$

Nhưng nhớ rằng code RMQ trước đây của bạn trả **vị trí cực tiểu**. Vì thế phải làm hai bước:

$$t = \mathrm{RMQ}_{LCP}(p+1, q)$$

$$\boxed{\mathrm{LCE}(i, j) = LCP[t]}$$

Trong ví dụ:

$$p = 2, \quad q = 4$$

$$\mathrm{RMQ}_{LCP}(3, 4) = 3$$

vì $LCP[3] = 2 < LCP[4] = 3$. Cuối cùng:

$$\mathrm{LCE}(1,7) = LCP[3] = 2$$

Số 3 là vị trí trong mảng LCP; số 2 mới là độ dài LCE.

Pseudocode:

```
LCE(i, j):
    nếu i == j:
        trả số ký tự văn bản gốc còn lại từ vị trí i

    p = rank[i]
    q = rank[j]

    nếu p > q:
        đổi chỗ p và q

    t = RMQ_trên_LCP(p + 1, q)
    return LCP[t]
```

10. Dùng cấu trúc RMQ nào thì LCE hưởng chi phí tương ứng

Sau khi đã có `SA`, `rank` và `LCP`:

Cách xử lý RMQ trên LCP	Tiền xử lý thêm	Một truy vấn LCE
Mỗi lần quét đoạn LCP	Không xây cấu trúc	$O(n)$ tệ nhất
Sparse Table	$O(n \log n)$	$O(1)$
Cấu trúc RMQ tuyến tính vừa học	$O(n)$	$O(1)$

Nếu tính toàn bộ quá trình, phải cộng cả chi phí xây suffix array, rank và LCP. Video giả sử dùng cách xây các mảng đó trong $O(n)$; khi ấy RMQ có tiền xử lý $P(n)$, query $Q(n)$ sẽ cho LCE có tiền xử lý $O(n) + P(n)$, query $O(1) + Q(n)$. www2.wild-inter...

Không phải mọi cách tự xây suffix array đều tuyến tính; chi phí thực tế còn phụ thuộc thuật toán xây SA được chọn.

11. Quan hệ với LCA trong video

Video còn nhắc một đường giải khác: nếu dùng **suffix tree**, tiền tố chung của hai hậu tố chính là phần đường đi chung từ gốc đến LCA của hai lá tương ứng.

Khi đó:

$$\mathrm{LCE}(i,j) = \text{string-depth của LCA hai lá}$$

String-depth đếm số ký tự trên đường đi, không chỉ đếm số cạnh, vì một cạnh suffix tree có thể mang nhiều ký tự.

Suffix array + LCP + RMQ cung cấp cách giải dựa trên mảng cho cùng loại truy vấn này.

static.aminer.org

Một ứng dụng trực tiếp: muốn kiểm tra hai đoạn dài d, bắt đầu tại i, j, có bằng nhau không — giả sử cả hai đoạn đều nằm trong văn bản — chỉ cần kiểm tra:

$$\mathrm{LCE}(i, j) \geq d$$

Trong ví dụ, $\mathrm{LCE}(1, 7) = 2$, nên hai đoạn dài 2 đều là **an** . Đây là cách biến một thao tác so sánh chuỗi thành một truy vấn RMQ và vài phép tra mảng.

You said:

tui thấy phần ứng dụng LCE này nó kiểu không phải ứng dụng quan trọng lắm mà còn phức tạp thì bỏ được không

Worked for 47s

ChatGPT said:

Bỏ được. Với seminar của bạn, mình nghiêng về bỏ LCE khỏi phần trình bày chính. Yêu cầu ban đầu chỉ cần có ứng dụng hoặc bài tập ứng dụng, không bắt buộc phải có LCE.

LCE có vai trò trong xử lý chuỗi, nhưng để trình bày cho người chưa học, bạn phải giải thích thêm **suffix array, rank, LCP**, rồi mới quay về RMQ. Phần kiến thức nền này tốn thời gian so với mục tiêu seminar hiện tại.

Bạn có thể thay bằng phần **bài tập ứng dụng và lựa chọn CTDL**, khoảng 5 phút:

Tình huống	Nội dung trình bày
Mảng cố định, nhiều truy vấn min	Một bài Static RMQ: chạy ví dụ bằng Sparse Table
Có cập nhật giá trị xen kẽ	Đổi điều kiện bài trên, giải thích vì sao chuyển sang Segment Tree
Muốn có ứng dụng trên cây	Giới thiệu ngắn LCA qua Euler tour + RMQ, nếu còn thời gian

Hai tình huống đầu đã đủ đáp ứng mục "bài tập ứng dụng", đồng thời gắn trực tiếp với phần so sánh và benchmark của bạn. LCA có thể là phần mở rộng tùy chọn; Cartesian tree vẫn giữ để giải thích cách tối ưu RMQ, dù bạn không trình bày ứng dụng LCA riêng.

Thời gian tiết kiệm từ LCE nên dành cho những phần người nghe cần hiểu chắc hơn:

- Vì sao Sparse Table trả lời bằng hai đoạn chồng lấn.
- Vì sao các khối cùng hình dạng Cartesian tree dùng chung bảng tra.
- Benchmark tính cả build và query, rồi đánh giá khi nào nên chọn từng cấu trúc.

Bạn có thể chuyển LCE thành **một dòng đọc thêm ở cuối tài liệu**, không cần dành riêng một phần để giảng.

You said:

BenderFa00-lca.pdf
PDF

1 selection

giờ bạn trình bày cho tui về nội dung paper

Worked for 3m 21s

ChatGPT said:

Paper **"The LCA Problem Revisited" — Michael A. Bender và Martín Farach-Colton (2000)** trình bày một cách giải LCA với:

$$\boxed{\text{Tiền xử lý } O(n), \quad \text{query } O(1), \quad \text{bộ nhớ } O(n)}$$

Sau đó, tác giả dùng kết quả này để giải cả **RMQ tổng quát** với cùng độ phức tạp.

Mình trình bày theo bản PDF bạn gửi: BenderFa00-lca.pdf

Điểm cần phân biệt với video vừa học: **paper dùng tính chất ± 1 của mảng độ sâu**, thay vì phân loại từng khối bằng hình dạng Cartesian tree.

1. Đóng góp của paper là gì?

Trước paper này, kết quả LCA với $O(n)$ tiền xử lý và $O(1)$ query đã tồn tại. Tuy nhiên, các thuật toán trước được xem là khó hiểu và khó triển khai.

Mục tiêu của Bender–Farach-Colton là **trình bày lại, đơn giản hóa một hướng giải đã có**, để kết quả tối ưu trở nên dễ sử dụng hơn.

Mạch lập luận của paper:

Mục	Nội dung
§2	Chuyển LCA trên cây thành RMQ trên mảng độ sâu
§3	Giải RMQ bằng Sparse Table

Mục	Nội dung
§4	Tận dụng tính chất ± 1 để giảm tiền xử lý xuống $O(n)$
§5	Chuyển RMQ tổng quát thành LCA bằng Cartesian tree

Phần bạn nên tập trung nhất là **§2 và §4**. Sparse Table và Cartesian tree bạn đã học rồi.

2. §2: Chuyển LCA thành RMQ bằng cách duyệt cây

Trước đây bạn học:

> Trên Cartesian tree của mảng, RMQ tương ứng với LCA.

Ở đây paper đi theo **chiều ngược lại**:

> Cho một cây bất kỳ, biến câu hỏi LCA thành một câu hỏi min trên mảng.

Xét cây:

Ta biết:

$$\mathrm{LCA}(3,4) = 1$$

Bây giờ cần tìm đáp án đó bằng RMQ.

Bước đầu: ghi lại đường đi DFS

Ta bắt đầu ở gốc. Mỗi lần đi qua một cạnh, dù đi xuống hay quay lên, đều ghi node vừa đến.

Nếu thăm nhánh trái trước, dãy node là:

$$E = [0, 1, 3, 1, 4, 1, 0, 2, 0]$$

Đây là cách ghi **Euler tour** trong paper.

Đừng chỉ ghi mỗi node một lần. Sau khi thăm node 3, ta quay về 1 nên phải ghi lại node 1.

Đồng thời, ghi độ sâu tương ứng:

$$L = [0, 1, 2, 1, 2, 1, 0, 1, 0]$$

Vị trí trong tour	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Node $E[i]$	0	1	3	1	4	1	0	2	0
Độ sâu $L[i]$	0	1	2	1	2	1	0	1	0

Paper còn lưu mảng R: **vị trí xuất hiện đầu tiên của mỗi node trong tour**.

Ví dụ:

$$R[3] = 2, \qquad R[4] = 4, \qquad R[2] = 7$$

Ở phần minh họa này mình dùng chỉ số từ 0, còn paper dùng từ 1.

3. Tìm LCA bằng min độ sâu

Muốn tìm $\mathrm{LCA}(3, 4)$:

- Node 3 xuất hiện đầu tiên tại vị trí 2.
- Node 4 xuất hiện đầu tiên tại vị trí 4.
- Xét đoạn độ sâu $L[2..4]$.

Ta có:

$$L[2..4] = [2, 1, 2]$$

Min là 1, nằm tại **vị trí 3 của tour**. Tra lại:

$$E[3] = 1$$

Vậy:

$$\mathrm{LCA}(3,4) = 1$$

Công thức tổng quát:

$$a = \min(R[u], R[v]), \qquad b = \max(R[u], R[v])$$

$$\boxed{\mathrm{LCA}(u,v) = E[\mathrm{RMQ}_L(a,b)]}$$

Phải phân biệt ba thứ:

Đại lượng	Nó trả về gì?
$\mathrm{RMQ}_L(a,b)$	Vị trí trong tour
$L[\mathrm{RMQ}_L(a,b)]$	Độ sâu nhỏ nhất
$E[\mathrm{RMQ}_L(a,b)]$	**Node LCA cần tìm**

Vì sao cách này đúng?

Để đi từ nhánh chứa u sang nhánh chứa v, DFS phải quay về LCA của chúng.

Trong khoảng giữa hai lần thăm đầu tiên đó:

- DFS có đi qua LCA.
- DFS chưa cần đi lên cao hơn LCA, vì vẫn đang xử lý các nhánh trong cây con của LCA.

Vì vậy, node **nông nhất** xuất hiện trong đoạn tour ấy chính là LCA.

4. Tính chất đặc biệt: hai độ sâu liên tiếp luôn chênh đúng 1

Nhìn lại:

$$L = [0,1,2,1,2,1,0,1,0]$$

Mỗi lần đi qua một cạnh:

- Đi xuống: độ sâu tăng 1.
- Đi lên: độ sâu giảm 1.

Do đó:

$$\boxed{L[i+1] - L[i] \in \{-1, +1\}}$$

Đây là ± 1 **RMQ**: RMQ trên một mảng mà hai phần tử liên tiếp chênh đúng $+1$ hoặc -1.

Mảng RMQ thông thường như:

$$[7, 4, 9, 2]$$

không có tính chất này. Nhưng **mảng độ sâu Euler tour thì có**.

Tính chất này giúp tác giả đếm số loại khối dễ hơn rất nhiều.

Ngoài ra, cây n node có $n - 1$ cạnh. Mỗi cạnh được đi qua hai lần, cộng lần ghi gốc ban đầu, nên tour dài:

$$N = 2n - 1$$

Từ đây, mình dùng N để chỉ độ dài mảng độ sâu.

5. §3: Sparse Table là lời giải trung gian

Paper trước hết dùng Sparse Table để giải RMQ:

$$\langle O(N \log N), O(1) \rangle$$

Bạn đã học ý tưởng này:

- Lưu đáp án cho các đoạn dài $1, 2, 4, 8, \ldots$.
- Query bằng hai đoạn phủ kín đoạn hỏi.

Dùng ký hiệu quen thuộc của mình:

$$st[k][i] = \text{better}\,(st[k-1][i], st[k-1][i + 2^{k-1}])$$

Với đoạn đóng $[l, r]$, có thể chọn:

$$k = \lfloor \log_2(r - l + 1) \rfloor$$

rồi ghép hai ứng viên:

$$st[k][l], \qquad st[k][r - 2^k + 1]$$

Vai trò của Sparse Table ở đây là **một thành phần để xây lời giải tốt hơn**, không phải điểm kết thúc của paper.

6. §4: Chia khối, nhưng phân loại bằng mẫu tăng/giảm

Phần chia khối giống hướng bạn vừa học:

- Chia mảng thành các khối nhỏ.

- Lưu min của mỗi khối vào mảng đại diện.
- Xây Sparse Table trên mảng đại diện.
- Query khác khối được ghép từ hai phần biên và các khối đầy đủ ở giữa.

Điểm mới nằm ở cách xử lý **bên trong khối**.

Xét:

$$X = [7, 8, 7, 6]$$

$$Y = [12, 13, 12, 11]$$

Hai khối không bằng nhau, nhưng mọi phần tử của Y đều lớn hơn phần tử tương ứng của X đúng 5 đơn vị.

Vì thế, mọi so sánh lớn nhỏ và mọi vị trí cực tiểu đều giống nhau.

Ví dụ:

Đoạn nội khối	RMQ trên X	RMQ trên Y
$[0, 2]$	0	0
$[0, 3]$	3	3
$[1, 2]$	2	2

Ta có thể dùng chung bảng đáp án cho chúng.

Chuẩn hóa bằng cách trừ phần tử đầu

Với X, trừ 7 khỏi mọi phần tử:

$$[7, 8, 7, 6] - 7 = [0, 1, 0, -1]$$

Với Y, trừ 12:

$$[12, 13, 12, 11] - 12 = [0, 1, 0, -1]$$

Hai khối trở thành đúng một dãy.

Phép trừ cùng một hằng số **không thay đổi vị trí cực tiểu**, nên chuẩn hóa không làm mất thông tin RMQ nội khối.

7. Vì sao chỉ cần $b - 1$ bit để xác định một khối?

Sau chuẩn hóa, phần tử đầu tiên luôn bằng 0.

Các phần tử tiếp theo chỉ có hai khả năng so với phần tử trước:

- Tăng 1.
- Giảm 1.

Ví dụ:

$$[0, 1, 0, -1]$$

có các bước:

$$+1,\ -1,\ -1$$

Ta có thể quy ước:

- Tăng 1 ghi bit `1` .
- Giảm 1 ghi bit `0` .

Mã của khối là:

`100`

Từ mã này, bắt đầu ở 0, ta khôi phục được toàn bộ khối:

Bước	Giá trị nhận được
Bắt đầu	0
`1` : tăng 1	1
`0` : giảm 1	0
`0` : giảm 1	−1

Khối có b phần tử thì chỉ có $b - 1$ **bước chuyển**, nên có:

$$\boxed{2^{b-1}}$$

mẫu tăng/giảm có thể có.

Mỗi mẫu dùng một bảng tra chung cho mọi đoạn nội khối. Mỗi khối thực tế chỉ cần lưu mã mẫu của nó.

8. Vì sao paper chọn $b \approx \frac{1}{2}\log_2 N$?

Số mẫu khi đó là:

$$2^{b-1} \le 2^b = 2^{\frac{1}{2}\log_2 N} = \sqrt{N}$$

Vậy chỉ cần $O(\sqrt{N})$ bảng nội khối.

Mỗi bảng lưu $O(b^2)$ đáp án. Dùng quy hoạch động như phần "bảng mọi đoạn", cũng xây được mỗi bảng trong $O(b^2)$.

Tổng chi phí tạo các bảng là:

$$O(2^{b-1}b^2) = O(\sqrt{N}\log^2 N) = o(N)$$

Còn mảng đại diện có khoảng N/b phần tử, nên Sparse Table trên đó xây trong:

$$O\left(\frac{N}{b}\log\frac{N}{b}\right) = O(N)$$

Tính min và mã mẫu của tất cả khối tốn thêm $O(N)$. Tổng cộng:

$$\boxed{\text{Tiền xử lý } O(N), \quad \text{query } O(1)}$$

Bộ nhớ cũng là $O(N)$ word.

Hệ số $1/2$ ở đây không mâu thuẫn với hệ số $1/4$ bạn vừa học. Hai cách đang đếm loại khối khác nhau:

Nội dung	Paper Bender–Farach-Colton	Hướng trong video trước
Mảng đang xử lý	Mảng có tính chất ± 1	Mảng tổng quát
Mã loại khối	Mẫu tăng/giảm	Hình dạng Cartesian tree
Độ dài mã	$b-1$ bit	$2b$ bit
Chặn số loại	2^{b-1}	4^b
Kích thước khối thường chọn	$\frac{1}{2}\log N$	$\frac{1}{4}\log N$

Trong cả hai trường hợp, lựa chọn kích thước khối giúp số loại bị chặn bởi khoảng $\sqrt{N}$.

9. §5: Từ đây giải được cả RMQ tổng quát

Đến cuối §4, ta đã giải được:

- ± 1 RMQ trong $O(N)$ tiền xử lý, $O(1)$ query.
- LCA trên cây bất kỳ, vì Euler tour chuyển nó thành ± 1 RMQ.

Bây giờ lấy một mảng tổng quát A:

1. Dựng Cartesian tree C của A.
2. Tiền xử lý LCA trên C bằng thuật toán vừa giải.
3. Khi hỏi RMQ của $[l, r]$, trả về LCA của node l, r.

Đây chính là quan hệ bạn đã học:

$$\boxed{\text{RMQ}_A(l, r) = \text{LCA}_C(l, r)}$$

Vì Cartesian tree có thể dựng trong $O(n)$, toàn bộ tiền xử lý vẫn là $O(n)$.

Paper giải thích cách dựng tuyến tính bằng việc duy trì đường ngoài cùng bên phải của cây: mỗi node chỉ đi vào và rời đường này tối đa một lần. Stack đơn điệu là cách triển khai thuận tiện cho lập luận đó.

Có bị lập luận vòng không?

Không, vì thứ tự phụ thuộc là:

1. **Giải trực tiếp ± 1 RMQ** bằng bảng tra mẫu tăng/giảm.
2. Dùng kết quả ấy để **giải LCA**.
3. Dùng LCA đã giải được để **giải RMQ tổng quát**.

Ta không giả sử trước rằng RMQ tổng quát đã có lời giải tuyến tính.

10. Bạn nên dùng paper này thế nào trong seminar?

Paper phù hợp làm nguồn cho ba nội dung:

- **Ứng dụng:** tìm LCA bằng RMQ trên độ sâu Euler tour.
- **Tối ưu:** từ Sparse Table đến chia khối và bảng tra mẫu ± 1.
- **Cơ sở lý thuyết:** quan hệ hai chiều giữa RMQ và LCA.

Phạm vi của paper là **cây/mảng tĩnh**. Nó tập trung vào thuật toán và chứng minh độ phức tạp, không có phần benchmark đo thời gian chạy để bạn trích kết luận nhanh hơn CTDL khác bao nhiêu lần.

Khi đọc bản PDF, cũng nên đối chiếu công thức Sparse Table ở trang 5 với công thức đã học: dòng truy hồi có một số lỗi ký hiệu. Đặc biệt, **nửa phải bắt đầu tại** $i + 2^{k-1}$, và khi chọn ứng viên phải so sánh **giá trị mảng tại các chỉ số**, không so sánh trực tiếp giá trị với chỉ số.

You said:

cái +-1 đó chỉ áp dụng cho mảng có tính chất đặc biệt thôi hay sao

ChatGPT said:

Đúng. Thuật toán ± 1 RMQ chỉ áp dụng trực tiếp cho mảng mà hai phần tử liên tiếp chênh đúng $+1$ hoặc -1.

Ví dụ:

$$[3, 4, 3, 2]$$

Các bước thay đổi là:

$$+1,\ -1,\ -1$$

Mảng này thỏa tính chất ± 1. Còn:

$$[7, 4, 9, 2]$$

thì không thỏa, vì có những bước thay đổi như $-3, +5, -7$.

Nhưng paper vẫn giải được RMQ trên mảng bất kỳ bằng cách chuyển đổi bài toán:

1. Từ mảng ban đầu, dựng **Cartesian tree**. RMQ trên mảng trở thành LCA trên cây.
2. Duyệt Euler trên cây, tạo **mảng độ sâu**. LCA trở thành RMQ trên mảng độ sâu.
3. Mảng độ sâu luôn có tính chất ± 1, nên dùng được thuật toán đặc biệt.

Vì sao mảng độ sâu luôn thỏa? Mỗi lần đi qua một cạnh:

- Đi từ cha xuống con: độ sâu **tăng 1**.
- Đi từ con lên cha: độ sâu **giảm 1**.

Các số chênh ± 1 là độ sâu trong cây, không phải giá trị của mảng ban đầu.

Vậy có hai trường hợp:

Đầu vào	Cách dùng thuật toán
Mảng đã có tính chất ± 1	Áp dụng trực tiếp
Mảng bất kỳ	Chuyển qua Cartesian tree và Euler tour, rồi áp dụng

Các bước chuyển đổi đều tốn $O(n)$, nên tổng tiền xử lý vẫn là $O(n)$; mỗi query chỉ thêm vài phép tra mảng nên vẫn $O(1)$.

You said:

vậy 2 cách này đều như nhau hay có tối ưu hơn trong trường hợp cụ thể

Worked for 1m 18s >

ChatGPT said:

Hai hướng có cùng bậc độ phức tạp, nhưng không nhất thiết chạy nhanh hay tốn bộ nhớ như nhau. Lợi thế phụ thuộc vào đầu vào bạn đang có.

Mình đang so sánh:

- **Hướng trực tiếp:** phân loại khối bằng hình dạng Cartesian tree, như video bạn học.
- **Hướng Bender–Farach-Colton:** giải ± 1 RMQ; với mảng tổng quát thì chuyển qua Cartesian tree và Euler tour trước.

Cả hai đều đạt $O(n)$ tiền xử lý, $O(1)$ query và $O(n)$ word bộ nhớ.

1. Nếu đầu vào là mảng tổng quát

Ví dụ:

$$A = [7, 4, 9, 2]$$

Hướng trực tiếp xử lý ngay các khối của mảng này.

Còn hướng dùng ± 1 cần:

1. Dựng Cartesian tree.
2. Tạo Euler tour và mảng độ sâu.
3. Xử lý RMQ trên mảng độ sâu.

Mảng ban đầu dài n, nhưng mảng độ sâu dài $2n - 1$. Các bước và mảng phụ này tạo thêm chi phí, dù tổng vẫn là $O(n)$.

Hướng trực tiếp tránh được phần chuyển đổi đó, nên có lợi thế về chi phí trung gian khi xử lý mảng tổng quát. Đây cũng là một động lực của paper Fischer–Heun năm 2006. static.aminer.org

2. Nếu đầu vào đã có tính chất ± 1

Ví dụ:

$$[3, 4, 3, 2]$$

Ta có thể áp dụng ngay thuật toán ± 1, không cần dựng Cartesian tree hay Euler tour.

Việc xác định loại khối cũng đơn giản: chỉ đọc các bước **tăng 1 hoặc giảm 1**. Không cần tính loại theo hình dạng Cartesian tree.

Trường hợp này, hướng ± 1 khai thác trực tiếp cấu trúc sẵn có của dữ liệu.

3. Nếu bài toán ban đầu là LCA trên cây

Euler tour tự tạo ra mảng độ sâu có tính chất ± 1. Vì vậy, hướng Bender–Farach-Colton rất phù hợp:

> Đã có cây → tạo mảng độ sâu → dùng ± 1 RMQ.

Bạn vẫn có thể dùng RMQ tổng quát trên mảng độ sâu, nhưng sẽ không tận dụng riêng tính chất đặc biệt ấy.

Đầu vào	Hướng phù hợp để cân nhắc
Mảng tổng quát, truy vấn tĩnh	RMQ trực tiếp theo loại Cartesian tree
Mảng đã có tính chất ± 1	Thuật toán ± 1 trực tiếp
Cây cần trả lời LCA	Euler tour + ± 1 RMQ
Mảng có cập nhật thường xuyên	Hai cấu trúc tĩnh trên không còn phù hợp; cân nhắc Segment Tree

4. Có bằng chứng thực nghiệm về khác biệt không?

Có. Paper Fischer–Heun năm 2006 báo cáo phương pháp trực tiếp của họ trả lời query nhanh khoảng **2 lần** so với implementation Berkman–Vishkin dùng hướng quy giảm về ± 1, đồng thời dùng ít bộ nhớ hơn. Thí nghiệm chạy trên Athlon XP3000, RAM 2 GB, Linux. static.aminer.org

Đó là kết quả của những implementation và môi trường cụ thể. Để chọn cho chương trình của bạn, cần đo **build, query, bộ nhớ và tổng thời gian**, trên đúng loại đầu vào dự kiến.

You said:

theoretical_and_...
PDF

1 selection

giờ trình bày cho tui về nội dung paper này

Worked for 4m 30s

ChatGPT said:

Paper này đáng đọc vì nó trả lời hai câu hỏi rất sát với seminar của bạn:

- **Có thể giải RMQ tổng quát với build $O(n)$, query $O(1)$, nhưng dùng ít bộ nhớ hơn cách đi qua LCA không?**
- **Thuật toán có độ phức tạp tốt nhất có thực sự chạy nhanh nhất không?**

Mình sẽ nối với phần Cartesian tree và exhaustive tabulation bạn đã học, rồi giải thích những đóng góp mới của paper.

Nguồn: Fischer & Heun, *Theoretical and Practical Improvements on the RMQ-Problem, with Applications to LCA and LCE* (2006), đặc biệt §3–5.
theoretical_and_practical_improvements_on_the_rmq_problem_with_applications.pdf

1. Paper muốn cải thiện điều gì?

Ta vẫn xét bài toán RMQ trên **mảng tĩnh**:

$$\mathrm{RMQ}_A(l, r) = \text{chỉ số của phần tử nhỏ nhất trong } A[l..r].$$

Nếu có nhiều phần tử cùng nhỏ nhất, chọn vị trí bên trái nhất.

Ở cách bạn vừa học từ Bender–Farach-Colton, để giải RMQ tổng quát, ta có thể đi theo chuỗi:

$$\text{Mảng } A \longrightarrow \text{Cartesian tree} \longrightarrow \text{LCA} \longrightarrow \text{Euler tour} \longrightarrow \pm 1 \text{ RMQ}.$$

Cách đó đã đạt:

$$\text{Build } O(n), \qquad \text{Query } O(1).$$

Nhưng nó cần thêm các cấu trúc trung gian. Đặc biệt, Euler tour của cây n đỉnh có độ dài $2n - 1$.

Fischer–Heun đặt vấn đề:

> Ta có thể xử lý trực tiếp mảng ban đầu, giữ nguyên độ phức tạp, đồng thời tránh các cấu trúc trung gian này không?

Câu trả lời là có.

Lưu ý: đóng góp ở đây là **cách tổ chức thuật toán, bộ nhớ và hiệu năng thực tế**; không phải lần đầu tiên người ta đạt $O(n)$ build và $O(1)$ query.

---

2. Ý tưởng cốt lõi: các khối khác giá trị vẫn có thể dùng chung đáp án RMQ

Bạn đã học ý tưởng chia mảng thành những khối nhỏ. Vấn đề là nếu xây bảng mọi đoạn riêng cho từng khối thì vẫn tốn quá nhiều công.

Paper khai thác định lý sau:

> Hai mảng cùng độ dài có đáp án RMQ giống nhau trên mọi đoạn khi và chỉ khi chúng có cùng

hình dạng Cartesian tree chuẩn.

"Đáp án giống nhau" ở đây là **cùng vị trí tương đối**, không phải cùng giá trị.

Ví dụ:

$$X = [9, 4, 1, 6], \qquad Y = [8, 5, 3, 7].$$

Dùng chỉ số bắt đầu từ 0:

Đoạn hỏi	Đáp án trên X	Đáp án trên Y
$[0, 1]$	Vị trí 1, giá trị 4	Vị trí 1, giá trị 5
$[0, 3]$	Vị trí 2, giá trị 1	Vị trí 2, giá trị 3
$[1, 3]$	Vị trí 2, giá trị 1	Vị trí 2, giá trị 3
$[3, 3]$	Vị trí 3	Vị trí 3

Hai khối này có cùng cấu trúc:

- Cực tiểu toàn khối nằm ở vị trí 2.
- Trong phần bên trái $[0, 1]$, cực tiểu nằm ở vị trí 1.
- Phần bên phải chỉ có vị trí 3.

Vì vậy, chúng dùng chung một bảng đáp án.

Chẳng hạn bảng chung lưu:

$$P[\text{loại khối}][0][1] = 1.$$

Khi gặp khối X, trả vị trí tương đối 1. Khi gặp khối Y, cũng trả vị trí tương đối 1.

Ta chỉ xây bảng cho mỗi loại cấu trúc một lần, rồi cho nhiều khối dùng chung.

Để hiểu chiều ngược của định lý: nếu biết toàn bộ đáp án RMQ, ta biết gốc cây từ truy vấn toàn mảng; tiếp tục hỏi toàn bộ phần trái và phần phải thì khôi phục được hai cây con. Vì vậy, toàn bộ đáp án RMQ cũng xác định được Cartesian tree.

---

3. Thuật toán tổng thể: hai tầng xử lý mà bạn đã gặp trong video

Paper dùng s để chỉ kích thước khối; mình tiếp tục gọi là b cho quen.

Có hai tầng:

Tầng	Thông tin lưu	Mục đích
Giữa các khối	Sparse Table trên cực tiểu của từng khối	Xử lý những khối được lấy trọn
Bên trong khối	Bảng tra chung theo loại Cartesian tree	Xử lý một đoạn nhỏ trong khối

Các bảng chính trong paper là:

Ký hiệu	Ý nghĩa
$A'[t]$	Giá trị nhỏ nhất của khối t
$B[t]$	Chỉ số của giá trị đó trong mảng gốc
$T[t]$	Mã loại của khối t
M	Sparse Table trên A'
P	Bảng đáp án nội khối dùng chung

Với truy vấn đi qua nhiều khối, ta vẫn chia thành:

$$\boxed{\text{Đuôi khối trái}} + \boxed{\text{Các khối đầy đủ ở giữa}} + \boxed{\text{Đầu khối phải}}.$$

Hai phần biên dùng T, P; phần giữa dùng M, B. Cuối cùng so sánh tối đa ba ứng viên.

Do đó:

$$\boxed{\text{Query } O(1)}.$$

Nếu hai đầu nằm cùng khối thì chỉ tra một lần trong P.

Như vậy, **phần exhaustive tabulation bạn đã học chính là nền tảng để hiểu thuật toán trong paper này**. Phần đáng chú ý tiếp theo là đếm loại khối và tính mã loại đủ nhanh.

---

4. Paper đếm số loại khối chính xác hơn bằng số Catalan

Trước đây ta dùng chặn trên đơn giản:

$$\text{Số loại khối} \le 4^b.$$

Paper dùng kết quả chính xác hơn: số hình dạng cây nhị phân có b nút là số Catalan:

$$C_b = \frac{1}{b+1}\binom{2b}{b}.$$

Ví dụ:

Độ dài khối b	Số loại C_b
1	1
2	2
3	5
4	14
5	42

Với khối dài 4, các giá trị có thể nhận vô số khả năng, nhưng chỉ có **14 kiểu hành vi RMQ**.

Mỗi loại cần lưu đáp án cho:

$$\frac{b(b+1)}{2} = O(b^2)$$

đoạn. Do đó, kích thước bảng chung là:

$$O(C_b b^2).$$

Sử dụng xấp xỉ Catalan:

$$C_b = \Theta\left(\frac{4^b}{b^{3/2}}\right),$$

ta được:

$$\boxed{|P| = O(4^b\sqrt{b})}.$$

Đây là chặn chặt hơn so với $O(4^b b^2)$ mà mình dùng khi giải thích video.

Paper chọn:

$$b = \frac{1}{4}\log_2 n$$

— bỏ qua làm tròn khi phân tích.

Khi đó:

$$4^b = 4^{\frac{1}{4}\log_2 n} = \sqrt{n}.$$

Suy ra:

$$|P| = O\left(\sqrt{n \log n}\right) = O(n).$$

Còn số khối là:

$$m \approx \frac{4n}{\log_2 n}.$$

Sparse Table trên m cực tiểu khối tốn:

$$O(m \log m) = O(n).$$

Vậy cả hai tầng đều nằm trong ngân sách tuyến tính.

---

5. Điểm kỹ thuật quan trọng ở §3.1: tìm loại của mỗi khối trong $O(b)$

Có ít loại khối chưa đủ. Ta còn phải xác định được mỗi khối thuộc loại nào.

Nếu làm quá chậm ở bước này thì tổng build không còn $O(n)$.

Paper dùng:

- Một stack lưu bằng mảng.
- Quá trình mô phỏng dựng Cartesian tree.
- Các **ballot numbers** để chuyển cấu trúc thành mã số.

Bạn có thể hiểu thành hai bước.

Bước A: nhận diện cấu trúc bằng quá trình push/pop.

Với:

$$X = [9, 4, 1, 6],$$

quét từ trái sang phải bằng stack tăng dần:

Đọc phần tử	Thao tác
9	Push 9
4	Pop 9, push 4
1	Pop 4, push 1
6	Push 6

Số lần pop tương ứng:

$$[0, 1, 1, 0].$$

Khối:

$$Y = [8, 5, 3, 7]$$

cũng tạo ra cùng diễn biến push/pop. Đó là dấu hiệu của cùng cấu trúc Cartesian tree.

Mỗi phần tử được push một lần và pop tối đa một lần, nên tổng thời gian là:

$$O(b).$$

Bước B: chuyển cấu trúc thành một mã loại.

Ballot numbers giúp tính số cấu trúc có thể xuất hiện trong từng nhánh lựa chọn, từ đó đánh số các loại liên tiếp:

$$0, 1, \ldots, C_b - 1.$$

Với $b = 4$, mã chỉ cần nằm trong:

$$0, \ldots, 13.$$

Bạn chưa cần học hết chứng minh tổ hợp để trình bày ý tưởng. Điều cần nắm là:

> Paper tính trực tiếp mã loại bằng mảng và stack, không cần tạo cây với các nút và con trỏ.

Vì mỗi khối tốn $O(b)$, toàn bộ các khối tốn:

$$\frac{n}{b} \cdot O(b) = O(n).$$

Đây là một bước quan trọng để hoàn thiện chứng minh build tuyến tính.

---

6. Bộ nhớ được cải thiện như thế nào?

Hai cách cùng có bộ nhớ $O(n)$, nhưng **hệ số trước** n khác nhau.

Theo Table 1 của paper, bộ nhớ phụ, tính theo *word*, có các chặn đơn giản hóa:

Phương pháp được paper so sánh	Bộ nhớ phụ
Berkman–Vishkin, qua Cartesian tree và Euler tour	$9n + O(\sqrt{n}\,\log^2 n)$
Fischer–Heun trực tiếp	$4n + O(\sqrt{n \log n})$

Các con số này không tính mảng đầu vào. Bảng cũng chưa tính phần bộ nhớ để xây Cartesian tree của phương pháp cũ.

Vì sao giảm được nhiều?

Cách cũ cần ba mảng trung gian:

- Danh sách đỉnh trong Euler tour: $2n - 1$ phần tử.
- Độ sâu tương ứng: $2n - 1$ phần tử.
- Vị trí đại diện của mỗi đỉnh: n phần tử.

Tổng cộng:

$$(2n - 1) + (2n - 1) + n = 5n - 2.$$

Cách mới tránh được nhóm mảng này.

Đây là ví dụ rõ ràng cho việc hai thuật toán cùng $O(n)$ nhưng mức dùng bộ nhớ thực tế có thể khác đáng kể.

“Word” là đơn vị lưu trữ của mô hình thuật toán; không nên tự đổi các biểu thức này thành số byte đo được nếu chưa biết cách cài đặt.

---

7. Ứng dụng mới ở §4.1: LCA trên cây nhị phân bằng inorder

Phần này phù hợp để bạn đưa vào seminar thay cho việc đi sâu vào LCE.

Bạn đã biết cách dùng Euler tour và RMQ trên độ sâu để giải LCA. Paper chỉ ra rằng với **cây nhị phân**, có thể dùng **duyệt inorder** để tạo mảng dài đúng n.

Ví dụ cây:

Quy ước 1 là con trái của 0, 2 là con phải; $3, 4$ lần lượt là con trái và phải của 1.

Duyệt inorder — trái, gốc, phải — thu được:

$$I = [3, 1, 4, 0, 2].$$

Ghi độ sâu của các đỉnh theo thứ tự đó:

$$H = [2, 1, 2, 0, 1].$$

Vị trí inorder	**0**	**1**	**2**	**3**	**4**
Đỉnh $I[i]$	3	1	4	0	2
Độ sâu $H[i]$	2	1	2	0	1

Muốn tìm $\mathrm{LCA}(3, 4)$:

- Đỉnh 3 ở vị trí inorder 0.
- Đỉnh 4 ở vị trí inorder 2.
- Query $H[0..2] = [2, 1, 2]$.
- Cực tiểu ở vị trí 1, tương ứng đỉnh $I[1] = 1$.

Vậy:

$$\mathrm{LCA}(3,4) = 1.$$

Tương tự, với hai đỉnh $4, 2$:

$$H[2..4] = [2, 0, 1],$$

cực tiểu ứng với đỉnh 0, nên:

$$\mathrm{LCA}(4,2) = 0.$$

Lý do đúng: trong inorder của cây nhị phân, LCA nằm giữa hai đỉnh đang hỏi; các đỉnh trong đoạn đó đều thuộc cây con của LCA. Vì vậy, LCA là đỉnh có độ sâu nhỏ nhất trong đoạn.

Điểm liên quan trực tiếp đến câu hỏi ± 1 trước đó của bạn: mảng H ở đây có bước $2 \to 0$, nên không có tính chất ± 1. RMQ tổng quát của Fischer–Heun xử lý được ngay.

Cách chuyển LCA thành RMQ	Độ dài mảng	Tính chất
Euler tour	$2n - 1$	Độ sâu kề nhau chênh ± 1
Inorder trên cây nhị phân	n	Không bảo đảm ± 1

Phần §4.2 còn ứng dụng vào LCE thông qua suffix array và LCP. Vì bạn đã chọn bỏ LCE khỏi phần chính, chỉ cần giới thiệu ngắn như một ứng dụng xử lý chuỗi.

---

8. Benchmark ở §5: thuật toán $O(1)$ chưa chắc nhanh nhất

Đây là phần rất đáng đưa vào seminar của bạn.

Paper so sánh năm phương pháp:

Phương pháp	Build	Query xấu nhất
Fischer–Heun	$O(n)$	$O(1)$
Berkman–Vishkin	$O(n)$	$O(1)$
Chia khối $b = \frac{1}{2}\log_2 n$, quét nội khối	$O(n)$	$O(\log n)$
Chia khối $b = \frac{1}{4}\log_2 n$, quét nội khối	$O(n)$	$O(\log n)$

Phương pháp	Build	Query xấu nhất
Quét trực tiếp đoạn hỏi	Không tiền xử lý	$O(r-l+1)$

Hai biến thể "quét nội khối" vẫn dùng Sparse Table trên cực tiểu các khối, nhưng **bỏ bảng tra chung** P. Khi query, chúng quét hai phần biên.

Thí nghiệm dùng C++, máy Athlon XP3000, RAM 2 GB, Linux; kết quả lấy trung bình qua năm lần chạy. Đây là benchmark của năm 2006.
theoretical_and_practical_improvements_on_the_rmq_problem_with_applications.pdf

Kết quả A: Fischer–Heun tốt hơn phương pháp tối ưu cũ trong các bài thử.

Trong thí nghiệm thay đổi độ dài query, với:

$$n = 10^7, \qquad 10^6 \text{ truy vấn ngẫu nhiên},$$

Fischer–Heun có thời gian query khoảng **một nửa** Berkman–Vishkin.

Tiền xử lý của phương pháp cũ cũng chậm hơn đáng kể; tác giả quy nguyên nhân cho bước dựng Cartesian tree tường minh.

Kết quả B: hai biến thể query $O(\log n)$ còn nhanh hơn Fischer–Heun một chút.

Tại sao?

Với:

$$n = 10^7,$$

kích thước khối:

$$b \approx \frac{\log_2 10^7}{2} \approx 11.$$

Quét một phần của khối chỉ dài khoảng 11 phần tử có thể rất rẻ.

Trong khi đó, tra bảng $O(1)$ vẫn cần tính chỉ số và truy cập các bảng. Những thao tác này có chi phí thực tế.

Vì vậy:

> $O(1)$ nói rằng số thao tác không tăng theo kích thước đầu vào; nó không bảo đảm chi phí nhỏ hơn mọi thuật toán $O(\log n)$ trên những đầu vào đang xét.

Kết quả C: query ngắn thì quét trực tiếp rất cạnh tranh.

Trong thí nghiệm đó, quét trực tiếp chạy rất nhanh với các đoạn ngắn, tới khoảng 100 phần tử. Khi đoạn dài hơn, chi phí tăng theo chiều dài đoạn và trở nên bất lợi.

Con số 100 là kết quả của cấu hình thử nghiệm ấy, không phải ngưỡng chung cho mọi máy.

Kết quả D: bộ nhớ và cache ảnh hưởng hiệu năng.

Ở thí nghiệm thay đổi n, phương pháp cũ chỉ xử lý được mảng khoảng 3×10^7 phần tử trên máy thử nghiệm, trong khi các phương pháp còn lại đạt khoảng 6×10^7.

Tác giả cũng quan sát những điểm thời gian query tăng mạnh và cho rằng chúng có thể liên quan đến cache. Paper không đo trực tiếp để khẳng định cơ chế này.

Các biểu đồ thời gian query **không tính thời gian tiền xử lý**, nên khi đánh giá tổng chi phí, bạn cần xét:

$$T_{\text{tổng}} = T_{\text{build}} + q\,T_{\text{query}}.$$

---

9. Đối chiếu với Sparse Table và Segment Tree

Bảng dưới đây giúp đặt paper vào những CTDL bạn đã biết; **đây là so sánh lý thuyết, không phải các phương pháp đều được benchmark trong paper**.

CTDL / thuật toán	Build	Query RMQ	Bộ nhớ	Phù hợp
Sparse Table	$O(n \log n)$	$O(1)$	$O(n \log n)$	Mảng tĩnh, ưu tiên cài đặt đơn giản
Segment Tree	$O(n)$	$O(\log n)$	$O(n)$	Có cập nhật phần tử
Chia khối + Sparse Table + quét biên	$O(n)$*	$O(\log n)$*	$O(n)$*	Mảng tĩnh, biến thể thực dụng trong paper
Fischer–Heun	$O(n)$	$O(1)$	$O(n)$	Mảng tĩnh, cần bảo đảm tối ưu tiệm cận

$*$ Với kích thước khối $\Theta(\log n)$.

Fischer–Heun phức tạp hơn về cài đặt và không hỗ trợ cập nhật hiệu quả như Segment Tree. Nếu mảng thường xuyên thay đổi, kết quả build/query cho bài toán tĩnh không còn là tiêu chí đủ để chọn nó.

Đối với seminar, mình đề xuất bạn lấy ba ý chính từ paper:

1. **Cùng hình dạng Cartesian tree thì dùng chung bảng RMQ nội khối.**
2. **Xử lý RMQ trực tiếp giúp tránh Euler tour và giảm bộ nhớ dù độ phức tạp vẫn là $O(n)$.**

3. **Benchmark cho thấy cần xét cả hằng số, độ dài query và bộ nhớ; độ phức tạp tiệm cận chưa quyết định hoàn toàn tốc độ thực tế.**

Phần LCA bằng inorder là ví dụ mở rộng cụ thể, còn kết quả "quét khối nhỏ thắng tra bảng $O(1)$" là điểm tốt để bạn dẫn vào phần đánh giá và thảo luận.

You said:

rmq (1).pdf
PDF

1 selection

Trình bày dễ hiểu cho tui nội dung paper này

Worked for 1m 13s

ChatGPT said:

Điểm chính của paper này là: chỉ cần khoảng 2 bit cho mỗi phần tử để lưu đủ thông tin trả lời RMQ, đồng thời vẫn query $O(1)$.

Nghe có vẻ khó tin: một số nguyên thường đã cần 32 hoặc 64 bit, sao chỉ cần khoảng 2 bit?

Mấu chốt là **paper lưu cấu trúc quyết định vị trí cực tiểu, không lưu lại toàn bộ giá trị của mảng**.

Mình giải thích theo Introduction và §5 của bản bạn gửi. rmq (1).pdf

1. Từ paper 2006 sang paper này: mục tiêu thay đổi thế nào?

Ở Fischer–Heun 2006, bạn đã học:

$$\text{Build } O(n), \qquad \text{Query } O(1), \qquad \text{Bộ nhớ } O(n) \text{ word}.$$

Paper này tiếp tục hỏi:

> Có thể giữ nguyên thời gian, nhưng giảm bộ nhớ xuống gần mức tối thiểu về số bit không?

Cần phân biệt hai đơn vị:

Đơn vị	Ý nghĩa
Bit	Một giá trị 0 hoặc 1
Word	Một ô máy đủ chứa chỉ số hoặc dữ liệu cần thao tác

Để lưu một chỉ số trong n vị trí, cần khoảng:

$$\log_2 n \text{ bit}.$$

Vì vậy, trong mô hình thông dụng với word dài $\Theta(\log n)$ bit:

$$O(n) \text{ word} = O(n \log n) \text{ bit}.$$

Còn kết quả nổi bật của paper là:

$$\boxed{2n + o(n) \text{ bit}}.$$

Đây là cải thiện về bậc tăng trưởng bộ nhớ khi đo bằng bit.

---

2. $2n + o(n)$ bit nghĩa là gì?

Tách thành hai phần:

$$\underbrace{2n}_{\text{thông tin cấu trúc chính}} + \underbrace{o(n)}_{\text{thông tin phụ để truy vấn nhanh}}.$$

Ký hiệu $o(n)$, đọc là "o nhỏ của n", chỉ một lượng tăng chậm hơn n, theo nghĩa:

$$\frac{o(n)}{n} \longrightarrow 0 \qquad \text{khi } n \longrightarrow \infty.$$

Ví dụ:

$$\sqrt{n} = o(n), \qquad \frac{n}{\log n} = o(n).$$

Nhưng:

$$0.1n \neq o(n),$$

vì chia cho n vẫn còn 0.1, không tiến về 0.

Do đó:

$$\frac{2n + o(n)}{n} = 2 + o(1).$$

Nghĩa là **số bit trung bình cho mỗi phần tử tiến gần đến 2 khi n tăng**.

Ví dụ với một triệu phần tử, riêng phần $2n$ là:

$$2\,000\,000 \text{ bit} = 250\,000 \text{ byte} \approx 0.25 \text{ MB}.$$

Vẫn phải cộng phần phụ $o(n)$. Công thức tiệm cận không cho biết chính xác phần phụ lớn bao nhiêu ở một kích thước cụ thể.

---

3. Sao có thể bỏ mảng gốc mà vẫn trả lời RMQ?

Vì RMQ trong paper trả về **vị trí**, không nhất thiết trả về giá trị nhỏ nhất.

Ví dụ:

$$A = [9, 4, 1, 6], \qquad B = [90, 40, 10, 60].$$

Hai mảng khác giá trị, nhưng mọi truy vấn RMQ đều trả về cùng vị trí:

$$\text{RMQ}_A(l, r) = \text{RMQ}_B(l, r).$$

Ta không cần phân biệt 1 với 10 nếu câu hỏi chỉ là:

> Phần tử nhỏ nhất nằm ở đâu?

Paper phân biệt hai mô hình:

Mô hình	Khi query có đọc lại A không?	Thông tin cần lưu
Systematic, hay indexing	Có thể đọc	Mảng A và chỉ mục phụ
Non-systematic, hay encoding	Không cần đọc	Một cấu trúc tự trả lời được vị trí cực tiểu

Kết quả $2n + o(n)$ bit ở §5 thuộc mô hình thứ hai.

Sau khi xây xong, có thể bỏ A **nếu ứng dụng chỉ cần vị trí RMQ và không cần mảng cho việc khác**.

Nếu muốn biết cả giá trị:

$$\min A[l..r] = A[\mathrm{RMQ}(l, r)],$$

thì vẫn cần cách truy cập giá trị của A.

Đây là lý do cấu trúc RMQ nhỏ như vậy: nó không phải một phương pháp nén có thể khôi phục toàn bộ mảng số ban đầu.

---

4. Vì sao con số 2 là gần tối ưu?

Bạn đã biết:

> Các đáp án RMQ trên mọi đoạn tương đương với hình dạng Cartesian tree của mảng.

Với n phần tử, số hình dạng Cartesian tree có thể có là số Catalan:

$$C_n = \frac{1}{n+1}\binom{2n}{n}.$$

Muốn phân biệt được C_n trường hợp, trong trường hợp xấu nhất phải cần ít nhất khoảng:

$$\log_2 C_n$$

bit.

Dùng xấp xỉ Catalan:

$$C_n \approx \frac{4^n}{\sqrt{\pi}\, n^{3/2}},$$

suy ra:

$$\log_2 C_n = 2n - \frac{3}{2}\log_2 n - O(1).$$

Bạn không cần nhớ hết công thức. Ý nghĩa là:

$$\boxed{\text{Mức tối thiểu có số hạng chính là } 2n \text{ bit.}}$$

Vậy cấu trúc dùng:

$$2n + o(n)$$

bit đã đạt gần sát giới hạn thông tin đó. Loại cấu trúc như vậy được gọi là **succinct data structure**.

Lưu ý: đây là giới hạn trong trường hợp xấu nhất cho mô hình tự trả lời RMQ, không phải mọi mảng cụ thể đều cần gần $2n$ bit.

---

5. §5 dùng cây gì? Một cây mới gọi là 2d-Min-Heap

Để đạt kết quả trên, paper giới thiệu **2-dimensional Min-Heap**.

Tên có chữ "2-dimensional", nhưng ta vẫn đang giải RMQ trên mảng một chiều.

Cách dựng cây khá dễ hiểu:

1. Thêm phần tử giả $A[0] = -\infty$.
2. Với mỗi vị trí i, tìm vị trí gần nhất bên trái có giá trị **nhỏ hơn nghiêm ngặt** $A[i]$.
3. Cho vị trí đó làm cha của i.
4. Sắp các con từ trái sang phải theo chỉ số tăng dần.

Paper gọi vị trí cha là:

$$\mathrm{psv}_A(i) = \max\{j < i : A[j] < A[i]\}.$$

Ví dụ:

$$A = [5, 2, 4, 1, 3].$$

Ở phần này mình dùng chỉ số $1..5$, giống paper.

Vị trí i	$A[i]$	Vị trí nhỏ hơn gần nhất bên trái	Cha
1	5	Chỉ có phần tử giả	0
2	2	Chỉ có phần tử giả	0
3	4	Vị trí 2, giá trị 2	2
4	1	Chỉ có phần tử giả	0
5	3	Vị trí 4, giá trị 1	4

Thu được cây:

Mỗi nhãn có dạng “chỉ số: giá trị”.

Cây này có ba tính chất đáng nhớ:

- Cha có giá trị nhỏ hơn con.
- Các con của cùng một cha có giá trị không tăng khi đi từ trái sang phải.
- **Duyệt preorder thu được đúng thứ tự chỉ số** $0, 1, \ldots, n$.

Trong ví dụ:

$$0, 1, 2, 3, 4, 5.$$

Tính chất cuối rất quan trọng: **chỉ số mảng gắn tự nhiên với thứ tự các nút**, giúp tránh phải lưu một bảng ánh xạ lớn.

---

6. RMQ trên cây mới được trả lời thế nào?

Đây là chỗ cần phân biệt với Cartesian tree.

Trong Cartesian tree:

$$\mathrm{RMQ}(i, j) = \mathrm{LCA}(i, j).$$

Nhưng trên **2d-Min-Heap**, quy tắc là:

Gọi:

$$\ell = \mathrm{LCA}(i, j), \qquad i < j.$$

- Nếu $\ell = i$, đáp án là i.
- Nếu $\ell \neq i$, đáp án là **đứa con trực tiếp của ℓ nằm trên đường đi đến j**.

Xem hai ví dụ trên cùng cây.

Trường hợp A: $\mathrm{RMQ}(2, 3)$.

Đoạn hỏi:

$$A[2..3] = [2, 4].$$

Trên cây, 2 là cha của 3, nên:

$$\mathrm{LCA}(2, 3) = 2.$$

LCA chính là đầu trái, vì vậy trả về 2.

Trường hợp B: $\mathrm{RMQ}(3, 5)$.

Đoạn hỏi:

$$A[3..5] = [4, 1, 3].$$

Trên cây:

$$\mathrm{LCA}(3, 5) = 0.$$

Đường từ 0 đến 5 là:

$$0 \to 4 \to 5.$$

Đứa con trực tiếp của 0 trên đường này là 4, nên đáp án là:

$$\boxed{\mathrm{RMQ}(3, 5) = 4}.$$

Đúng vì $A[4] = 1$.

Tại sao lấy nhánh phía j? Trực quan là:

- Mỗi cây con chiếm một đoạn chỉ số liên tiếp.
- Gốc mỗi cây con nhỏ hơn các phần tử phía dưới.
- Các gốc anh em càng về bên phải có giá trị càng nhỏ hoặc bằng.

Vì vậy, khi đoạn hỏi đi qua nhiều nhánh, gốc của nhánh chứa đầu phải j quyết định cực tiểu.

§5 dùng quy ước chọn **cực tiểu bên phải nhất** khi hòa; paper cũng giải thích có thể điều chỉnh để chọn bên trái nhất.

---

7. Nén cả cây xuống khoảng $2n$ bit bằng cách nào?

Nếu lưu cây bằng con trỏ, mỗi con trỏ đã cần một word. Ta sẽ quay lại mức $O(n \log n)$ bit.

Paper dùng một cách mã hóa cây có tên:

DFUDS — Depth-First Unary Degree Sequence.

Bạn có thể hiểu là:

> Duyệt cây theo preorder và ghi số con của từng nút bằng một đoạn bit.

Nút có d con được ghi bằng:

- d dấu mở `(`;
- một dấu đóng `)`.

Ví dụ:

Số con	Mã
0	`)`
1	`()`
2	`(())`
3	`((())`

Thêm một dấu mở đầu tiên để tạo chuỗi ngoặc cân bằng theo định nghĩa DFUDS.

Với cây có N nút:

- Tổng số con của mọi nút bằng số cạnh: $N - 1$.
- Có một dấu đóng cho mỗi nút: N.
- Có một dấu mở bổ sung.

Tổng cộng:

$$(N - 1) + N + 1 = 2N$$

ký hiệu.

Mỗi ký hiệu chỉ có hai khả năng, nên mã hóa bằng một bit:

$$(\leftrightarrow 1, \qquad) \leftrightarrow 0.$$

Vậy cây được lưu trong $2N$ bit.

Cây của ta có thêm một gốc giả, nên chênh lệch với $2n$ chỉ là hằng số, được hấp thụ trong phần $o(n)$.

Đây là nguồn gốc của số hạng $2n$: lưu hình dạng cây bằng bit thay vì lưu con trỏ.

---

8. Nhưng cây đã nén rồi thì tìm kiếm có chậm không?

Nếu cứ quét chuỗi ngoặc để tìm cha hoặc tổ tiên, query có thể mất $O(n)$.

Vì vậy, cần thêm các chỉ mục nhỏ để hỗ trợ những phép như:

Phép toán	Ý nghĩa dễ hiểu
`rank`	Đếm số dấu thuộc một loại trong một đoạn đầu
`select`	Tìm vị trí dấu thứ k của một loại
`findopen`	Tìm dấu mở tương ứng với một dấu đóng

Các chỉ mục này giúp thao tác trực tiếp trên cây đã nén.

Paper còn dùng **excess sequence**:

$$E[p] = \text{số dấu mở trong } U[1..p] - \text{số dấu đóng trong } U[1..p].$$

Ví dụ:

Chuỗi ngoặc	(	(	)	)
Excess	1	2	1	0

Mỗi bước:

- Đọc (thì tăng 1.
- Đọc) thì giảm 1.

Do đó:

$$E[p+1] - E[p] \in \{-1, +1\}.$$

Bài toán ± 1 RMQ bạn học trước đó xuất hiện trở lại ở đây.

Điểm tiết kiệm là không lưu E thành một mảng số nguyên riêng. Nó được biểu diễn ngầm bởi chuỗi ngoặc.

Kết hợp các phép trên và ± 1 RMQ, paper trả lời mỗi query bằng một số cố định thao tác $O(1)$, với tổng bộ nhớ:

$$\boxed{2n + O\left(\frac{n \log\log n}{\log n}\right) \text{ bit.}}$$

Vì:

$$\frac{\log\log n}{\log n} \to 0,$$

phần phía sau là $o(n)$.

Thời gian $O(1)$ ở đây được chứng minh trong mô hình word-RAM mà paper sử dụng.

---

9. Xây cấu trúc nhỏ như vậy có cần bộ nhớ tạm rất lớn không?

Đây là một đóng góp quan trọng khác.

Một cấu trúc cuối cùng chỉ chiếm ít bộ nhớ vẫn có thể cần rất nhiều bộ nhớ trong lúc xây. Paper chú ý cả hai giai đoạn.

Đầu tiên, tác giả đưa ra thuật toán dùng stack, quét mảng từ phải sang trái và sinh trực tiếp mã DFUDS.

Mỗi vị trí được push và pop tối đa một lần:

$$\boxed{\text{Thời gian xây } O(n)}$$

nếu mỗi lần đọc phần tử mảng mất $O(1)$.

Nhưng stack chứa chỉ số có thể cần:

$$O(n \log n) \text{ bit.}$$

Tác giả tiếp tục thay stack này bằng:

- Một bit-vector đánh dấu vị trí đang nằm trong stack.
- Các bảng phụ theo khối để tìm nhanh phần tử ở đỉnh stack.

Nhờ đó, bộ nhớ làm việc bổ sung giảm xuống:

$$n + o(n) \text{ bit},$$

ngoài đầu vào và vùng chứa kết quả.

Vì vậy, cần phân biệt:

Đại lượng	**Kết quả**
Thời gian xây	$O(n)$, nếu đọc $A[i]$ mất $O(1)$
Thời gian query	$O(1)$
Cấu trúc sau khi xây	$2n + o(n)$ bit
Bộ nhớ làm việc bổ sung khi xây	$n + o(n)$ bit

---

10. Paper còn những nội dung nào ngoài kết quả $2n + o(n)$?

Paper có hai nhánh khác đáng biết, nhưng bạn có thể giới thiệu ngắn.

§3: đánh đổi bộ nhớ và thời gian khi vẫn giữ mảng A.

Bằng cách chọn tham số c, có thể đạt gần:

$$\text{Chỉ mục phụ } \approx \frac{2n}{c} \text{ bit}, \qquad \text{Query } O(c),$$

giả sử đọc phần tử mất $O(1)$.

Chẳng hạn chọn $c = 10$, số hạng chính của chỉ mục phụ giảm còn khoảng $0.2n$ bit, nhưng query làm nhiều việc hơn.

Điều này không trái với giới hạn gần $2n$ bit đã nói, vì **thuật toán vẫn được phép đọc lại mảng A** để lấy thông tin còn thiếu.

§4: nén theo mức độ lặp của dữ liệu.

Các loại khối xuất hiện thường xuyên được gán mã ngắn hơn. Paper phân tích bộ nhớ bằng entropy thực nghiệm H_k.

Bạn chỉ cần hiểu ý chính:

Nếu dữ liệu có tính lặp có thể khai thác được, chỉ mục RMQ cũng có thể được nén thêm.

---

11. Nên rút gì vào seminar?

Đặt cạnh paper 2006:

Nội dung	Fischer–Heun 2006 bạn vừa học	Kết quả nổi bật ở §5 paper này
Trọng tâm	RMQ trực tiếp và cải thiện thực tế	Bộ nhớ gần giới hạn thông tin
Cấu trúc chính	Chia khối, loại Cartesian tree, bảng tra	2d-Min-Heap và DFUDS
Đọc lại mảng khi query	Có	Không cần
Bộ nhớ	$O(n)$ word	$2n + o(n)$ bit
Build / query	$O(n)$ / $O(1)$	$O(n)$ / $O(1)$
Benchmark	Có thực nghiệm	Tập trung chứng minh lý thuyết

Với seminar 30 phút, phần mở rộng này có thể trình bày trong khoảng **3–4 phút**:

1. Phân biệt $O(n)$ word với $O(n)$ bit.
2. Giải thích chỉ lưu vị trí cực tiểu nên có thể bỏ giá trị gốc.
3. Dùng Catalan để giải thích giới hạn gần $2n$ bit.
4. Giới thiệu cây nén và kết quả $2n + o(n)$ bit, query $O(1)$.

Ví dụ 2d-Min-Heap ở trên giúp bạn giải thích cơ chế nếu được hỏi sâu. Không nhất thiết đưa toàn bộ `rank`, `select` và chứng minh DFUDS lên slide chính.

You said:

có cần phải đưa nội dung này vào không vì tui thấy nó hơi nâng cao lun á

ChatGPT said:

Không bắt buộc đâu. Với seminar 30 phút của bạn, mình nghĩ nên bỏ phần kỹ thuật của paper 2011 khỏi nội dung chính. Bạn đã có khá nhiều nội dung cần giải thích: Sparse Table, Cartesian tree, RMQ–LCA và chia khối kết hợp bảng tra để đạt $O(n)$ build, $O(1)$ query.

Nếu thêm 2d-Min-Heap, DFUDS, `rank/select` thì phải giới thiệu nhiều khái niệm mới. Phần này dễ chiếm thời gian dành cho **ví dụ, chứng minh, so sánh CTDL và benchmark** — những phần bạn muốn làm kỹ để tránh thiếu sót như các nhóm trước.

Bạn chỉ cần dành **20–30 giây ở phần mở rộng**, nói:

> "Ngoài tối ưu thời gian, RMQ còn có hướng tối ưu bộ nhớ. Fischer–Heun đạt cấu trúc chỉ dùng $2n + o(n)$ bit, vẫn truy vấn $O(1)$. Cấu trúc này lưu đủ thông tin để trả về vị trí cực tiểu mà không cần truy cập mảng gốc."

Không cần trình bày cách xây hay chứng minh. Có thể để tài liệu tham khảo hoặc một slide phụ nếu thầy hỏi sâu.

Phần nâng cao chính nên dừng ở chia khối + loại Cartesian tree + bảng tra chung. Giải thích rõ vì sao cách đó đạt $O(n)/O(1)$, rồi đối chiếu với Sparse Table, Segment Tree và kết quả thực nghiệm là đã đủ chiều sâu cho buổi 30 phút.

You said:

hiện tại có method để phân loại cartesian tree, 1 là dùng dãy bit, 2 là dùng +-1, 3 là dùng stack pop đúng không

ChatGPT said:

Gần đúng, nhưng ba cách này không hoàn toàn cùng một loại. Bạn đang gộp **cách mã hóa loại khối** với **cách tính mã loại**.

Cách bạn nhắc	Thực chất làm gì?	Áp dụng cho
Dãy bit mô tả Cartesian tree	Mã hóa **hình dạng cây** để nhận diện các khối có cùng đáp án RMQ	Khối của mảng tổng quát
Dãy dấu ± 1	Mã hóa **các bước tăng/giảm** của một khối có tính chất đặc biệt	Khối mà hai phần tử kề nhau chênh đúng 1
Stack pop	Nhận diện cấu trúc Cartesian tree; kết hợp ballot numbers để tính **mã loại Catalan**	Khối của mảng tổng quát

1. Dãy bit: ghi lại hình dạng Cartesian tree

Ta dựng hoặc mô phỏng Cartesian tree của khối, rồi mã hóa hình dạng cây theo một quy ước bit.

Hai khối có cùng hình dạng cây sẽ có cùng mã, nên dùng chung bảng đáp án RMQ.

Với cách mã hóa $2b$ bit đã học:

$$\text{Số mã có thể có} \le 2^{2b} = 4^b.$$

Đây là **chặn trên**; không phải mọi chuỗi bit đều biểu diễn một cây hợp lệ.

2. Dãy ± 1: nhận diện loại của khối đặc biệt

Ví dụ:

$$X = [4, 5, 4, 3], \qquad Y = [10, 11, 10, 9].$$

Cả hai có các bước:

$$[+1, -1, -1].$$

Trừ phần tử đầu tiên khỏi mọi phần tử, cả hai trở thành:

$$[0, 1, 0, -1].$$

Vì vậy chúng có cùng đáp án RMQ tương đối và dùng chung bảng tra.

Ở đây ta mã hóa trực tiếp dạng tăng/giảm của khối, không cần mã hóa Cartesian tree.

Đặc biệt, cách này không áp dụng cho mảng bất kỳ chỉ bằng việc ghi "tăng hay giảm". Ví dụ:

$$[1, 4, 2], \qquad [2, 4, 1].$$

Cả hai đều "tăng rồi giảm", nhưng cực tiểu toàn khối nằm ở hai vị trí khác nhau. Điều kiện **chênh đúng** 1 mới giúp chuỗi dấu xác định được khối sau khi chuẩn hóa.

3. Stack pop: cách tính mã loại Cartesian tree

Với:

$$[9, 4, 1, 6],$$

số phần tử bị pop khi lần lượt đọc từng phần tử là:

$$[0, 1, 1, 0].$$

Diễn biến này mô tả quá trình hình thành Cartesian tree. Fischer–Heun kết hợp nó với **ballot numbers** để tính mã loại:

$$0 \le \text{type} < C_b.$$

Ví dụ khối dài 4 có $C_4 = 14$ loại, nên mã nằm trong $0..13$.

Stack pop là cơ chế tính mã; mã Catalan là kết quả. Stack cũng có thể được dùng để dựng cây rồi xuất mã bit, nên "dãy bit" và "stack" không phải hai lựa chọn loại trừ nhau.

Bạn có thể ghi trong seminar như sau:

> **Có hai hướng nhận diện khối: dùng cấu trúc Cartesian tree cho RMQ tổng quát, hoặc dùng chuỗi bước ± 1 cho RMQ đặc biệt. Với hướng Cartesian tree, có thể mã hóa hình dạng bằng bit hoặc tính mã Catalan bằng stack và ballot numbers.**

You said:

rmq_seminar.pdf
PDF

xem slide này và cho tui biết những nội dung trong slide thì tui đã tìm hiểu hết chưa

Worked for 1m 18s

ChatGPT said:

Chưa hết, nhưng bạn đã tìm hiểu phần lý thuyết cốt lõi của RMQ trong slide rồi. Những phần còn thiếu chủ yếu là **bitmask, cách đọc benchmark cụ thể, một số CTDL và ứng dụng mở rộng**.

Mình đã xem toàn bộ 59 trang của slide. Đánh giá dưới đây dựa trên những nội dung mình và bạn đã trao đổi; "đã tìm hiểu" nghĩa là đã được giải thích, chưa khẳng định bạn đã tự chứng minh hoặc cài đặt thành thạo.

1. Những phần bạn đã tìm hiểu khá đầy đủ

Nội dung	Slide	Đối chiếu với những gì bạn đã học
Phát biểu RMQ, trả vị trí hay giá trị, xử lý phần tử bằng nhau	6	Đã học qua phần introduction và code lưu chỉ số
Duyệt trực tiếp, tiền xử lý mọi đoạn	5, 9	Đã gặp trong Primitive RMQ
Sparse Table: xây bảng, truy vấn hai đoạn chồng lấn	12–15, 17	Đã học; bạn còn hỏi kỹ `i + half` và công thức `lg[]`
Cartesian tree và quan hệ RMQ–LCA	24	Đã học và yêu cầu chứng minh lại
Chia khối, Sparse Table trên cực tiểu các khối	22–23	Đã học kỹ về $\Theta(\log n)$ và vì sao tầng trên build $O(n)$

Nội dung	Slide	Đối chiếu với những gì bạn đã học
Khối cùng hình dạng cây dùng chung bảng tra	25	Đã học trong exhaustive tabulation
Đếm loại khối, chọn kích thước khối, phân tích $O(n)/O(1)$	25–26	Đã giải thích nhiều bước
Euler tour, LCA quy về ± 1 RMQ	27	Đã học qua Bender–Farach-Colton
Mã bit và stack pop để nhận diện loại khối	Liên quan 24–26	Vừa phân biệt ở câu hỏi trước
Bộ nhớ $2n + o(n)$ bit, bit khác word	Một phần slide 30, 55	Đã học qua Fischer–Heun 2011; đủ để giới thiệu ngắn
LCE qua suffix array, LCP và RMQ	46	Đã được giới thiệu, nhưng bạn đã chọn bỏ khỏi phần chính

Như vậy, chuỗi lập luận quan trọng nhất của slide — từ Sparse Table đến RMQ build $O(n)$, query $O(1)$ — bạn đã đi qua rồi.

---

2. Những phần mới được nhắc đến, chưa tìm hiểu đủ để giảng theo slide

Nội dung	Slide	Bạn còn cần nắm gì?
Vì sao prefix-min không trả lời được RMQ tùy ý	7	Tự giải thích được phản ví dụ hai mảng có cùng prefix-min
Chia khối $\sqrt{n}$	10	Khác chia khối $\Theta(\log n)$ ở đâu; query và cập nhật thế nào
Segment Tree	11, 18–20	Cách build, query, cập nhật và lý do $O(\log n)$; hiện chủ yếu đã so sánh tổng quan
Bài CSES và lựa chọn CTDL	18–20	Từ giới hạn đề suy ra cách giải, phân biệt tĩnh và động
LCA: binary lifting, Euler tour và khoảng cách	44–45	Bạn đã học Euler–RMQ; chưa đi sâu binary lifting và so sánh đầy đủ

Nội dung	Slide	Bạn còn cần nắm gì?
Benchmark	32–41	Bạn đã học kết quả paper 2006; chưa phân tích cụ thể toàn bộ thí nghiệm và số liệu trong bộ slide này

Đặc biệt, **đọc kết luận benchmark chưa đủ để thuyết trình phần benchmark**. Bạn cần giải thích được:

- Build và query được đo riêng hay cộng chung?
- Bộ nhớ đo là payload hay toàn bộ RAM tiến trình?
- Vì sao thay đổi số query q có thể đổi phương pháp thắng?
- Ngưỡng hòa vốn được tính thế nào?
- Kết luận nào chỉ đúng trong cấu hình đã đo?

---

3. Những phần chưa thấy bạn học trong cuộc trao đổi này

Nội dung	Slide	Mức độ cần thiết
RMQ bằng bitmask 64 bit, `clz`, `ctz`, lọc mask theo $[l, r]$	**28–29**	**Cần học nếu giữ phần cài đặt và benchmark hiện tại**
Fenwick hai cây cho RMQ	19–20, 40	Có thể chỉ nhắc trong bảng so sánh
Deque đơn điệu cho cửa sổ trượt	19, 43–44	Một mở rộng hữu ích, có thể chọn học
Lazy Segment Tree: cộng/gán đoạn	43, 56	Học nếu trình bày cập nhật đoạn
Offline RMQ: Arpa's trick và co mảng cho lô truy vấn nhỏ	43, 58	Mở rộng, không bắt buộc
Disjoint Sparse Table, Sqrt Tree	13, 20, 43, 58	Mở rộng sang phép toán khác
RMQ hai chiều	43, 57	Có thể để phụ lục
Liệt kê tài liệu bằng RMQ trên mảng vị trí xuất hiện trước	47	Ứng dụng mới, chưa học
Histogram, xử lý ảnh, OLAP, bộ lọc chuỗi thời gian	44, 48	Chưa phân tích các thuật toán cụ thể
Phản ví dụ: RMQ trên Euler tour không giải trực tiếp min trọng số đường đi	49	Nên hiểu nếu nói về ứng dụng trên cây

Nội dung	Slide	Mức độ cần thiết
Cận dưới cell-probe, RMQ động	Một phần slide 30	Nâng cao, không cần chứng minh
RMQ song song, GPU, ray tracing	31, 40, 43	Chỉ nên giới thiệu ngắn
Các bài tập ứng dụng cụ thể	50	Chưa thấy bạn chạy lời giải từng bài

Bitmask là khoảng trống lớn nhất hiện tại. Nó dùng stack đơn điệu, nhưng không phải chỉ là "stack pop để tính mã loại Catalan" mà bạn vừa hỏi. Nó lưu trực tiếp trạng thái stack bằng bit, rồi dùng thao tác bit để trả lời query nội khối.

---

4. Có vài chỗ trong slide cần chỉnh trước khi bạn học thuộc

Thứ nhất, bitmask khối cố định 64 chưa tự động cho build $O(n)$ theo phân tích tiệm cận.

Slide 28 ghi tầng trên là Sparse Table trên khoảng $n/64$ khối. Khi đó, chi phí build thực sự là:

$$O\left(n + \frac{n}{64}\log\frac{n}{64}\right).$$

Nếu coi 64 là hằng số, biểu thức này vẫn là $O(n\log n)$, dù nhỏ hơn Sparse Table toàn mảng rất nhiều trong thực tế.

Vì vậy, cần phân biệt **bản cài đặt khối 64 thực dụng** với **chứng minh tuyến tính của Fischer–Heun**. Không nên gộp hai bản thành một kết quả lý thuyết mà không giải thích thêm.

Thứ hai, code ở slide 15 và 29 trả giá trị min.

Trong khi định nghĩa ở slide 6 là trả **chỉ số argmin**. Hai biến thể đều hợp lệ, nhưng cần nói rõ khi chuyển sang phần code.

Thứ ba, Sparse Table không bắt buộc phải xây lại toàn bộ $O(n\log n)$ sau một cập nhật điểm.

Có thể tính lại các ô bị ảnh hưởng trong $O(n)$ thời gian xấu nhất. Nó vẫn không phù hợp bằng Segment Tree cho cập nhật thường xuyên, nhưng chữ "phải xây lại" ở slide 18 đang quá mạnh.

Thứ tư, lộ trình hiện ghi 40 phút và vẫn có LCE.

Bạn đang hướng tới khoảng 30 phút và đã muốn bỏ LCE, nên bộ slide này chưa khớp với lựa chọn nội dung hiện tại. PDF cũng chỉ thể hiện ảnh chụp của demo; muốn dùng demo khi trình bày cần kiểm tra bản chạy thực tế.

---

5. Bạn nên học tiếp gì trước?

Với mục tiêu seminar của bạn, mình ưu tiên theo thứ tự:

1. **Segment Tree và chia khối $\sqrt{n}$** — để phần so sánh CTDL có cơ sở.
2. **Bitmask ở slide 28–29** — vì nó xuất hiện trong code và xuyên suốt benchmark.
3. **Benchmark ở slide 32–41** — đọc đồ thị, tính tổng thời gian và điểm hòa vốn.
4. **Một ứng dụng cây hoàn chỉnh** — Euler tour → RMQ → LCA → khoảng cách.
5. Chọn **một mở rộng dễ minh họa**, chẳng hạn deque cho cửa sổ trượt.

Bạn chưa cần học hết Fenwick hai cây, offline RMQ, RMQ 2D, GPU hay cận dưới cell-probe. **Chúng là phần mở rộng của bộ slide, không phải những lỗ hổng trong nền tảng RMQ bạn vừa học.**

You said:

2 selections

trình bày cho tui 3 nội dung này

Worked for 30s

ChatGPT said:

Mình trình bày theo thứ tự: **prefix-min → chia khối $\sqrt{n}$ → LCA và khoảng cách trên cây**. Các ví dụ bên dưới đều dùng chỉ số bắt đầu từ 1.

1. Vì sao prefix-min không trả lời được RMQ tùy ý?

Trước tiên, nhớ lại vì sao prefix sum hoạt động.

Đặt:

$$S[i] = A[1] + A[2] + \cdots + A[i].$$

Muốn tính tổng đoạn $[l, r]$:

$$\text{sum}(l, r) = S[r] - S[l-1].$$

Ví dụ:

$$A = [3, 5, 2, 7].$$

Ta có:

$$S[4] = 17, \qquad S[2] = 8.$$

Vậy:

$$\text{sum}(3,4) = 17 - 8 = 9.$$

Phép trừ đã **loại bỏ đóng góp của đoạn đứng trước** l.

Với cực tiểu, thử xây:

$$P[i] = \min(A[1], A[2], \ldots, A[i]).$$

Ví dụ:

$$A = [1,5,4] \quad \Rightarrow \quad P = [1,1,1].$$

Giờ hỏi:

$$\min(2,3) = ?$$

Cả $P[3]$ và $P[1]$ đều bằng 1. Nhưng số 1 nằm ngoài đoạn hỏi, và ta không có thao tác nào để "bỏ số 1" rồi tìm lại cực tiểu còn lại từ hai giá trị này.

Prefix-min đã quên các giá trị lớn hơn cực tiểu trước đó.

Để chứng minh chắc chắn, xét hai mảng:

Mảng	Prefix-min	Min đoạn $[2,3]$
$X = [1,5,4]$	$[1,1,1]$	4
$Y = [1,2,4]$	$[1,1,1]$	2

Toàn bộ prefix-min giống hệt nhau, nhưng đáp án khác nhau.

Vậy bất kỳ thuật toán nào **chỉ biết prefix-min** cũng không thể phân biệt hai trường hợp này để trả lời đúng mọi đoạn.

Bạn có thể nói trong seminar:

> "Prefix-min trả lời tốt các đoạn bắt đầu từ đầu mảng. Với đoạn bất kỳ, thông tin về các phần tử lớn hơn cực tiểu trước đó đã bị mất, nên không thể khôi phục đáp án như prefix sum."

---

2. Chia khối $\sqrt{n}$: nằm giữa quét trực tiếp và tiền xử lý nhiều

Ý tưởng là:

> Chia mảng thành các khối. Mỗi khối lưu một giá trị nhỏ nhất. Khi query, khối nào lấy trọn thì đọc cực tiểu đã lưu; hai phần biên thì quét trực tiếp.

Lấy mảng 16 phần tử, chia thành khối dài $4 = \sqrt{16}$:

Khối	Chỉ số	Các phần tử	Min khối
1	$1..4$	$7, 4, 9, 6$	4
2	$5..8$	$3, 8, 5, 10$	3
3	$9..12$	$2, 11, 6, 9$	2
4	$13..16$	$5, 1, 8, 12$	1

Ta chỉ lưu thêm:

$$M = [4, 3, 2, 1].$$

Ví dụ query $[3, 14]$

Chia thành ba phần:

Phần	Cách xử lý	Kết quả
Đuôi khối 1: $A[3..4] = [9, 6]$	Quét hai phần tử	6
Hai khối đầy đủ 2 và 3	Đọc $M[2] = 3,\ M[3] = 2$	2
Đầu khối 4: $A[13..14] = [5, 1]$	Quét hai phần tử	1

Kết quả:

$$\min(6, 3, 2, 1) = 1.$$

Ở giữa, ta đọc **hai giá trị đại diện**, thay vì quét tám phần tử của hai khối.

Nếu hai đầu nằm cùng khối, ta quét trực tiếp đoạn hỏi trong khối đó.

Vì sao chọn kích thước $\sqrt{n}$**?**

Giả sử mỗi khối dài b:

- Hai phần biên cần quét tối đa khoảng $2b$ phần tử.
- Ở giữa có tối đa khoảng n/b khối, mỗi khối đọc một cực tiểu.

Vậy:

$$Q(n) = O\left(b + \frac{n}{b}\right).$$

Có một sự đánh đổi:

Chọn b	Phần biên	Số đại diện phải đọc
Nhỏ	Quét ít	Nhiều khối
Lớn	Quét nhiều	Ít khối

Chọn hai lượng cân bằng:

$$b \approx \frac{n}{b} \quad \Rightarrow \quad b \approx \sqrt{n}.$$

Khi đó:

$$\boxed{Q(n) = O(\sqrt{n})}.$$

Cập nhật một phần tử thế nào?

Ví dụ sửa:

$$A[9] : 2 \longrightarrow 20.$$

Khối 3 trở thành:

$$[20, 11, 6, 9].$$

Cực tiểu cũ là 2, nhưng nó không còn tồn tại. Ta phải quét lại khối 3 để tìm cực tiểu mới là 6.

Chỉ một khối bị ảnh hưởng, nên:

$$\boxed{\text{Cập nhật điểm } O(b) = O(\sqrt{n})}.$$

Nếu giá trị mới nhỏ hơn cực tiểu cũ thì có thể cập nhật nhanh hơn, nhưng **trường hợp xấu nhất** vẫn phải quét lại khối.

Tổng hợp:

Thao tác	Chi phí
Xây các cực tiểu khối	$O(n)$
Query	$O(\sqrt{n})$
Cập nhật điểm	$O(\sqrt{n})$
Bộ nhớ phụ, ngoài mảng A	$O(\sqrt{n})$

Khác gì với chia khối $\Theta(\log n)$ bạn đã học?

Chúng dùng cùng ý tưởng chia đoạn thành hai biên và phần giữa, nhưng xử lý khác nhau:

Thành phần	Chia khối $\sqrt{n}$ cơ bản	Cách tối ưu bạn đã học
Kích thước khối	$\Theta(\sqrt{n})$	$\Theta(\log n)$
Các khối đầy đủ ở giữa	Duyệt từng cực tiểu khối	Sparse Table: $O(1)$
Đoạn trong một khối	Quét trực tiếp	Bảng tra theo loại khối: $O(1)$
Query tổng thể	$O(\sqrt{n})$	$O(1)$

Chỉ đổi kích thước khối thành $\log n$ là chưa đủ. Nếu vẫn duyệt từng cực tiểu khối ở giữa, query sẽ mất:

$$O\left(\log n + \frac{n}{\log n}\right).$$

Sự cải thiện đến từ cả **Sparse Table tầng trên** và **bảng tra nội khối**.

---

3. LCA: binary lifting, Euler tour và khoảng cách trên cây

Xét cây có gốc 1:

LCA của hai đỉnh là **tổ tiên chung nằm sâu nhất**:

$$\mathrm{LCA}(4,5) = 2,$$

$$\mathrm{LCA}(4,6) = 1,$$

$$\mathrm{LCA}(2,5) = 2.$$

Một đỉnh được tính là tổ tiên của chính nó.

Cách A: Binary lifting — nhảy lên theo lũy thừa của 2

Nếu mỗi lần chỉ leo lên cha một bước, cây dạng dây có thể khiến một query mất $O(n)$.

Binary lifting tiền xử lý để có thể nhảy:

$$1,\ 2,\ 4,\ 8, \ldots$$

cạnh trong một bước tra bảng.

Đặt:

$$up[k][v] = \text{tổ tiên cách } v \text{ đúng } 2^k \text{ cạnh}.$$

Khi đã lên đến gốc thì quy ước tiếp tục ở gốc:

$$up[k][1] = 1.$$

Công thức xây:

$$up[0][v] = parent[v],$$

$$\boxed{up[k][v] = up[k-1][up[k-1][v]]}.$$

Nghĩa là: **nhảy 2^{k-1} cạnh hai lần**, tổng cộng 2^k cạnh.

Trên cây ví dụ:

Đỉnh v	Độ sâu	$up[0][v]$: lên 1 cạnh	$up[1][v]$: lên 2 cạnh
1	0	1	1
2	1	1	1
3	1	1	1
4	2	2	1
5	2	2	1
6	2	3	1
7	2	3	1

Một truy vấn LCA có hai giai đoạn.

Giai đoạn 1: đưa hai đỉnh về cùng độ sâu.

Ví dụ tìm $\mathrm{LCA}(4, 3)$:

- Đỉnh 4 sâu 2.
- Đỉnh 3 sâu 1.
- Nâng 4 lên một cạnh, thành đỉnh 2.

Giờ bài toán còn:

$$\mathrm{LCA}(2, 3).$$

Nếu chênh lệch độ sâu lớn, phân tích nó thành tổng các lũy thừa của 2. Chẳng hạn chênh 13 cạnh:

$$13 = 8 + 4 + 1,$$

nên chỉ cần ba lần nhảy.

Nếu sau khi cân bằng mà hai đỉnh trùng nhau, đó chính là LCA. Ví dụ 2 và 5: nâng 5 lên 2, trả về 2.

Giai đoạn 2: nâng đồng thời, nhưng dừng ngay dưới LCA.

Ví dụ tìm $\mathrm{LCA}(4, 6)$. Hai đỉnh đã cùng độ sâu.

Xét bước nhảy từ lớn xuống nhỏ:

- Nhảy lên hai cạnh: cả hai cùng đến 1. Không nhảy, vì sẽ chạm LCA hoặc vượt qua nó.
- Nhảy lên một cạnh: $4 \to 2,\ 6 \to 3$. Hai tổ tiên khác nhau, nên thực hiện bước nhảy.

Giờ đứng ở 2 và 3. Cha của chúng đều là 1, nên:

$$\mathrm{LCA}(4,6) = 1.$$

Quy tắc là:

> Khi hai tổ tiên ở cùng mức nhảy còn khác nhau, ta có thể nâng cả hai lên đó mà vẫn ở dưới LCA.

Có $O(\log n)$ mức:

$$\boxed{\text{Build } O(n \log n), \quad \text{Query } O(\log n), \quad \text{Bộ nhớ } O(n \log n).}$$

Cách B: Euler tour + RMQ — tìm đỉnh nông nhất giữa hai lần xuất hiện

Bạn đã học cách này; mình nối lại bằng cùng ví dụ.

Ghi đỉnh khi bắt đầu thăm và mỗi lần quay về từ một cây con:

$$E = [1, 2, 4, 2, 5, 2, 1, 3, 6, 3, 7, 3, 1].$$

Ghi độ sâu tương ứng:

$$D = [0, 1, 2, 1, 2, 1, 0, 1, 2, 1, 2, 1, 0].$$

Lưu thêm:

$$first[v] = \text{vị trí xuất hiện đầu tiên của } v \text{ trong } E.$$

Muốn tìm $\mathrm{LCA}(4,6)$:

$$first[4] = 3, \qquad first[6] = 9.$$

Xét đoạn:

$$E[3..9] = [4, 2, 5, 2, 1, 3, 6].$$

Độ sâu:

$$D[3..9] = [2, 1, 2, 1, 0, 1, 2].$$

Độ sâu nhỏ nhất là 0, ứng với đỉnh 1. Vậy:

$$\mathrm{LCA}(4, 6) = 1.$$

Công thức, sau khi sắp hai vị trí đầu mút theo thứ tự tăng dần:

$$\boxed{\mathrm{LCA}(u, v) = E[\mathrm{RMQ}_D(first[u], first[v])]}.$$

Nếu dùng Sparse Table:

$$\text{Build } O(n \log n), \qquad \text{Query } O(1).$$

Nếu dùng thuật toán ± 1 RMQ tối ưu:

$$\text{Build } O(n), \qquad \text{Query } O(1).$$

Từ LCA suy ra khoảng cách

Gọi $depth[v]$ là số cạnh từ gốc đến v, và:

$$w = \mathrm{LCA}(u, v).$$

Đường đi từ u đến v gồm:

- Từ u lên w: $depth[u] - depth[w]$ cạnh.
- Từ w xuống v: $depth[v] - depth[w]$ cạnh.

Cộng lại:

$$\boxed{dist(u, v) = depth[u] + depth[v] - 2depth[w].}$$

Ví dụ:

$$dist(4, 6) = 2 + 2 - 2 \cdot 0 = 4,$$

đúng với đường:

$$4 \to 2 \to 1 \to 3 \to 6.$$

Còn:

$$dist(4,5) = 2 + 2 - 2 \cdot 1 = 2.$$

Với cây có trọng số cạnh, thay `depth` trong công thức khoảng cách bằng **tổng trọng số từ gốc**. Việc tìm LCA vẫn dựa trên cấu trúc cây.

So sánh để trình bày trong seminar

Phương pháp LCA	Build	Query	Điểm phù hợp
Leo cha từng bước	$O(n)$	$O(h)$	Ít query, cài đặt đơn giản
Binary lifting	$O(n \log n)$	$O(\log n)$	Cần thêm thao tác tìm tổ tiên thứ k
Euler tour + Sparse Table	$O(n \log n)$	$O(1)$	Nhiều query LCA trên cây tĩnh
Euler tour + ± 1 RMQ tối ưu	$O(n)$	$O(1)$	Trình bày kết quả tối ưu tiệm cận

Ở phần ứng dụng RMQ, bạn có thể chạy tay **Euler tour → LCA → khoảng cách** như trên. Binary lifting đóng vai trò phương pháp đối chiếu, giúp giải thích RMQ mang lại lợi ích gì.

You said:

1 selection

trình bày cho tui nội dung này

Worked for 1m 19s

ChatGPT said:

Hai cách này cùng tận dụng một điều kiện:

> **Ta biết trước toàn bộ truy vấn, nên có thể tổ chức công việc theo đúng những truy vấn cần trả lời.**

Nhưng chúng tối ưu theo hai hướng khác nhau:

- **Arpa's trick:** sắp thứ tự xử lý truy vấn để quét mảng một lần.
- **Co mảng:** thu nhỏ mảng dựa trên các đầu mút truy vấn, rồi giải RMQ trên mảng nhỏ.

1. “Offline” nghĩa là gì?

Giả sử có ba truy vấn:

$$Q_1 = [4, 7], \qquad Q_2 = [2, 4], \qquad Q_3 = [3, 4].$$

Nếu xử lý **online**, ta phải trả lời theo lúc truy vấn được đưa đến, không dựa vào các truy vấn tương lai.

Nếu xử lý **offline**, ta nhận được cả ba trước khi bắt đầu. Ta có thể đổi thứ tự:

$$Q_2,\ Q_3,\ Q_1,$$

tính đáp án, rồi xuất lại theo thứ tự ban đầu.

Offline không có nghĩa là mảng được cập nhật. Hai cách dưới đây đều xét mảng tĩnh.

---

2. Arpa’s trick: trả lời truy vấn khi quét đến đầu phải

Ý tưởng:

> Quét vị trí r từ trái sang phải. Khi đang đứng tại r, trả lời tất cả truy vấn có đầu phải bằng r.

Ta kết hợp:

- **Stack đơn điệu:** giữ những vị trí còn có thể làm cực tiểu.
- **DSU:** nối một vị trí đến một ứng viên tốt hơn, giúp nhảy nhanh đến đáp án.

Đây là cách xử lý được mô tả trong tài liệu CP-Algorithms và bài viết của Arpa. Algorithms for ... +1

A. Vì sao được loại một phần tử khỏi stack?

Giả sử đang xét vị trí r, và đỉnh stack là vị trí p, với:

$$p < r, \qquad A[p] > A[r].$$

Đối với những truy vấn có đầu phải từ r trở đi:

- Nếu đoạn hỏi chứa p, nó cũng chứa r.
- $A[r]$ nhỏ hơn $A[p]$.

Vậy p không thể là cực tiểu của những đoạn đó nữa. Ta có thể nối:

$$parent[p] = r.$$

Ý nghĩa:

> “Nếu tìm cực tiểu mà đến vị trí p, hãy đi tiếp sang r, vì ở đó có một phần tử tốt hơn.”

Lặp lại việc pop cho đến khi stack không còn phần tử lớn hơn $A[r]$, rồi push r.

B. Chạy tay một ví dụ

Xét:

$$A = [5, 2, 6, 3, 1, 7, 4, 8].$$

Ta có các truy vấn:

$$Q_1 = [4, 7], \qquad Q_2 = [2, 4], \qquad Q_3 = [3, 4].$$

Stack lưu **chỉ số**, nhưng mình ghi cả giá trị để dễ theo dõi.

Đang xét	Thao tác	Stack sau xử lý
$r = 1,\ A[1] = 5$	Push 1	$1 : 5$
$r = 2,\ A[2] = 2$	Pop 1, nối $1 \to 2$; push 2	$2 : 2$
$r = 3,\ A[3] = 6$	Push 3	$2 : 2,\ 3 : 6$
$r = 4,\ A[4] = 3$	Pop 3, nối $3 \to 4$; push 4	$2 : 2,\ 4 : 3$

Tại $r = 4$, trả lời hai truy vấn kết thúc ở 4.

Với $[2, 4]$:

$$find(2) = 2.$$

Vậy đáp án là vị trí 2, giá trị 2.

Với $[3, 4]$, ta đã có liên kết:

$$3 \to 4.$$

Nên:

$$find(3) = 4.$$

Đáp án là vị trí 4, giá trị 3.

Tiếp tục quét:

Đang xét	Thao tác	Stack sau xử lý
$r = 5,\ A[5] = 1$	Pop 4 và 2; nối $4 \to 5,\ 2 \to 5$; push 5	$5 : 1$
$r = 6,\ A[6] = 7$	Push 6	$5 : 1,\ 6 : 7$
$r = 7,\ A[7] = 4$	Pop 6, nối $6 \to 7$; push 7	$5 : 1,\ 7 : 4$

Tại $r = 7$, trả lời $[4, 7]$:

$$4 \to 5.$$

Do đó:

$$find(4) = 5,$$

đúng vì:

$$A[4..7] = [3, 1, 7, 4].$$

C. DSU làm gì ở đây?

Trong cách biểu diễn đơn giản này, `find(l)` đi theo các liên kết để tìm đại diện cực tiểu.

Ví dụ có đường:

$$3 \to 4 \to 5.$$

Sau khi tìm thấy 5, DSU có thể nén đường đi, cho 3 trỏ thẳng đến 5. Lần sau tìm từ 3 sẽ nhanh hơn.

Điểm cần nhớ:

$$\boxed{\text{Sau khi xử lý vị trí } r, \quad find(l) \text{ cho vị trí cực tiểu của } A[l..r].}$$

Nếu chỉ pop khi `>` thì giữ vị trí cực tiểu bên trái nhất khi có giá trị trùng nhau.

D. Vì sao phải trả lời đúng lúc r?

Sau khi quét qua r, những phần tử mới có thể làm thay đổi đại diện.

Trong ví dụ, tại $r = 4$:

$$find(3) = 4.$$

Nhưng sau khi xử lý vị trí 5:

$$find(3) = 5.$$

Kết quả mới đúng cho $[3, 5]$, nhưng sai nếu dùng để trả lời $[3, 4]$.

Vì vậy, phải gom truy vấn theo đầu phải và trả lời ngay ở thời điểm tương ứng.

E. Độ phức tạp

Mỗi phần tử được push và pop tối đa một lần. Kết hợp DSU tối ưu, tổng thời gian thường được phát biểu:

$$\boxed{O((n+q)\alpha(n))}$$

và bộ nhớ:

$$\boxed{O(n+q)}.$$

$\alpha(n)$ là hàm nghịch đảo Ackermann, tăng cực chậm; đây là chi phí khấu hao, không phải bảo đảm từng query luôn chỉ làm một bước. Có thể gom truy vấn vào các nhóm theo r, tránh phải sort bằng phép so sánh. Codeforces +1

---

3. Co mảng: chỉ giữ thông tin mà lô truy vấn có thể phân biệt

Cách này đặc biệt hữu ích khi:

$$q \ll n.$$

Ví dụ mảng có hàng triệu phần tử nhưng chỉ có vài trăm truy vấn. Ta muốn phần cấu trúc RMQ được xây trên kích thước phụ thuộc vào q.

Ý tưởng của Alzamel và cộng sự:

> Giữ riêng các đầu mút truy vấn. Mỗi đoạn liên tiếp không chứa đầu mút nào được thay bằng một phần tử cực tiểu. arxiv.org

A. Ví dụ cụ thể

Dùng mảng:

$$A = [7, 4, 9, 6, 3, 8, 5, 10, 2, 11, 6, 9, 5, 1, 8, 12].$$

Chỉ có hai truy vấn:

$$Q_1 = [3, 12], \qquad Q_2 = [6, 15].$$

Các đầu mút cần đánh dấu là:

$$3,\ 6,\ 12,\ 15.$$

Chia mảng theo những điểm đó:

Phần mảng gốc	Cách xử lý	Giá trị giữ lại	Chỉ số gốc
$A[1..2]$	Bỏ: ngoài mọi truy vấn	—	—
$A[3]$	Giữ đầu mút	9	3
$A[4..5] = [6, 3]$	Co thành cực tiểu	3	5
$A[6]$	Giữ đầu mút	8	6
$A[7..11] = [5, 10, 2, 11, 6]$	Co thành cực tiểu	2	9
$A[12]$	Giữ đầu mút	9	12
$A[13..14] = [5, 1]$	Co thành cực tiểu	1	14
$A[15]$	Giữ đầu mút	8	15
$A[16]$	Bỏ: ngoài mọi truy vấn	—	—

Mảng mới:

$$B = [9, 3, 8, 2, 9, 1, 8].$$

Lưu thêm ánh xạ về chỉ số gốc:

$$F = [3, 5, 6, 9, 12, 14, 15].$$

B. Chuyển truy vấn sang mảng mới

Truy vấn cũ:

$$Q_1 = [3, 12]$$

trở thành:

$$Q_1' = [1, 5].$$

Vì vị trí gốc 3 được đưa vào $B[1]$, còn vị trí gốc 12 được đưa vào $B[5]$.

Ta có:

$$B[1..5] = [9, 3, 8, 2, 9].$$

Cực tiểu nằm tại $B[4]$. Đổi về chỉ số gốc:

$$F[4] = 9.$$

Vậy:

$$\mathrm{RMQ}_A(3, 12) = 9.$$

Tương tự:

$$Q_2 = [6, 15] \longrightarrow Q_2' = [3, 7].$$

Cực tiểu nằm tại $B[6] = 1$, tương ứng:

$$F[6] = 14.$$

C. Vì sao co như vậy không làm sai đáp án?

Xét đoạn bị co:

$$A[7..11].$$

Không có truy vấn nào bắt đầu hoặc kết thúc bên trong đoạn này.

Vì vậy, mỗi truy vấn trong lô chỉ có thể:

- Lấy trọn đoạn đó.

- Hoặc không lấy đoạn đó.

Không truy vấn nào chỉ lấy một phần của nó.

Nếu lấy trọn, ta chỉ cần biết:

$$\min A[7..11] = 2.$$

Đó là lý do thay cả đoạn bằng một cực tiểu vẫn giữ đúng đáp án.

Chỉ giữ các đầu mút thì chưa đủ: trong ví dụ, đáp án của $[3, 12]$ nằm ở vị trí 9, bên trong một đoạn bị co. Ta phải giữ cực tiểu của đoạn ấy.

D. Mảng mới dài bao nhiêu?

Có tối đa $2q$ đầu mút khác nhau.

Giữa chúng có tối đa $2q - 1$ đoạn trống. Mỗi đoạn trống giữ một đại diện, mỗi đầu mút giữ một phần tử.

Sau khi bỏ hai phần ngoài cùng không cần thiết:

$$|B| \leq 4q - 1 = O(q).$$

Lưu ý: đây là **co theo đầu mút**, không phải chia thành các khối đều nhau như $\sqrt{n}$.

---

4. Co xong thì dùng thuật toán nào? Chi phí ra sao?

Co mảng là một **bước thu nhỏ đầu vào**. Sau đó vẫn phải giải RMQ trên B.

Cách xử lý sau khi co	Tổng thời gian của bản cài đặt đơn giản	Bộ nhớ phụ
Sort đầu mút, co mảng, xây Sparse Table thông thường	$O(n + q \log q)$	$O(q \log q)$
Các biến thể được tối ưu trong paper	Có thể đạt $O(n + q)$	$O(q)$

Không thể tự động kết luận "co xuống $O(q)$ phần tử thì toàn bộ thuật toán chỉ mất $O(q)$".

Ta vẫn phải đọc các phần mảng cần tìm cực tiểu; trong trường hợp xấu nhất là $O(n)$. Paper còn dùng kỹ thuật đánh dấu trên đầu vào có thể ghi sửa để tránh chi phí sort; đồng thời tối ưu bộ giải trên mảng đã co. arxiv.org

Một mở rộng thực dụng khác là xây Sparse Table theo khối trên mảng đã co, được nghiên cứu trong *Faster Batched Range Minimum Queries*. arxiv.org

---

5. Phân biệt hai cách để đưa vào seminar

Tiêu chí	Arpa's trick	Co mảng
Tận dụng việc biết trước query để làm gì?	Gom theo đầu phải	Xác định những vùng được gộp
Công cụ chính	Stack đơn điệu + DSU	Đầu mút + cực tiểu đại diện + bộ giải RMQ
Có thu nhỏ đầu vào xuống $O(q)$ không?	Bản thông thường không	Có
Thế mạnh	Xử lý cả lô bằng một lượt quét	Giảm cấu trúc phụ khi $q \ll n$
Trả lời query mới tùy ý sau khi làm xong?	Không bằng trạng thái quét cuối	Không bảo đảm nếu đầu mút mới cắt bên trong vùng đã co

Với phần mở rộng seminar, bạn có thể trình bày **ví dụ co mảng** trước: nó dễ nhìn thấy lợi ích. Arpa's trick phù hợp nếu bạn muốn giải thích thêm cách **stack đơn điệu kết hợp DSU** biến việc quét mảng thành lời giải cho cả lô truy vấn.

You said:

cái này có cần phải đưa vào seminar không vậy

ChatGPT said:

Không bắt buộc. Với seminar 30 phút của bạn, mình nghĩ nên để Offline RMQ ở phần đọc thêm hoặc slide phụ.

Bạn đã có đủ nội dung chính: Sparse Table, Cartesian tree, chia khối + bảng tra, ứng dụng LCA, so sánh CTDL và benchmark. Những phần này cần thời gian cho ví dụ và giải thích tính đúng.

Hai nội dung vừa học có thể xử lý như sau:

Nội dung	Nên đưa vào đâu?
Arpa's trick	Slide phụ; muốn giải thích rõ phải giới thiệu thêm DSU
Co mảng khi $q \ll n$	Có thể nhắc 20–30 giây trong phần mở rộng

Nếu muốn giữ một ý, **co mảng dễ minh họa hơn**. Chỉ cần nói:

> “Khi biết trước toàn bộ truy vấn và số truy vấn rất nhỏ so với kích thước mảng, ta có thể giữ các đầu mút, thay mỗi vùng giữa chúng bằng cực tiểu đại diện, rồi giải RMQ trên mảng chỉ còn $O(q)$ phần tử.”

Không cần trình bày thuật toán chi tiết hay chứng minh. Phần mở rộng của seminar cũng không nhất thiết phải có nhiều thuật toán: **giải thích chắc một hướng mở rộng, kèm ví dụ và điều kiện áp dụng, là đủ có giá trị.**

You said:

Fenwick 2 cây cho RMQ?

Worked for 49s

ChatGPT said:

Fenwick hai cây cho RMQ là một biến thể hỗ trợ cả truy vấn min đoạn và cập nhật điểm. Với seminar của bạn, chỉ cần nhắc trong bảng so sánh, không cần trình bày chi tiết.

Nó là một hướng thay thế cho Segment Tree, được Dima–Ceterchi trình bày năm 2015. ioi.te.lv

1. Vì sao cần hai cây?

Fenwick thông thường lưu tổng trên các đoạn, rồi dùng phép trừ để lấy tổng đoạn bất kỳ.

Với min, ta không có phép “trừ bỏ cực tiểu”. Vì vậy, biến thể này lưu các đoạn theo **hai hướng**.

Đặt:

$$lowbit(i) = i \,\&\, (-i).$$

Hai bảng lưu:

$$L[i] = \min A[i - lowbit(i) + 1..i],$$

$$R[i] = \min A[i..\min(n, i + lowbit(i) - 1)].$$

Hiểu đơn giản:

- $L[i]$: cực tiểu của một đoạn **kết thúc tại** i.
- $R[i]$: cực tiểu của một đoạn **bắt đầu tại** i.

2. Ví dụ truy vấn

Mình dùng lại mảng:

$$A = [5, 2, 6, 3, 1, 7, 4, 8].$$

Hỏi:

$$\min A[3..7].$$

Hai hướng Fenwick cho phép ghép các phần:

Phần	Lấy từ đâu?	Giá trị
$[3, 3]$	$R[3]$	6
$[4, 4]$	$A[4]$, điểm gặp nhau	3
$[5, 6]$	$L[6]$, vì $lowbit(6) = 2$	1
$[7, 7]$	$L[7]$	4

Vậy:

$$\min A[3..7] = \min(6, 3, 1, 4) = 1.$$

Nó ghép các đoạn nằm trong vùng hỏi, không lấy hai prefix-min rồi tìm cách trừ nhau. Vì thế không mâu thuẫn với phần prefix-min bạn vừa học.

3. Cập nhật có đơn giản như Fenwick tổng không?

Không. Giả sử:

$$A[5] : 1 \longrightarrow 10.$$

Trước đó:

$$L[6] = \min(A[5], A[6]) = \min(1, 7) = 1.$$

Sau cập nhật phải thành:

$$L[6] = \min(10, 7) = 7.$$

Nếu chỉ viết:

```
L[6] = min(L[6], 10);
```

thì vẫn nhận 1, tức là sai.

Khi cực tiểu cũ bị tăng lên, phải tính lại các đoạn bị ảnh hưởng. Paper phối hợp hai cây để tái sử dụng kết quả, công bố chi phí cập nhật $O(\log n)$ **khấu hao**; cách tính lại riêng từng nút bằng query có thể thành $O(\log^2 n)$. ioi.te.lv

4. Đưa vào seminar thế nào?

Bạn có thể dùng một dòng so sánh:

Cấu trúc	Build	Query	Cập nhật điểm	Bộ nhớ
Fenwick hai cây	$O(n)$	$O(\log n)$	$O(\log n)$ khấu hao, theo paper	$O(n)$
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$ xấu nhất	$O(n)$

Phần chính vẫn nên dùng Segment Tree để minh họa RMQ động. Fenwick hai cây là một mở rộng đáng biết, nhưng thêm phần dựng và cập nhật hai cây sẽ tốn thời gian mà chưa giúp nhiều cho mạch trình bày RMQ → Sparse Table → Fischer–Heun của bạn.

You said:

Disjoint Sparse Table, Sqrt Tree, RMQ hai chiều, Cận dưới cell-probe, RMQ động,RMQ song song, GPU, ray tracing, các phần này có cần đề cập ko

You stopped after 0s

You said:

Continue

ChatGPT said:

Không cần đưa hết. Với seminar khoảng 30 phút, bạn nên giữ RMQ động để so sánh, chọn thêm một mở rộng dễ minh họa; các hướng chuyên sâu có thể bỏ.

Nội dung	Mức độ nên đưa	Cách trình bày phù hợp
RMQ động	**Nên có**	Giải thích khi có cập nhật thì chọn Segment Tree; so sánh với Sparse Table
Disjoint Sparse Table	Nhắc ngắn, tùy chọn	Mở rộng từ phép min sang những phép kết hợp không lũy đẳng

Nội dung	Mức độ nên đưa	Cách trình bày phù hợp
Sqrt Tree	Có thể bỏ	Thêm một CTDL khá phức tạp, dễ trùng vai trò so sánh với các cách đã có
RMQ hai chiều	Một lựa chọn tốt cho phần mở rộng	Ví dụ tìm min trong hình chữ nhật của ma trận
Cận dưới cell-probe	Bỏ khỏi phần chính	Cần giải thích mô hình tính toán và điều kiện của cận dưới
RMQ song song	Không bắt buộc	Chỉ nhắc như hướng nghiên cứu nếu còn thời gian
GPU, ray tracing	Không bắt buộc	Để tài liệu đọc thêm; giải thích thuyết phục cần cả kiến thức phần cứng và benchmark

1. RMQ động: nên giữ, nhưng không cần học một thuật toán mới quá sâu

Bạn chỉ cần đưa một tình huống:

$$A = [5, 2, 6, 3, 1, 7].$$

Ban đầu:

$$\min A[2..5] = 1.$$

Sau khi cập nhật $A[5] = 10$:

$$\min A[2..5] = 2.$$

Từ đó giải thích: thông tin tiền xử lý có thể hết đúng khi mảng thay đổi. Segment Tree hỗ trợ cả query và cập nhật điểm trong $O(\log n)$.

Phân biệt tĩnh–động và so sánh CTDL là đủ; không cần trình bày cận dưới của RMQ động.

2. Disjoint Sparse Table: chỉ cần nêu giới hạn của Sparse Table thông thường

Bạn đã giải thích truy vấn hai đoạn chồng lấn nhờ:

$$\min(x, x) = x.$$

Có thể thêm một câu:

> “Với phép kết hợp không lũy đẳng như tổng, công thức hai đoạn chồng lấn không còn đúng. Disjoint Sparse Table dùng các đoạn không giao nhau để khắc phục.”

Không cần đưa code hay cách xây. Với tổng, prefix sum vẫn là phương án đơn giản hơn; Disjoint Sparse Table đáng chú ý vì áp dụng cho **phép kết hợp tổng quát**.

3. RMQ hai chiều: chọn nếu muốn một mở rộng có ví dụ trực quan

Chuyển câu hỏi từ:

> “Min trong đoạn $[l, r]$ là gì?”

sang:

> “Min trong hình chữ nhật của ma trận là gì?”

Nếu trình bày Sparse Table 2D, minh họa một hình chữ nhật được phủ bằng **bốn hình chữ nhật đã tiền xử lý** là đủ. Không cần đi vào bộ nhớ nén hay cận dưới hai chiều.

Mình đề xuất phần mở rộng chỉ gồm RMQ động và RMQ hai chiều. Disjoint Sparse Table để một câu hoặc slide phụ; Sqrt Tree, cell-probe, song song, GPU và ray tracing để tài liệu tham khảo. Bạn sẽ có thời gian giải thích rõ ví dụ và benchmark, thay vì phải giới thiệu nhiều tên thuật toán mà chưa làm rõ cơ chế.